ProjectPulse

Academic Project Governance, Originality and Milestone Risk Monitor

Course	DBMS Experiential Learning, Level 3 Project
Instructor	Dr. Nagasundari S
Team	Chiranthan and Aadish
Domain	Education Technology / Academic Project Management
Databases	PostgreSQL (SQL) · MongoDB (NoSQL) · PostgreSQL with pgvector (vector)
Application	Next.js (App Router) with Route Handlers
Date	26 October 2026

Contents

1. Problem statement and motivation
2. Objectives
3. Scope
4. Functional requirements
5. ER diagram
6. Relational schema
7. Database choice and justification
8. Data dictionary
9. SQL implementation
10. NoSQL implementation (MongoDB)
11. Vector implementation (pgvector)
12. Important queries
13. System architecture and security design
14. Application screenshots
15. Testing results
16. Conclusion and future enhancements
17. Team contributions and repository
18. References
19. Appendix A: complete data dictionary
20. Appendix B: how to run the project

1. Problem statement and motivation

In colleges, student teams working on mini-projects, capstones, dissertations and research assignments often face late or missed milestones, unclear task ownership, unequal individual contribution, incomplete documentation, unresolved faculty feedback, and project proposals that substantially overlap with other teams. Faculty guides commonly discover these issues only near final submission, when there is little time left to intervene, correct the scope, recover missed work, or assess individual participation fairly.

ProjectPulse is a web-based academic project-governance platform. Students, faculty guides, reviewers and administrators manage the complete project lifecycle in one place: proposal, team formation, guide allocation, milestone planning, task allocation, evidence submission, document-version tracking, feedback resolution, approval workflows, contribution evidence, and delivery-risk monitoring. The system also uses **semantic similarity search** to flag potentially overlapping project proposals for faculty review.

The platform gives explainable progress, contribution and risk indicators based on recorded evidence (verified task completion, document uploads, feedback resolution, milestone status and deadlines). It supports faculty judgement; it never grades automatically and never declares plagiarism.

Why it matters

- A faculty member supervises many teams at once and needs a single dashboard to see which projects are late or at risk, *before* the final week.
- A final report alone is weak evidence of who did what, how the work evolved, or whether feedback was acted on.
- Teams need a clear, shared way to plan work, split responsibilities, submit evidence and answer guide feedback.
- An institution needs an early signal when two proposals describe the same idea, before both teams spend weeks building it.
- Governance and fairness improve while humans keep the final say on grades and originality.

2. Objectives

1. Provide a structured workflow from proposal approval to final evaluation.
2. Track teams, guides, milestones, tasks, deadlines, deliverables, document versions, reviews and feedback.

3. Keep evidence-based contribution records for every team member.
4. Calculate **explainable** delivery-risk levels from overdue work, missing evidence, unresolved feedback, inactivity and contribution imbalance.
5. Offer role-based dashboards for students, faculty guides, reviewers and administrators.
6. Use semantic similarity search to retrieve and flag conceptually similar proposals for faculty review.
7. Demonstrate practical SQL, NoSQL, vector-database and advanced DBMS concepts through a working Next.js application.

3. Scope

Included: proposal approval with immutable history, role-based access, team and guide allocation, milestone and task tracking, deliverable submission with version history and file upload, faculty feedback workflow with discussion threads, contribution evidence and peer evaluation, risk analytics, dashboards and filtered reports with CSV export, document storage, semantic proposal-similarity screening, an audit log, and an administration area (users, roles, departments, courses, project templates, allocations).

Out of scope (future work): automated plagiarism verdicts, automated final grading, mandatory GitHub OAuth sync (commit references are entered manually), OCR of scanned documents, WhatsApp / e-mail delivery, and machine-learning risk prediction. In-app notifications and rate limiting were optional (P2) items and were not built (see section 16).

4. Functional requirements

All P0 (must) and P1 (should) requirements of the specification are implemented and covered by automated tests; of the P2 (nice-to-have) items, CSV export and free-text semantic search are done.

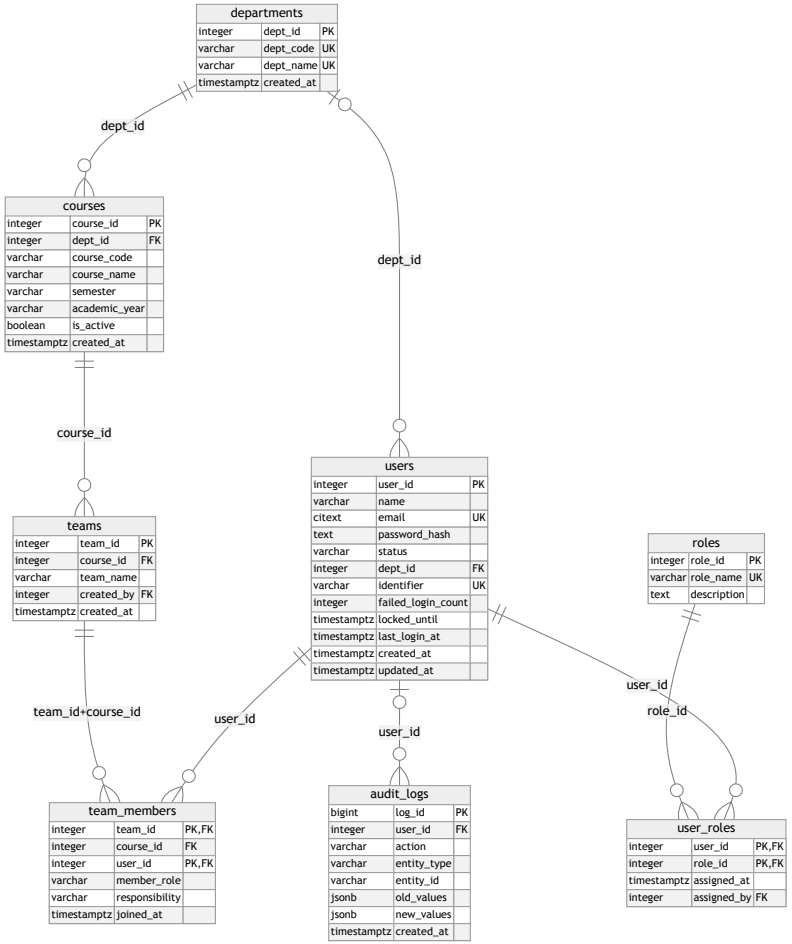
Area	Requirement	Status
Authentication	Email + password with salted bcrypt hashes; JWT in an HttpOnly cookie; logout and expiry (AUTH-01, 04)	Done
	Role-based access for Student, Faculty Guide, Reviewer, Admin in API and UI; role switcher for multi-role users (AUTH-02, 03)	Done
	Lockout after 5 failed logins for 15 minutes; change password; admin can suspend a user (AUTH-05, 06)	Done
Proposals	Draft, edit, submit, approve / reject / request revision with comments; immutable decision history; resubmission (PROP-01 to 06)	Done
	"Check originality" hint for students that reveals nothing about other teams (PROP-07)	Done
Teams	Create teams, add members, responsibilities, one leader; one team per student per course (database-enforced); guide and reviewer allocation; guide cannot be on the team (TEAM-01 to 04)	Done
Milestones and tasks	Dated, weighted milestones (total at most 100), templates, tasks with owner and collaborators, planned and actual dates, controlled workflow with guide verification, approval, Kanban board and timeline (MS-01 to 08)	Done
Deliverables	File upload or link, version numbers, metadata in PostgreSQL, flexible document in MongoDB, **atomic milestone submission**, mandatory-type detection (DEL-01 to 05)	Done
Review and feedback	Reviews with rubric scores, feedback items assigned to one student, respond / resolve / verify / reopen, MongoDB discussion thread (FB-01 to 04)	Done
Contribution	Evidence created automatically from verified tasks, uploads and feedback; manual commit references; peer evaluation; guide verification; student sees only their own record (CON-01 to 05)	Done
Risk	Low / moderate / high / critical with reasons, stored history, overdue-milestone trigger, sweep job (button and script) (RISK-01 to 04)	Done
Similarity	Embed on submit, top-k above a threshold with metadata filter, matching text and score for faculty, saved decision, no automatic verdict, free-text search (SIM-01 to 05)	Done
Dashboards and reports	Role dashboards; delayed teams, unapproved milestones, unresolved feedback, contribution imbalance, high-risk projects, missing deliverables; filters, sorting, CSV (DASH, RPT)	Done
Administration	Users, roles, departments, courses, templates, allocations, system reports; audit-log viewer (ADM-01, AUD-01)	Done
Not built (P2)	In-app notifications (NOT-01), login/similarity rate limiting, a full Content-Security-Policy	Future work

5. ER diagram

The diagrams below are **generated from the live database catalogue** (`npm run docs:data-dictionary`): 32 tables and 63 foreign-key relationships. One picture of all tables would be unreadable, so the schema is drawn as five connected domain diagrams. Inside a domain every attribute is shown with its type and its key marks (PK, FK, UK); a table drawn with only its key is referenced from another domain. A line with a circle at the parent end is an optional relationship (nullable foreign key); the crow's foot is the "many" side. "Who did it" columns such as `created_by`, `approved_by` and `verified_by` (all foreign keys to `users`) are marked FK inside the boxes but not drawn as lines, so the pictures stay readable; all of them are listed in Appendix A. The same diagrams are in `docs/er_*.mmd`, and `docs/er_diagram.mmd` holds the whole schema in one file.

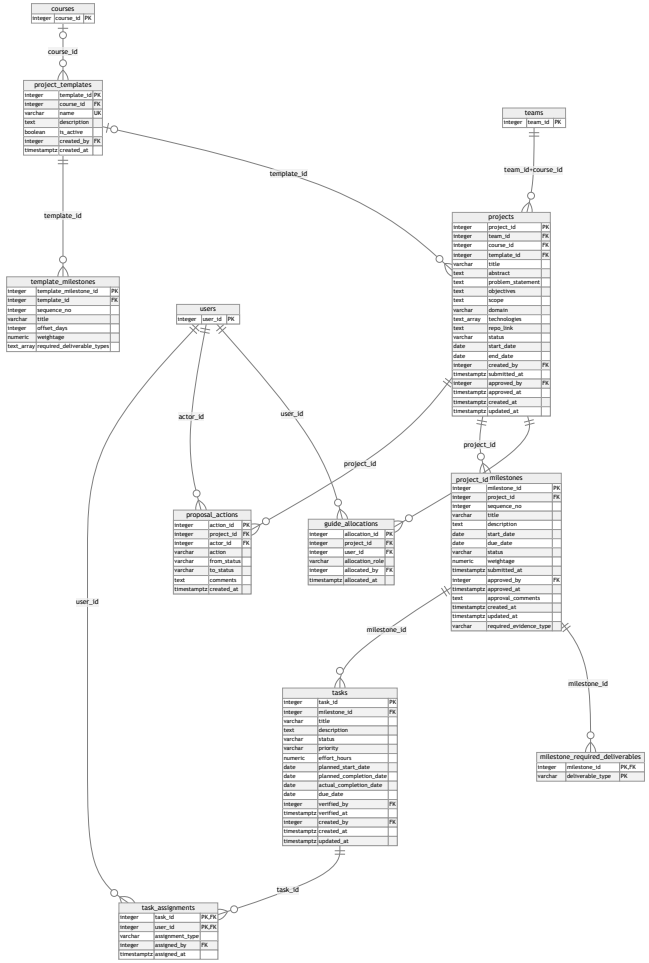
ER 1. Identity, academic context and teams

Who the users are, the departments and courses they belong to, and how students form teams (one team per student per course).



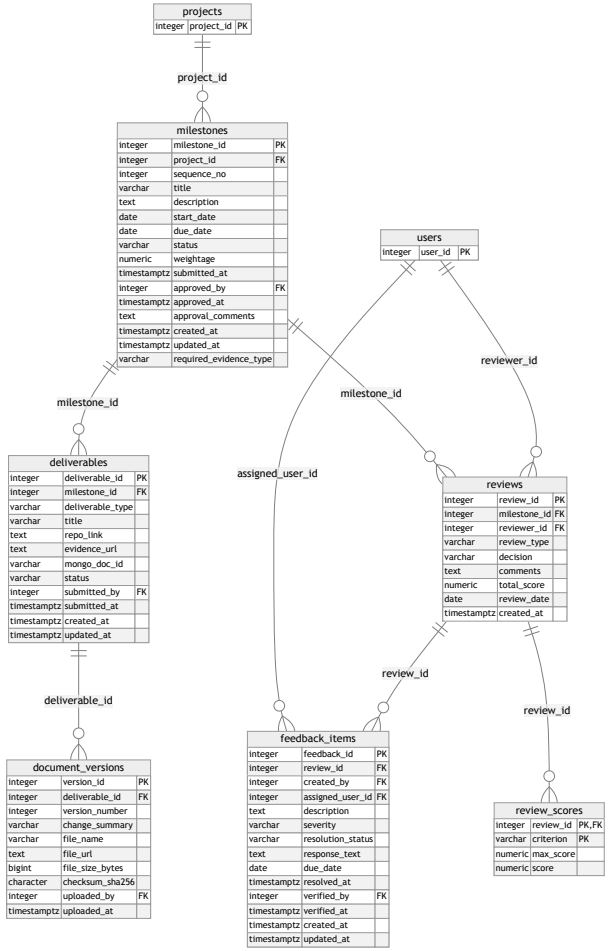
ER 2. Projects, allocation and planning

A team owns one active project; guides and reviewers are allocated; milestones and tasks plan the work; templates seed the plan.



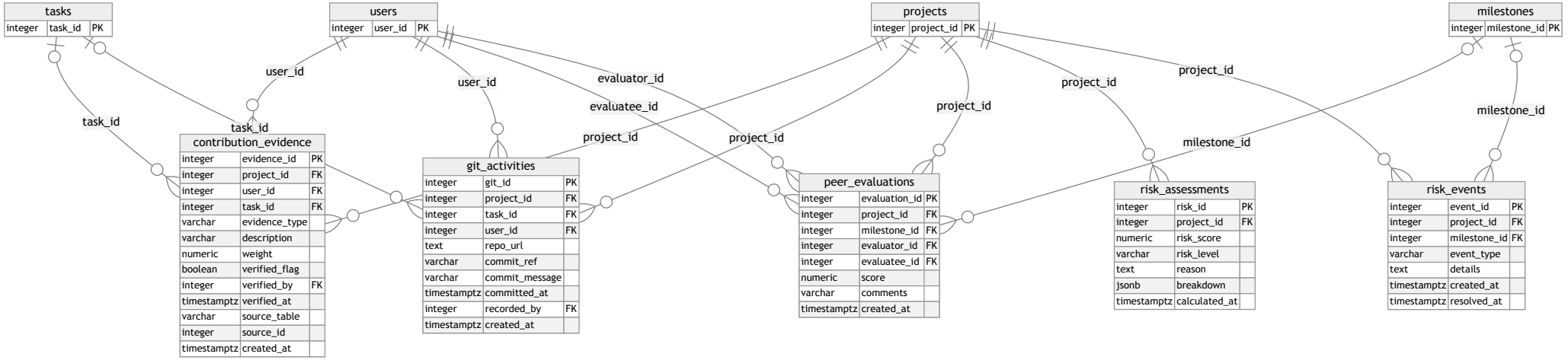
ER 3. Deliverables, review and feedback

Submissions with versions, milestone reviews with rubric scores, and trackable feedback assigned to a student.



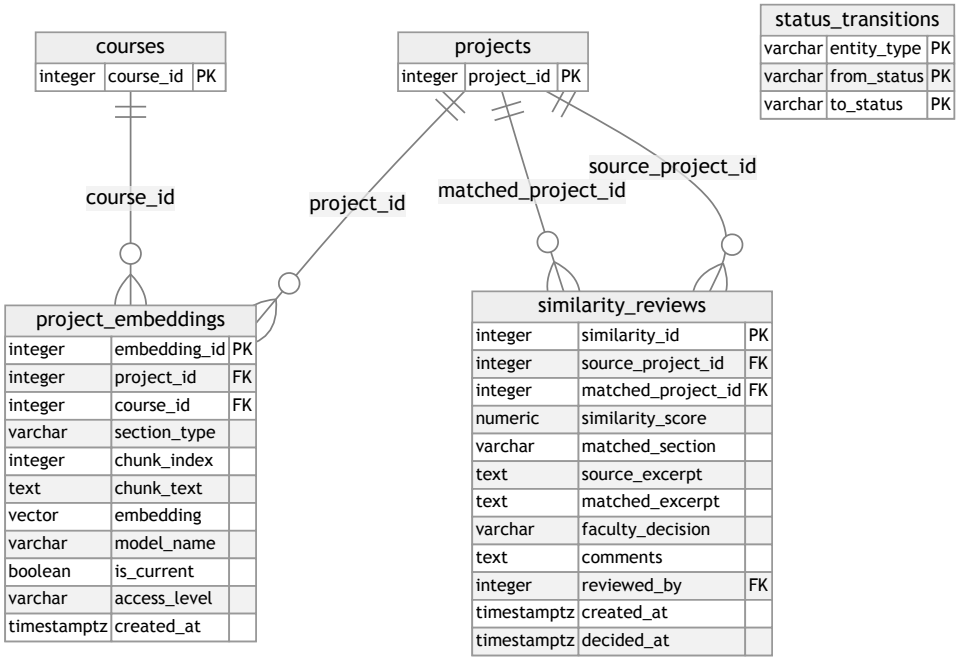
ER 4. Evidence, contribution and risk

Verifiable evidence per student, commit references, peer ratings, and the stored delivery-risk assessments and events.



ER 5. Originality (vector) and workflow control

Embeddings of proposal sections (pgvector) and the faculty decisions on possible overlap; the table that lists every legal status change.



Reading the model. *Identity and academic context*: departments, courses, users, roles, teams. *Project governance*: projects (one active per team), proposal actions, guide allocations, templates. *Delivery*: milestones, tasks, assignments, deliverables, versions. *Quality loop*: reviews, rubric scores, feedback items. *Evidence and analytics*: contribution evidence, git references, peer evaluations, risk assessments and events. *Originality*: project embeddings (pgvector) and similarity reviews. *Control*: status transitions (the allowed workflow edges), audit logs.

6. Relational schema

The relational schema below is generated from the same catalogue (underlined = primary key, italic = foreign key with its target). The full constraint detail (UNIQUE, CHECK, ON DELETE rules, indexes, triggers) is in Appendix A.

Generated from the live catalogue. Notation: underlined = primary key, italic = foreign key (arrow gives the referenced table). Unique keys and CHECK rules are in the data dictionary.

audit_logs (log_id, *user_id*→users, action, entity_type, entity_id, old_values, new_values, created_at)

contribution_evidence (evidence_id, *project_id*→projects, *user_id*→users, *task_id*→tasks, evidence_type, description, weight, verified_flag, *verified_by*→users, verified_at, source_table, source_id, created_at)

courses (course_id, *dept_id*→departments, course_code, course_name, semester, academic_year, is_active, created_at)

deliverables (deliverable_id, *milestone_id*→milestones, deliverable_type, title, repo_link, evidence_url, mongo_doc_id, status, *submitted_by*→users, submitted_at, created_at, updated_at)

departments (dept_id, dept_code, dept_name, created_at)

document_versions (version_id, *deliverable_id*→deliverables, version_number, change_summary, file_name, file_url, file_size_bytes, checksum_sha256, *uploaded_by*→users, uploaded_at)

feedback_items (feedback_id, *review_id*→reviews, *created_by*→users, *assigned_user_id*→users, description, severity, resolution_status, response_text, due_date, resolved_at, *verified_by*→users, verified_at, created_at, updated_at)

git_activities (git_id, *project_id*→projects, *task_id*→tasks, *user_id*→users, repo_url, commit_ref, commit_message, committed_at, *recorded_by*→users, created_at)

guide_allocations (allocation_id, *project_id*→projects, *user_id*→users, allocation_role, *allocated_by*→users, allocated_at)

milestone_required_deliverables (*milestone_id*→milestones, deliverable_type)

milestones (milestone_id, *project_id*→projects, sequence_no, title, description, start_date, due_date, status, weightage, submitted_at, *approved_by*→users, approved_at, approval_comments, created_at, updated_at, required_evidence_type)

notifications (notification_id, *user_id*→users, type, message, link, is_read, created_at)

peer_evaluations (evaluation_id, *project_id*→projects, *milestone_id*→milestones, *evaluator_id*→users, *evaluatee_id*→users, score, comments, created_at)

project_embeddings (embedding_id, *project_id*→projects, *course_id*→courses, section_type, chunk_index, chunk_text, embedding, model_name, is_current, access_level, created_at)

project_templates (template_id, *course_id*→courses, name, description, is_active, *created_by*→users, created_at)

projects (project_id, *team_id*→teams, *course_id*→teams, *template_id*→project_templates, title, abstract, problem_statement, objectives, scope, domain, technologies, repo_link, status, start_date, end_date, *created_by*→users, submitted_at, *approved_by*→users, approved_at, created_at, updated_at)

proposal_actions (action_id, *project_id*→projects, *actor_id*→users, action, from_status, to_status, comments, created_at)

review_scores (*review_id*→reviews, criterion, max_score, score)

reviews (review_id, *milestone_id*→milestones, *reviewer_id*→users, review_type, decision, comments, total_score, review_date, created_at)

risk_assessments (risk_id, *project_id*→projects, risk_score, risk_level, reason, breakdown, calculated_at)

risk_events (event_id, *project_id*→projects, *milestone_id*→milestones, event_type, details, created_at, resolved_at)

roles (role_id, role_name, description)

similarity_reviews (similarity_id, *source_project_id*→projects, *matched_project_id*→projects, similarity_score, matched_section, source_excerpt, matched_excerpt, faculty_decision, comments, *reviewed_by*→users, created_at, decided_at)

status_transitions (entity_type, from_status, to_status)

task_assignments (*task_id*→tasks, *user_id*→users, assignment_type, *assigned_by*→users, assigned_at)

tasks (task_id, *milestone_id*→milestones, title, description, status, priority, effort_hours, planned_start_date, planned_completion_date, actual_completion_date, due_date, *verified_by*→users, verified_at, *created_by*→users, created_at, updated_at)

team_members (*team_id*→teams, *course_id*→teams, *user_id*→users, member_role, responsibility, joined_at)

teams (team_id, *course_id*→courses, team_name, *created_by*→users, created_at)

template_milestones (template_milestone_id, *template_id*→project_templates, sequence_no, title, offset_days, weightage, required_deliverable_types)

user_roles (*user_id*→users, *role_id*→roles, assigned_at, *assigned_by*→users)

users (user_id, name, email, password_hash, status, *dept_id*→departments, identifier, failed_login_count, locked_until, last_login_at, created_at, updated_at)

Design decisions that shape the schema

- A student is in at most one team per course. `team_members` carries `course_id` through a *composite* foreign key to `teams(team_id, course_id)` so that `UNIQUE (course_id, user_id)` is enforced by the database itself.
- A team has at most one *active* project, and no two active proposals in a course share a title: both are **partial unique indexes** (`WHERE status NOT IN ('rejected','archived')`), so a rejected proposal does not block a new one.
- Workflow is data: the table `status_transitions` lists every legal status change; a trigger consults it, so illegal jumps are impossible whichever client writes.
- Foreign keys use deliberate rules: `ON DELETE RESTRICT` for history that must never vanish (projects, milestones, reviews, evidence, audit), `CASCADE` for pure child rows, `SET NULL` for optional references.

7. Database choice and justification

The instructor's rule is that each kind of data must use the technology that suits its nature. ProjectPulse uses three paradigms, each for a specific need.

Paradigm	Technology	What it stores	Why this technology fits
SQL	PostgreSQL	Users, roles, teams, projects, milestones, tasks, assignments, deliverable metadata, reviews, feedback, contribution evidence, risk assessments, audit logs, similarity decisions	The data is highly relational and rule-heavy. We need primary / foreign keys, CHECK constraints, transactions, views, triggers, stored functions and indexes. A workflow such as "may this task become *verified*?" or "may this milestone be approved?" must hold for every client, which only the database can guarantee.
NoSQL	MongoDB	Document records (versions, sections, tags, extracted text), feedback discussion threads, guide meeting notes, attachment metadata	A Software Requirements Specification, a test report and a meeting note have *different shapes*. Forcing them into fixed columns would create many NULLs or constant migrations. Threads are append-only lists of messages that grow without bound: a natural document. Aggregation pipelines summarise them directly.
Vector	PostgreSQL + pgvector	384-dimensional embeddings of proposal sections	"Are these two abstracts about the same idea?" cannot be answered by `LIKE` or full-text search; it needs a meaning-based distance. Keeping vectors **inside PostgreSQL** lets one query combine the vector search with relational filters (course, current / rejected, access level).

Why not one database for everything? Putting documents into relational columns, or workflow rules into a document store, would trade away either flexibility or integrity. PostgreSQL stays the **system of record** for workflow state; MongoDB documents reference it by integer ids. A write that spans both is ordered so that nothing can be left half-done: the MongoDB document is written first as `pending`, the SQL transaction runs, then the document is `confirmed`; if SQL rolls back, the pending document and any uploaded file are removed (section 9.4). A repair script (`npm run mongo:reconcile`) rebuilds the document side from PostgreSQL.

8. Data dictionary

The complete dictionary (every table with column, type, nullability, default, key and constraint, plus indexes and triggers) is generated from the catalogue and reproduced in **Appendix A**. Summary of the 32 tables:

Table	Columns	Purpose
audit_logs	8	Append-only log of important actions written by triggers and the API.
contribution_evidence	13	Verifiable per-student evidence (tasks, uploads, feedback fixes, commits). Only verified rows count.
courses	8	Project courses per semester and academic year.
deliverables	12	Submission metadata (type, links, status) linked to a MongoDB document.
departments	4	Academic departments.
document_versions	10	Every uploaded or linked version of a deliverable, with checksum and size.
feedback_items	14	Trackable feedback assigned to one student: open, in progress, resolved, verified, reopened.
git_activities	10	Manually recorded commit references.
guide_allocations	6	Which faculty guide (one) and reviewers are allocated to a project.
milestone_required_deliverables	2	Deliverable types that must exist before a milestone can be approved.
milestones	16	Dated, weighted milestones of a project with a status workflow.
notifications	7	In-app notifications (reserved; not used in version 1).
peer_evaluations	8	Controlled teammate ratings, one per pair per milestone.
project_embeddings	11	pgvector table: 384-dimensional sentence embeddings of proposal sections.
project_templates	7	Reusable project plans managed by admins.
projects	21	A team's proposal and project: text, status workflow, dates. One active project per team.
proposal_actions	8	Append-only history of proposal decisions (submit, approve, reject, request revision).
review_scores	4	Rubric criteria and scores of a review.
reviews	9	A guide's or reviewer's evaluation of a milestone.
risk_assessments	7	Historical delivery-risk snapshots with score, level, reasons and per-factor breakdown.
risk_events	7	Flags raised for overdue milestones and similar events.
roles	3	The four roles: student, faculty_guide, reviewer, admin.
schema_migrations	2	Which migration files have been applied.
similarity_reviews	12	Pairs of proposals flagged as possibly overlapping, with the faculty decision.
status_transitions	3	Data-driven workflow: which status changes are legal for each entity type.
task_assignments	5	Task owner (exactly one) and collaborators.
tasks	16	Work items under a milestone with planned and actual dates and a todo-to-verified workflow.
team_members	6	Student membership with a leader flag and responsibility; one team per student per course.
teams	5	Student teams, one set per course.
template_milestones	7	Milestones of a template (offset from project start, weightage, mandatory deliverable types).
user_roles	4	Many-to-many link between users and roles.
users	12	Login identity, account status and lockout state. Passwords exist only as bcrypt hashes.

9. SQL implementation

The database has 32 tables, 7 views, 23 functions and procedures, 39 triggers and 80 indexes, created by 14 versioned migration files in `db/migrations`. Each file is applied in one transaction by `npm run db:migrate`, with a progress bar (S-18); `npm run db:reset` rebuilds everything from nothing, which was done after every slice to prove the migrations are repeatable.

File	Content
001	Extensions (`vector`, `citext`), schema, `updated_at` trigger function
002 to 006	Tables: identity, projects and templates, milestones and tasks, deliverables / reviews / feedback, evidence / risk / similarity / audit
007	All B-tree, partial, GIN (full-text) and HNSW (vector) indexes
008	Functions and procedures: completion %, documentation completeness, contribution indicator, delivery risk, overdue flags
009	Views: project progress, unresolved feedback, contribution summary, faculty risk dashboard (+ three more)
010	Triggers: transition guard, audit, immutability, weightage, membership rules, version numbering, evidence generation
011 / 012	**ALTER** and **DROP** demonstrations
013	Least-privilege database role `pp_app` (password supplied from the environment)
014	Reference data: roles and the workflow transition table

9.1 DDL: CREATE, constraints, ALTER, DROP

A table with column CHECKs, a table-level CHECK, composite foreign key, and the partial unique index that allows only one active project per team:

```
CREATE TABLE projects (
  project_id         INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  team_id            INT NOT NULL,
  course_id          INT NOT NULL,
  template_id        INT REFERENCES project_templates(template_id) ON DELETE SET NULL,
  title              VARCHAR(200) NOT NULL CHECK (length(trim(title)) >= 5),
  abstract           TEXT,
  problem_statement  TEXT,
  objectives         TEXT,
  scope              TEXT,
  domain             VARCHAR(80),
  technologies       TEXT[] NOT NULL DEFAULT '{}',
  repo_link          TEXT CHECK (repo_link IS NULL OR repo_link ~* '^https?://'),
  status             VARCHAR(20) NOT NULL DEFAULT 'draft'
                     CHECK (status IN ('draft','submitted','revision_requested','approved',
                                       'rejected','in_progress','completed','archived')),
  start_date         DATE,
  end_date           DATE,
  created_by         INT NOT NULL REFERENCES users(user_id) ON DELETE RESTRICT,
  submitted_at       TIMESTAMPTZ,
  approved_by        INT REFERENCES users(user_id) ON DELETE SET NULL,
  approved_at        TIMESTAMPTZ,
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  FOREIGN KEY (team_id, course_id) REFERENCES teams(team_id, course_id)
      ON DELETE RESTRICT ON UPDATE CASCADE,
  CHECK (end_date IS NULL OR start_date IS NULL OR end_date >= start_date),
  -- Drafts may be incomplete; anything beyond draft must have the full proposal text.
  CHECK (status = 'draft' OR (
        abstract IS NOT NULL AND length(abstract) >= 50 AND
        problem_statement IS NOT NULL AND length(problem_statement) >= 30 AND
        objectives IS NOT NULL AND scope IS NOT NULL AND domain IS NOT NULL))
);
```

```
-- One ACTIVE project per team (rejected/archived ones do not block a new proposal).
CREATE UNIQUE INDEX uq_projects_active_team ON projects(team_id)
  WHERE status NOT IN ('rejected','archived');
```

ALTER (migration 011): a documented schema change, with a new CHECK constraint and a function that starts using the new column.

```
ALTER TABLE milestones
  ADD COLUMN required_evidence_type VARCHAR(30)
  CHECK (required_evidence_type IS NULL OR required_evidence_type IN
        ('proposal','srs','design_doc','source_code','screenshot','test_report','presentation','final_report','other'));

-- Why: guards against typos such as 4000 hours instead of 40.
ALTER TABLE tasks ADD CONSTRAINT chk_task_effort_max CHECK (effort_hours <= 200);

-- Why: make the "no technologies listed" default explicit.
ALTER TABLE projects ALTER COLUMN technologies SET DEFAULT '{}';
```

DROP (migration 012): objects created while experimenting are removed safely.

```
CREATE INDEX IF NOT EXISTS idx_tmp_tasks_title ON tasks(title);       -- created only to be dropped
CREATE VIEW v_tmp_task_list AS SELECT task_id, title FROM tasks;      -- created only to be dropped
DROP VIEW  IF EXISTS v_tmp_task_list;
DROP INDEX IF EXISTS idx_tmp_tasks_title;

ALTER TABLE git_activities ADD COLUMN tmp_notes TEXT;
ALTER TABLE git_activities DROP COLUMN tmp_notes;                      -- temporary column removed
```

Indexes (migration 007): composite B-tree indexes for dashboard filters, *partial* indexes for the open-work queries, a GIN index for full-text search and an HNSW index for vector search.

```
CREATE INDEX idx_projects_status          ON projects(status);
CREATE INDEX idx_projects_course_status   ON projects(course_id, status);
CREATE INDEX idx_milestones_due_status    ON milestones(due_date, status);
CREATE INDEX idx_milestones_project       ON milestones(project_id, sequence_no);
CREATE INDEX idx_tasks_milestone_status   ON tasks(milestone_id, status);
-- partial: only open tasks can be overdue, so the index stays small
CREATE INDEX idx_tasks_due_open           ON tasks(due_date) WHERE status NOT IN ('done','verified');
CREATE INDEX idx_task_assign_user         ON task_assignments(user_id);        -- "task assignee"
CREATE INDEX idx_feedback_status          ON feedback_items(resolution_status);
CREATE INDEX idx_feedback_assignee_open   ON feedback_items(assigned_user_id)
                                          WHERE resolution_status IN ('open','in_progress','reopened');
CREATE INDEX idx_reviews_milestone        ON reviews(milestone_id);
CREATE INDEX idx_risk_project_time        ON risk_assessments(project_id, calculated_at DESC);
CREATE INDEX idx_risk_level               ON risk_assessments(risk_level);      -- "risk category"
CREATE INDEX idx_evidence_project_user    ON contribution_evidence(project_id, user_id) WHERE verified_flag;
CREATE INDEX idx_deliverables_milestone   ON deliverables(milestone_id, deliverable_type, status); -- "document metadata"
CREATE INDEX idx_docver_deliverable       ON document_versions(deliverable_id, version_number DESC);
CREATE INDEX idx_audit_entity             ON audit_logs(entity_type, entity_id, created_at DESC);
CREATE INDEX idx_audit_user_time          ON audit_logs(user_id, created_at DESC);
CREATE INDEX idx_team_members_user        ON team_members(user_id);
CREATE INDEX idx_similarity_decision      ON similarity_reviews(faculty_decision);
CREATE INDEX idx_notifications_unread     ON notifications(user_id) WHERE NOT is_read;

-- Full-text search on proposals (extra advanced feature). The query must use the SAME expression.
CREATE INDEX idx_projects_fts ON projects USING GIN (
  to_tsvector('english', coalesce(title,'') || ' ' || coalesce(abstract,'') || ' ' || coalesce(problem_statement,'')));
```

```
-- Vector index (pgvector): cosine distance, HNSW
CREATE INDEX idx_embeddings_hnsw ON project_embeddings
  USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);
CREATE INDEX idx_embeddings_filter ON project_embeddings(course_id, section_type) WHERE is_current;
```

Evidence that the indexes matter (`npm run docs:explain` adds synthetic rows inside a transaction, measures the query with and without the index, then rolls everything back):

Summary

Query	Index	Before	After	Speed-up
Q2 overdue open tasks (60,000 tasks, 0.5% open)	idx_tasks_due_open (partial index)	5.301 ms	0.235 ms	22.6x
Q11 open feedback per student (60,000 items, 0.4% open)	idx_feedback_assignee_open (partial index)	5.289 ms	0.306 ms	17.3x
Q6 five nearest vectors (6,000 vectors, 384 dims)	idx_embeddings_hnsw (HNSW, cosine)	4.717 ms	1.975 ms	2.4x

Q4 (milestones with unresolved feedback) uses `<>`, which no B-tree index can answer, so it is not index-driven; its joins use the primary-key and `idx_reviews_milestone` indexes. On the real demo data (a few dozen rows per table) PostgreSQL chooses sequential scans whatever indexes exist, which is why the synthetic rows are needed to show the effect.

9.2 Views, functions and procedures

The four required views (progress, unresolved feedback, contribution summary, faculty risk dashboard) are in migration 009. As an example, the contribution summary counts **only verified** evidence and shows each member's share of the team total:

```
-- 3. Contribution summary per member (verified evidence only) + share of team total
CREATE OR REPLACE VIEW v_contribution_summary AS
WITH per_member AS (
  SELECT p.project_id, tm.user_id,
         COALESCE(SUM(ce.weight * CASE ce.evidence_type
                WHEN 'task_completed' THEN 3 WHEN 'document_upload' THEN 2
                WHEN 'feedback_resolved' THEN 2 WHEN 'review_participation' THEN 1
                WHEN 'git_activity' THEN 1 END) FILTER (WHERE ce.verified_flag), 0) AS evidence_points,
         COUNT(ce.evidence_id) FILTER (WHERE ce.verified_flag)   AS verified_items,
         COUNT(ce.evidence_id) FILTER (WHERE NOT ce.verified_flag) AS unverified_items
  FROM projects p
  JOIN team_members tm ON tm.team_id = p.team_id
  LEFT JOIN contribution_evidence ce ON ce.project_id = p.project_id AND ce.user_id = tm.user_id
  GROUP BY p.project_id, tm.user_id
)
SELECT pm.project_id, pm.user_id, u.name, pm.verified_items, pm.unverified_items, pm.evidence_points,
       ROUND(100 * pm.evidence_points / NULLIF(SUM(pm.evidence_points) OVER (PARTITION BY pm.project_id), 0), 1)
                                                              AS share_pct,
       ROUND(AVG(pe.score), 2)                                AS avg_peer_score
FROM per_member pm
JOIN users u ON u.user_id = pm.user_id
LEFT JOIN peer_evaluations pe ON pe.project_id = pm.project_id AND pe.evaluatee_id = pm.user_id
GROUP BY pm.project_id, pm.user_id, u.name, pm.verified_items, pm.unverified_items, pm.evidence_points;
```

Delivery risk is calculated *inside the database* so the API, the sweep script and `psql` always agree. The function returns the score, the level, a plain-English reason and the points of each factor; the procedure stores a snapshot:

```
-- Delivery risk: returns score (0-100), level, plain-English reason, and per-factor breakdown.
CREATE OR REPLACE FUNCTION fn_calculate_project_risk(p_project_id INT)
RETURNS TABLE (risk_score NUMERIC, risk_level TEXT, reason TEXT, breakdown JSONB)
LANGUAGE plpgsql STABLE AS $$
DECLARE
  v_overdue_ms INT; v_overdue_tasks INT; v_missing INT; v_open_fb INT; v_overdue_fb INT;
```

```
  v_last_activity TIMESTAMPTZ; v_idle_days INT;
  v_members INT; v_total_pts NUMERIC; v_max_pts NUMERIC;
  pts_ms NUMERIC; pts_tasks NUMERIC; pts_docs NUMERIC; pts_fb NUMERIC; pts_idle NUMERIC; pts_imb NUMERIC;
  v_total NUMERIC; v_level TEXT; v_reasons TEXT[] := ARRAY[]::TEXT[];
BEGIN
  SELECT COUNT(*) INTO v_overdue_ms FROM milestones m
   WHERE m.project_id = p_project_id AND m.due_date < CURRENT_DATE AND m.status <> 'approved';

  SELECT COUNT(*) INTO v_overdue_tasks FROM tasks t JOIN milestones m ON m.milestone_id = t.milestone_id
   WHERE m.project_id = p_project_id AND t.due_date < CURRENT_DATE AND t.status NOT IN ('done','verified');

  SELECT COUNT(*) INTO v_missing FROM milestone_required_deliverables r
    JOIN milestones m ON m.milestone_id = r.milestone_id
   WHERE m.project_id = p_project_id AND m.due_date <= CURRENT_DATE + 7 AND m.status <> 'approved'
     AND NOT EXISTS (SELECT 1 FROM deliverables d WHERE d.milestone_id = m.milestone_id
                       AND d.deliverable_type = r.deliverable_type AND d.status IN ('submitted','accepted'));

  SELECT COUNT(*), COUNT(*) FILTER (WHERE f.due_date < CURRENT_DATE) INTO v_open_fb, v_overdue_fb
    FROM feedback_items f JOIN reviews rv ON rv.review_id = f.review_id
    JOIN milestones m ON m.milestone_id = rv.milestone_id
   WHERE m.project_id = p_project_id AND f.resolution_status <> 'verified';

  SELECT GREATEST(
     COALESCE((SELECT MAX(t.updated_at) FROM tasks t JOIN milestones m ON m.milestone_id = t.milestone_id
                WHERE m.project_id = p_project_id), '-infinity'::timestamptz),
     COALESCE((SELECT MAX(dv.uploaded_at) FROM document_versions dv
                JOIN deliverables d ON d.deliverable_id = dv.deliverable_id
                JOIN milestones m ON m.milestone_id = d.milestone_id
                WHERE m.project_id = p_project_id), '-infinity'::timestamptz),
     COALESCE((SELECT MAX(ce.created_at) FROM contribution_evidence ce
                WHERE ce.project_id = p_project_id), '-infinity'::timestamptz))
    INTO v_last_activity;
  v_idle_days := CASE WHEN v_last_activity = '-infinity'::timestamptz THEN 30
                      ELSE CURRENT_DATE - v_last_activity::date END;

  SELECT COUNT(*) INTO v_members FROM team_members tm JOIN projects p ON p.team_id = tm.team_id
   WHERE p.project_id = p_project_id;
  SELECT COALESCE(SUM(cs.evidence_points), 0), COALESCE(MAX(cs.evidence_points), 0)
    INTO v_total_pts, v_max_pts FROM v_contribution_summary cs WHERE cs.project_id = p_project_id;

  -- Points per factor (max 25 + 20 + 20 + 15 + 10 + 10 = 100)
  pts_ms    := LEAST(25, v_overdue_ms * 12.5);
  pts_tasks := LEAST(20, v_overdue_tasks * 4);
  pts_docs  := LEAST(20, v_missing * 10);
  pts_fb    := LEAST(15, v_open_fb * 3 + v_overdue_fb * 2);
  pts_idle  := CASE WHEN v_idle_days <= 5 THEN 0 WHEN v_idle_days <= 10 THEN 5 ELSE 10 END;
  pts_imb   := CASE WHEN v_members >= 2 AND v_total_pts >= 5
                    THEN ROUND(10 * GREATEST(0, (v_max_pts / v_total_pts - 1.0 / v_members)
                                        / (1 - 1.0 / v_members)), 2)
                    ELSE 0 END;
  v_total := LEAST(100, pts_ms + pts_tasks + pts_docs + pts_fb + pts_idle + pts_imb);
  v_level := CASE WHEN v_total < 25 THEN 'low' WHEN v_total < 50 THEN 'moderate'
                  WHEN v_total < 75 THEN 'high' ELSE 'critical' END;

  IF v_overdue_ms   > 0 THEN v_reasons := v_reasons || format('%s milestone(s) overdue (+%s)', v_overdue_ms, pts_ms); END
IF;
  IF v_overdue_tasks> 0 THEN v_reasons := v_reasons || format('%s task(s) overdue (+%s)', v_overdue_tasks, pts_tasks); END
IF;
```

```
  IF v_missing      > 0 THEN v_reasons := v_reasons || format('%s mandatory deliverable(s) missing (+%s)', v_missing,
pts_docs); END IF;
  IF v_open_fb      > 0 THEN v_reasons := v_reasons || format('%s unresolved feedback item(s), %s overdue (+%s)', v_open_fb,
v_overdue_fb, pts_fb); END IF;
  IF pts_idle       > 0 THEN v_reasons := v_reasons || format('no recorded activity for %s days (+%s)', v_idle_days,
pts_idle); END IF;
  IF pts_imb        > 0 THEN v_reasons := v_reasons || format('uneven contribution across members (+%s)', pts_imb); END IF;

  RETURN QUERY SELECT v_total, v_level,
      COALESCE(NULLIF(array_to_string(v_reasons, '; '), ''), 'No risk factors detected'),
      jsonb_build_object('overdue_milestones', pts_ms, 'overdue_tasks', pts_tasks,
                         'missing_deliverables', pts_docs, 'unresolved_feedback', pts_fb,
                         'inactivity', pts_idle, 'contribution_imbalance', pts_imb);
END $$;
```

```
CREATE OR REPLACE PROCEDURE sp_recalculate_risk(p_project_id INT)
LANGUAGE plpgsql AS $$
DECLARE r RECORD;
BEGIN
  SELECT * INTO r FROM fn_calculate_project_risk(p_project_id);
  INSERT INTO risk_assessments(project_id, risk_score, risk_level, reason, breakdown)
  VALUES (p_project_id, r.risk_score, r.risk_level, r.reason, r.breakdown);
END $$;
```

Risk formula (also shown in the application under "How is this calculated?"): overdue milestones 12.5 each (max 25) + overdue tasks 4 each (max 20) + missing mandatory deliverables 10 each (max 20) + unresolved feedback 3 each plus 2 if overdue (max 15) + inactivity 0 / 5 / 10 (max 10) + contribution imbalance up to 10. Levels: low below 25, moderate below 50, high below 75, critical from 75. A `risk_assessments` CHECK constraint refuses a level that does not match its score.

9.3 Triggers

Rules that must hold *whatever client writes the data* live in triggers. The generic **transition guard** reads the `status_transitions` table; the **audit** trigger records who changed which status (the API sets `app.current_user_id` per transaction); two triggers make `audit_logs` and `proposal_actions` append-only; evidence is generated automatically when a task is verified.

```
CREATE OR REPLACE FUNCTION fn_validate_transition() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE v_col TEXT := TG_ARGV[0]; v_entity TEXT := TG_ARGV[1]; v_old TEXT; v_new TEXT;
BEGIN
  v_old := to_jsonb(OLD) ->> v_col;
  v_new := to_jsonb(NEW) ->> v_col;
  IF v_old IS DISTINCT FROM v_new AND NOT EXISTS (
       SELECT 1 FROM status_transitions
        WHERE entity_type = v_entity AND from_status = v_old AND to_status = v_new) THEN
    RAISE EXCEPTION 'Invalid % status change: % -> %', v_entity, v_old, v_new
      USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$;
```

```
CREATE OR REPLACE FUNCTION fn_audit_status_change() RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE
  v_user INT := NULLIF(current_setting('app.current_user_id', true), '')::INT;
  v_pk TEXT := to_jsonb(NEW) ->> TG_ARGV[0];
  v_old TEXT := to_jsonb(OLD) ->> TG_ARGV[1];
  v_new TEXT := to_jsonb(NEW) ->> TG_ARGV[1];
BEGIN
```

```
  IF v_old IS DISTINCT FROM v_new THEN
    INSERT INTO audit_logs(user_id, action, entity_type, entity_id, old_values, new_values)
    VALUES (v_user, 'STATUS_CHANGE', TG_TABLE_NAME, v_pk,
            jsonb_build_object(TG_ARGV[1], v_old), jsonb_build_object(TG_ARGV[1], v_new));
  END IF;
  RETURN NEW;
END $$;
```

```
CREATE OR REPLACE FUNCTION fn_block_modify_immutable() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  RAISE EXCEPTION '% is append-only; % is not allowed', TG_TABLE_NAME, TG_OP;
END $$;
```

```
CREATE OR REPLACE FUNCTION fn_evidence_from_task() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.status = 'verified' AND OLD.status IS DISTINCT FROM 'verified' THEN
    INSERT INTO contribution_evidence(project_id, user_id, task_id, evidence_type, description,
                                      weight, verified_flag, verified_by, verified_at, source_table, source_id)
    SELECT m.project_id, ta.user_id, NEW.task_id, 'task_completed',
           'Verified task: ' || NEW.title,
           LEAST(10, GREATEST(1, ROUND(NEW.effort_hours / 4.0, 2))),
           TRUE, NEW.verified_by, NEW.verified_at, 'tasks', NEW.task_id
      FROM task_assignments ta JOIN milestones m ON m.milestone_id = NEW.milestone_id
     WHERE ta.task_id = NEW.task_id
    ON CONFLICT DO NOTHING;
  END IF;
  RETURN NEW;
END $$;
CREATE TRIGGER trg_task_evidence AFTER UPDATE OF status ON tasks
  FOR EACH ROW EXECUTE FUNCTION fn_evidence_from_task();
```

Other triggers: weightage total at most 100 (deferred constraint trigger, checked at COMMIT), team members must be students, a guide must hold the guide role and not be on the team, task and feedback assignees must be team members, automatic document version numbers, overdue-milestone risk events, and "a milestone cannot be approved while a mandatory deliverable type is missing". S-19 shows the trigger errors in `psql`.

9.4 Transactions: the atomic milestone submission

Submitting a milestone must never leave the database half-updated. One transaction writes the deliverable, its version, the milestone status, the review requests and an audit row; the lock in step 1 makes a double click harmless.

```
    const result = await withTransaction(user.userId, async (c) => {
      // 1. lock the milestone (a second concurrent submit waits here) and verify it can be submitted
      step(1);
      const { rows: [m] } = await c.query(`SELECT status FROM milestones WHERE milestone_id = $1 FOR UPDATE`,
[milestoneId]);
      if (!OPEN_FOR_UPLOADS.includes(m.status)) {
        throw new ApiError(409, 'bad_state', m.status === 'planned' ? 'Start the milestone before submitting it.' : `This
milestone is already ${m.status.replace('_', ' ')}.`);
      }

      // 2. deliverable metadata
      step(2);
      let newDeliverableId: number | null = null;
      if (meta && v) {
        const { rows: [d] } = await c.query(
          `INSERT INTO deliverables(milestone_id, deliverable_type, title, repo_link, evidence_url, mongo_doc_id, status,
```

```
submitted_by, submitted_at)
           VALUES ($1,$2,$3,$4,$5,$6,'submitted',$7, now()) RETURNING deliverable_id`,
          [milestoneId, meta.deliverableType, meta.title, meta.repoLink || null, meta.evidenceUrl || null, mongoId,
user.userId]);
        newDeliverableId = d.deliverable_id;

        // 3. document version (version_number is assigned by a trigger). A bad payload fails HERE, e.g. a
        //    negative file_size_bytes violates a CHECK, and everything above is rolled back.
        step(3);
        await insertVersion(c, d.deliverable_id, v, user.userId);
      }
      // Drafts the team prepared earlier are submitted together with the milestone.
      const flipped = await c.query(
        `UPDATE deliverables SET status = 'submitted', submitted_at = now()
          WHERE milestone_id = $1 AND status = 'draft'
            AND EXISTS (SELECT 1 FROM document_versions dv WHERE dv.deliverable_id = deliverables.deliverable_id)`,
[milestoneId]);
      const { rows: [cnt] } = await c.query(`SELECT COUNT(*)::int AS n FROM deliverables WHERE milestone_id = $1 AND status
= 'submitted'`, [milestoneId]);
      if (cnt.n === 0) {
        throw new ApiError(422, 'nothing_to_submit', 'Attach at least one deliverable (file or link) before submitting.', {
file: 'Attach a file or provide a link' });
      }

      // 4. milestone status
      step(4);
      await c.query(`UPDATE milestones SET status = 'submitted', submitted_at = now() WHERE milestone_id = $1`,
[milestoneId]);

      // 5. review requests: one pending review per allocated guide / reviewer (none twice)
      step(5);
      const reviews = await c.query(
        `INSERT INTO reviews(milestone_id, reviewer_id, review_type)
         SELECT $1, ga.user_id, 'milestone' FROM guide_allocations ga
          WHERE ga.project_id = $2
            AND NOT EXISTS (SELECT 1 FROM reviews r WHERE r.milestone_id = $1 AND r.reviewer_id = ga.user_id AND r.decision
= 'pending')`,
        [milestoneId, projectId]);

      // 6. audit row
      step(6);
      await c.query(
        `INSERT INTO audit_logs(user_id, action, entity_type, entity_id, new_values) VALUES
($1,'MILESTONE_SUBMITTED','milestone',$2,$3::jsonb)`,
        [user.userId, String(milestoneId), JSON.stringify({ newDeliverableId, draftsSubmitted: flipped.rowCount,
reviewRequests: reviews.rowCount })]);

      return { newDeliverableId, draftsSubmitted: flipped.rowCount ?? 0, reviewRequests: reviews.rowCount ?? 0 };
    });
```

Two things live outside PostgreSQL, so they are compensated: the uploaded **file** and the **MongoDB document** (written first as `pending`). If any step fails, the transaction rolls back and the file and pending document are deleted. The automated tests (T-15) force a failure at step 3 (a negative file size violates a CHECK), and at steps 5 and 6 with a real file, and then verify that no deliverable, version, review, audit row, file or Mongo document exists and that the milestone status is unchanged. S-11 shows the message the user sees. Other transactional operations: proposal decision (status + immutable history + audit), template application (milestones + required deliverables), team creation with members, feedback actions, similarity decisions.

9.5 Security at the database level

- Passwords are bcrypt hashes (cost 12); the application connects as the least-privilege role `pp_app`, which cannot create tables and cannot UPDATE or DELETE `audit_logs` or `proposal_actions`:

```
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'pp_app') THEN
    CREATE ROLE pp_app LOGIN PASSWORD :'APP_DB_PASSWORD';
  ELSE
    ALTER ROLE pp_app LOGIN PASSWORD :'APP_DB_PASSWORD';
  END IF;
END $$;

GRANT USAGE ON SCHEMA projectpulse TO pp_app;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA projectpulse TO pp_app;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA projectpulse TO pp_app;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA projectpulse TO pp_app;
GRANT EXECUTE ON ALL PROCEDURES IN SCHEMA projectpulse TO pp_app;
-- Append-only history: even the app role cannot rewrite it (triggers block it too).
REVOKE UPDATE, DELETE ON projectpulse.audit_logs, projectpulse.proposal_actions FROM pp_app;
-- Migrations are applied by the owner; the app must never change the schema.
REVOKE CREATE ON SCHEMA projectpulse FROM pp_app;
ALTER ROLE pp_app SET search_path = projectpulse, public;
```

- Every query uses bound parameters; the only dynamic SQL is `ORDER BY`, chosen from a fixed whitelist.
- Database-level validation (spec section 15): dates, positive numbers, allowed values, uniqueness, referential integrity, status / timestamp consistency, immutability.

10. NoSQL implementation (MongoDB)

Collections (`db/mongo/collections.ts`, created by `npm run mongo:init`, each with a `$jsonSchema` validator and indexes):

Collection	Purpose	Key fields
`documents`	One document per deliverable with its versions, free-form `sections`, tags and extracted text	projectId, milestoneId, deliverableId, documentType, versions[], sections{}, tags[], state
`feedback_threads`	Discussion thread per feedback item	feedbackId (unique), projectId, theme, status, messages[]
`meeting_notes`	Guide meeting notes	projectId, date, attendees[], notes, actionItems[]
`attachments`	Metadata of uploaded files	projectId, deliverableId, fileName, storagePath, uploadedBy

Document model (a seeded SRS; ids are PostgreSQL integers, `sections` is free-form on purpose):


```
{
  "projectId": 1,
  "milestoneId": 2,
  "deliverableId": 2,
  "documentType": "SRS",
  "deliverableType": "srs",
  "title": "CampusConnect SRS",
  "versions": [
    {
      "versionNumber": 1,
      "uploadedBy": 8,
      "uploadedAt": "2026-09-30T04:13:31.728Z",
      "changeSummary": "First full draft",
      "fileUrl": "seed/m2/d2/srs_v1.md",
      "fileName": "srs_v1.md",
      "sizeBytes": 107
    },
    {
      "versionNumber": 2,
      "uploadedBy": 9,
      "uploadedAt": "2026-10-06T04:13:31.728Z",
      "changeSummary": "Incorporated organiser feedback",
      "fileUrl": "seed/m2/d2/srs_v2.md",
      "fileName": "srs_v2.md",
      "sizeBytes": 122
    },
    {
      "versionNumber": 3,
      "uploadedBy": 8,
      "uploadedAt": "2026-10-09T04:13:31.728Z",
      "changeSummary": "Final SRS after peer review",
      "fileUrl": "seed/m2/d2/srs_v3.md",
      "fileName": "srs_v3.md",
      "sizeBytes": 118
    }
  ],
  "sections": {
    "problemStatement": "Clubs coordinate events and volunteers through scattered chat groups and spreadsheets, which causes double bookings, missed volunteer shifts and no record of participation.",
    "objectives": [
      "Provide one place to create and approve campus events.",
      "Automate volunteer slot allocation and reminders.",
      "Track attendance with QR check-in.",
      "Report participation per club."
```

```
    ],
    "scope": "Event creation, registration, volunteer rostering, QR attendance and basic analytics for one campus. Payments
and ticketing are excluded."
  },
  "tags": [
    "Campus Management",
    "Next.js",
    "PostgreSQL",
    "Node.js"
  ],
  "wordCount": 17,
  "state": "confirmed",
  "createdAt": "2026-09-30T04:13:31.728Z"
}
```

Validator and indexes for `documents` (an insert with an unknown type or an empty `versions` array is rejected with error code 121; see S-20):

```
  documents: {
    validator: {
      $jsonSchema: {
        bsonType: 'object',
        required: ['projectId', 'milestoneId', 'documentType', 'title', 'versions', 'tags'],
        properties: {
          projectId: intLike, milestoneId: intLike, deliverableId: { bsonType: ['int', 'long', 'null'] },
          documentType: { enum: [...DOCUMENT_TYPES] },
          title: { bsonType: 'string' },
          versions: { bsonType: 'array', minItems: 1, items: versionSchema },
          tags: { bsonType: 'array', items: { bsonType: 'string' } },
          state: { enum: ['pending', 'confirmed'] },
          // `sections` is free-form on purpose: that is the reason this is not a SQL table.
        },
      },
    },
    indexes: [
      { key: { projectId: 1, milestoneId: 1 } },
      { key: { documentType: 1 } },
      { key: { 'versions.uploadedAt': -1 } },
      { key: { tags: 1 } },
      { key: { deliverableId: 1 }, options: { sparse: true } },
      { key: { state: 1, createdAt: 1 } },
      { key: { title: 'text', extractedText: 'text', 'sections.problemStatement': 'text' }, options: { name:
'documents_text' } },
    ],
  },
```

Aggregation pipelines (`db/mongo/aggregations.ts`). A3 is a SQL + MongoDB merge: SQL knows which types are *required*, MongoDB knows which documents *exist*; the result must equal the SQL function `fn_documentation_completeness` (tested).

A1: Version counts per document (project 1)

```
db.documents.aggregate([
  {
    "$match": {
      "projectId": 1,
      "state": "confirmed"
    }
  },
```

```
  {
    "$project": {
      "title": 1,
      "documentType": 1,
      "deliverableType": 1,
      "versionCount": {
        "$size": "$versions"
      },
      "lastUpdated": {
        "$max": "$versions.uploadedAt"
      }
    }
  },
  {
    "$sort": {
      "versionCount": -1,
      "title": 1
    }
  }
])
```

Output

```
[
  {
    "_id": "6aded37197102d7db62834f0",
    "documentType": "SRS",
    "deliverableType": "srs",
    "title": "CampusConnect SRS",
    "versionCount": 3,
    "lastUpdated": "2026-10-09T04:13:31.728Z"
  },
  {
    "_id": "6aded37197102d7db62834ef",
    "documentType": "Proposal",
    "deliverableType": "proposal",
    "title": "CampusConnect proposal",
    "versionCount": 2,
    "lastUpdated": "2026-09-18T04:13:31.728Z"
  },
  {
    "_id": "6aded37197102d7db62834f1",
    "documentType": "Design",
    "deliverableType": "design_doc",
    "title": "CampusConnect design document (draft)",
    "versionCount": 1,
    "lastUpdated": "2026-10-24T04:13:31.728Z"
  }
]
```

A2: Document-type distribution (projects 1-4)

```
db.documents.aggregate([
  {
    "$match": {
      "projectId": {
        "$in": [
```

```
          1,
          2,
          3,
          4
        ]
      },
      "state": "confirmed"
    }
  },
  {
    "$group": {
      "_id": "$documentType",
      "documents": {
        "$sum": 1
      },
      "avgVersions": {
        "$avg": {
          "$size": "$versions"
        }
      }
    }
  },
  {
    "$sort": {
      "documents": -1,
      "_id": 1
    }
  }
])
```

Output

```
[
  {
    "_id": "Proposal",
    "documents": 4,
    "avgVersions": 1.5
  },
  {
    "_id": "SRS",
    "documents": 3,
    "avgVersions": 2.3333333333333335
  },
  {
    "_id": "Design",
    "documents": 2,
    "avgVersions": 1.5
  },
  {
    "_id": "Other",
    "documents": 2,
    "avgVersions": 1.5
  }
]
```

A3: Which required (milestone, type) pairs exist: Mongo half of the completeness merge (project 4)

```
db.documents.aggregate([
  {
    "$match": {
      "deliverableId": {
        "$in": [
          7,
          8,
          9,
          10,
          11
        ]
      },
      "state": "confirmed"
    }
  },
  {
    "$group": {
      "_id": {
        "milestoneId": "$milestoneId",
        "type": "$deliverableType"
      },
      "versions": {
        "$sum": {
          "$size": "$versions"
        }
      }
    }
  }
])
```

Output

```
[
  {
    "_id": {
      "milestoneId": 18,
      "type": "design_doc"
    },
    "versions": 2
  },
  {
    "_id": {
      "milestoneId": 17,
      "type": "srs"
    },
    "versions": 2
  },
  {
    "_id": {
      "milestoneId": 19,
      "type": "screenshot"
    },
    "versions": 1
  },
  {
    "_id": {
      "milestoneId": 16,
```

```
      "type": "proposal"
    },
    "versions": 1
  },
  {
    "_id": {
      "milestoneId": 19,
      "type": "source_code"
    },
    "versions": 2
  }
]
```

A4: Open feedback themes (project 4)

```
db.feedback_threads.aggregate([
  {
    "$match": {
      "projectId": {
        "$in": [
          4
        ]
      },
      "status": {
        "$in": [
          "open",
          "in_progress",
          "reopened"
        ]
      }
    }
  },
  {
    "$group": {
      "_id": "$theme",
      "openThreads": {
        "$sum": 1
      },
      "avgMessages": {
        "$avg": {
          "$size": "$messages"
        }
      }
    }
  },
  {
    "$sort": {
      "openThreads": -1,
      "_id": 1
    }
  },
  {
    "$limit": 10
  }
])
```

Output

```
[
  {
    "_id": "design",
    "openThreads": 2,
    "avgMessages": 2
  },
  {
    "_id": "documentation",
    "openThreads": 1,
    "avgMessages": 1
  },
  {
    "_id": "requirements",
    "openThreads": 1,
    "avgMessages": 1
  }
]
```

A5: Most used tags (projects 1-4)

```
db.documents.aggregate([
  {
    "$match": {
      "projectId": {
        "$in": [
          1,
          2,
          3,
          4
        ]
      },
      "state": "confirmed"
    }
  },
  {
    "$unwind": "$tags"
  },
  {
    "$group": {
      "_id": "$tags",
      "n": {
        "$sum": 1
      }
    }
  },
  {
    "$sort": {
      "n": -1,
      "_id": 1
    }
  },
  {
    "$limit": 15
  }
])
```

Output

```
[
  {
    "_id": "PostgreSQL",
    "n": 11
  },
  {
    "_id": "Next.js",
    "n": 10
  },
  {
    "_id": "Agriculture",
    "n": 5
  },
  {
    "_id": "MongoDB",
    "n": 5
  },
  {
    "_id": "Campus Management",
    "n": 3
  },
  {
    "_id": "Node.js",
    "n": 3
  }
]
```

11. Vector implementation (pgvector)

Question	Answer
What is embedded?	Title, abstract, problem statement, objectives and scope of each submitted proposal; long text is split into ~200-word chunks with a 30-word overlap
Which model?	`Xenova/all-MiniLM-L6-v2` (sentence-transformer), run locally through `@huggingface/transformers`: no API key, 384 dimensions, vectors normalised to length 1
How are vectors stored?	Table `project_embeddings`: project, course, section, chunk text, `vector(384)`, model name, `is_current`, access level; unique per (project, section, chunk, model)
Which metric?	Cosine similarity. SQL operator `<=>` is cosine *distance*, so similarity = 1 − distance. Index: HNSW with `vector_cosine_ops`
How is it retrieved?	The proposal's sections are compared like-for-like (title with title, abstract with abstract ...) against current embeddings of the same course. The project score is the weighted mean of the best section similarities (title 0.15, abstract 0.30, problem 0.30, objectives 0.15, scope 0.10). A pair is flagged at **0.80** overall or when any single section reaches **0.90**; the top 5 are kept
Ethical control	Similarity is a *review signal*, never a verdict. Faculty record the decision (not similar, acceptable overlap, needs revision, escalate); the wording is "possible overlap". Students only receive a generic yes / no hint and never see another team's text, titles or scores

Retrieval query (the metadata filter and the vector search happen in one statement):

```
        const { rows } = await c.query(
          `SELECT e.project_id, e.chunk_text, 1 - (e.embedding <=> $1::vector) AS similarity
             FROM project_embeddings e
            WHERE e.is_current AND e.course_id = $2 AND e.project_id <> $3 AND e.section_type = $4 AND e.model_name = $5
            ORDER BY e.embedding <=> $1::vector
            LIMIT 30`, [chunk.vector, courseId, excludeProjectId ?? 0, s.section, MODEL_NAME]);
```

Scoring (pure functions, unit-tested):

```
export const SECTION_WEIGHTS = { title: 0.15, abstract: 0.30, problem_statement: 0.30, objectives: 0.15, scope: 0.10 } as
const;
export type Section = keyof typeof SECTION_WEIGHTS;
export const SECTIONS = Object.keys(SECTION_WEIGHTS) as Section[];

export const SECTION_LABEL: Record<Section, string> = {
  title: 'Title', abstract: 'Abstract', problem_statement: 'Problem statement', objectives: 'Objectives', scope: 'Scope',
};

export const config = () => ({
  threshold: Number(process.env.SIMILARITY_THRESHOLD ?? 0.8),   // project-level score that flags a pair for review
  sectionLimit: 0.9,                                             // ...or any single section at least this similar
  topK: Number(process.env.SIMILARITY_TOP_K ?? 5),
});

export interface SectionSimilarity { section: Section; similarity: number }

/** Weighted mean of the best similarity per section, over the sections both proposals have. 0..1. */
export function projectScore(sims: SectionSimilarity[]): number {
  let weighted = 0, weights = 0;
  for (const s of sims) { const w = SECTION_WEIGHTS[s.section]; weighted += w * s.similarity; weights += w; }
  return weights === 0 ? 0 : Math.min(1, Math.max(0, weighted / weights));
}

/** Flag for faculty review: overall score at or above the threshold, OR any single section very close. */
export function isFlagged(score: number, sims: SectionSimilarity[], c = config()): boolean {
```

Result on the demo data. Echo's proposal (*CollegeEvents Hub*) is a close paraphrase of Alpha's (*CampusConnect*): it is flagged at the review threshold, with the matching text shown to faculty. The unrelated proposals score far below it, and Foxtrot

(*FestFlow*, a similar topic in a **different course**) can never appear because of the course filter.

Flagged pair stored for faculty review (table `similarity_reviews`):

similarity_id	source_project_id	matched_project_id	similarity_score	matched_section	source	matched	faculty_decision
1	5	1	0.8671	objectives	CollegeEvents Hub: Campus Event Management and Volunteer Coordination Platform	CampusConnect: Event Management and Volunteer Coordination Portal	pending

Section-by-section cosine similarity (Echo against Alpha):

section	cosine_similarity
abstract	0.706
objectives	0.980
problem_statement	0.920
scope	0.972
title	0.900

Echo against every other proposal in its course (mean of the section similarities; only the first pair crosses the review threshold):

title	mean_section_similarity
CampusConnect: Event Management and Volunteer Coordination Portal	0.896
LibrarySeat: Real-Time Study Seat Reservation with Occupancy Analytics	0.383
ClinicQueue: Appointment Scheduling with Wait-Time Estimation	0.322
AgriAdvisor: Crop Disease Reporting and Advisory Portal	0.167

The seeded wording was calibrated against the *real* model scores (a looser paraphrase scored 0.69, below the threshold), not adjusted to fake a number; the threshold is configurable (`SIMILARITY_THRESHOLD`). Faculty screen: S-15. Student hint and decision workflow: tests T-22 and T-23.

12. Important queries

All queries are in `db/queries/important_queries.sql`, use bound parameters, and are executed by an automated test against the demo data. Each shows the concept it demonstrates and its real output.

Q1. High-risk projects of a guide (view, JOIN, ORDER BY)

```
SELECT d.project_id, d.title, d.team_name, d.course_name, d.risk_score, d.risk_level, d.reason, d.days_to_next_deadline
FROM v_faculty_risk_dashboard d
WHERE d.guide_id = $1 AND d.risk_level IN ('high','critical')
ORDER BY d.risk_score DESC, d.days_to_next_deadline ASC;
```

Output on the demo data

project_id	title	team_name	course_name	risk_score	risk_level	reason	days_to_next_deadline
2	LibrarySeat: Real-Time Study Seat Reservation with Occupancy Analytics	Bravo	DBMS Mini Project	59.21	high	1 milestone(s) overdue (+12.5); 4 task(s) overdue (+16); 1 mandatory deliverable(s) missin	-12

Q2. Teams with more than N overdue tasks (JOIN, GROUP BY, HAVING, date arithmetic)

```
SELECT t.team_id, t.team_name, COUNT(*) AS overdue_tasks, MAX(CURRENT_DATE - tk.due_date) AS max_days_late
FROM tasks tk
JOIN milestones m ON m.milestone_id = tk.milestone_id
JOIN projects p   ON p.project_id = m.project_id
JOIN teams t      ON t.team_id = p.team_id
WHERE tk.due_date < CURRENT_DATE AND tk.status NOT IN ('done','verified')
GROUP BY t.team_id, t.team_name
HAVING COUNT(*) > $1
ORDER BY overdue_tasks DESC;
```

Output on the demo data

team_id	team_name	overdue_tasks	max_days_late
2	Bravo	4	25
3	Charlie	2	4

Q3. Contribution comparison inside a team (LEFT JOIN, conditional aggregation)

```
SELECT u.name,
       COUNT(*) FILTER (WHERE ce.evidence_type = 'task_completed'   AND ce.verified_flag) AS tasks_verified,
       COUNT(*) FILTER (WHERE ce.evidence_type = 'document_upload'  AND ce.verified_flag) AS docs,
       COUNT(*) FILTER (WHERE ce.evidence_type = 'feedback_resolved' AND ce.verified_flag) AS feedback_resolved,
       SUM(ce.weight) FILTER (WHERE ce.verified_flag) AS weighted_points
FROM team_members tm
JOIN projects p ON p.team_id = tm.team_id
JOIN users u ON u.user_id = tm.user_id
LEFT JOIN contribution_evidence ce ON ce.project_id = p.project_id AND ce.user_id = tm.user_id
WHERE p.project_id = $1
GROUP BY u.user_id, u.name
ORDER BY weighted_points DESC NULLS LAST;
```

Output on the demo data

name	tasks_verified	docs	feedback_resolved	weighted_points
Karthik B	2	1	0	4.00

name	tasks_verified	docs	feedback_resolved	weighted_points
Sneha Hegde	1	0	0	1.00

Q4a. Milestones that still have unresolved feedback (JOIN, GROUP BY)

```
SELECT m.milestone_id, m.title, COUNT(f.feedback_id) AS open_items
FROM milestones m
JOIN reviews r ON r.milestone_id = m.milestone_id
JOIN feedback_items f ON f.review_id = r.review_id
WHERE m.project_id = $1 AND f.resolution_status <> 'verified'
GROUP BY m.milestone_id, m.title;
```

Output on the demo data

milestone_id	title	open_items
6	Proposal approval	1

Q4b. Approved milestones with no unresolved feedback (correlated subquery, NOT EXISTS)

```
SELECT m.milestone_id, m.title
FROM milestones m
WHERE m.status = 'approved' AND NOT EXISTS (
  SELECT 1 FROM reviews r JOIN feedback_items f ON f.review_id = r.review_id
  WHERE r.milestone_id = m.milestone_id AND f.resolution_status <> 'verified');
```

Output on the demo data

milestone_id	title
11	Proposal approval
17	Requirements (SRS)
16	Proposal approval
1	Proposal approval

Q5. Projects lacking mandatory deliverable types (anti-join, GROUP BY, HAVING)

```
SELECT p.project_id, p.title, COUNT(*) AS missing_types
FROM projects p
JOIN milestones m ON m.project_id = p.project_id
JOIN milestone_required_deliverables r ON r.milestone_id = m.milestone_id
LEFT JOIN deliverables d ON d.milestone_id = m.milestone_id
      AND d.deliverable_type = r.deliverable_type AND d.status IN ('submitted','accepted')
WHERE d.deliverable_id IS NULL AND m.due_date <= CURRENT_DATE + 7
GROUP BY p.project_id, p.title
HAVING COUNT(*) >= 1
ORDER BY missing_types DESC;
```

Output on the demo data

project_id	title	missing_types
2	LibrarySeat: Real-Time Study Seat Reservation with Occupancy Analytics	1
4	AgriAdvisor: Crop Disease Reporting and Advisory Portal	1

Q6. Similar proposals (pgvector cosine distance with a metadata filter)

```
SELECT p.project_id, p.title, e.section_type, e.chunk_text,
       1 - (e.embedding <=> $1::vector) AS cosine_similarity
```

```
FROM project_embeddings e
JOIN projects p ON p.project_id = e.project_id
WHERE e.is_current AND e.course_id = $2 AND e.project_id <> $3
ORDER BY e.embedding <=> $1::vector
LIMIT $4;
```

Output on the demo data

project_id	title	section_type	chunk_text	cosine_similarity
1	CampusConnect: Event Management and Volunteer Coordination Portal	abstract	CampusConnect is a web portal that lets student clubs create campus events, publish schedu	0.7063738318197159
1	CampusConnect: Event Management and Volunteer Coordination Portal	title	CampusConnect: Event Management and Volunteer Coordination Portal	0.6190932109133739
1	CampusConnect: Event Management and Volunteer Coordination Portal	objectives	1. Provide one place to create and approve campus events. 2. Automate volunteer slot alloc	0.575902819633488

Q7. Latest risk per project (DISTINCT ON)

```
SELECT DISTINCT ON (project_id) project_id, risk_score, risk_level, calculated_at
FROM risk_assessments ORDER BY project_id, calculated_at DESC;
```

Output on the demo data

project_id	risk_score	risk_level	calculated_at
1	4.74	low	2026-10-26
2	59.21	high	2026-10-26
3	27.17	moderate	2026-10-26
4	25.06	moderate	2026-10-26

Q8. Risk trend of one project (GROUP BY a derived date)

```
SELECT calculated_at::date AS day, MAX(risk_score) AS score FROM risk_assessments
WHERE project_id = $1 GROUP BY 1 ORDER BY 1;
```

Output on the demo data

day	score
2026-10-03	8.00
2026-10-10	25.00
2026-10-17	43.00
2026-10-25	59.21

Q9. Guide workload (JOIN, GROUP BY, FILTER aggregate)

```
SELECT u.name, COUNT(DISTINCT ga.project_id) AS projects,
       COUNT(DISTINCT ga.project_id) FILTER (WHERE lr.risk_level IN ('high','critical')) AS high_risk
FROM guide_allocations ga JOIN users u ON u.user_id = ga.user_id
LEFT JOIN v_faculty_risk_dashboard lr ON lr.project_id = ga.project_id
WHERE ga.allocation_role = 'guide' GROUP BY u.user_id, u.name ORDER BY projects DESC;
```

Output on the demo data

name	projects	high_risk
Dr. Nagasundari S	3	0
Prof. Meera Rao	2	1
Prof. Kiran Shetty	1	0

Q10. Unapproved milestones past due, by course

```
SELECT c.course_name, t.team_name, m.title, m.due_date, CURRENT_DATE - m.due_date AS days_overdue
FROM milestones m JOIN projects p ON p.project_id = m.project_id
JOIN teams t ON t.team_id = p.team_id JOIN courses c ON c.course_id = p.course_id
WHERE m.status <> 'approved' AND m.due_date < CURRENT_DATE ORDER BY days_overdue DESC;
```

Output on the demo data

course_name	team_name	title	due_date	days_overdue
DBMS Mini Project	Bravo	Requirements (SRS)	2026-10-13	12
DBMS Mini Project	Charlie	Requirements (SRS)	2026-10-20	5

Q11. Students with more open feedback than average (subquery in HAVING)

```
SELECT u.name, COUNT(*) AS open_items FROM feedback_items f JOIN users u ON u.user_id = f.assigned_user_id
WHERE f.resolution_status IN ('open','in_progress','reopened')
GROUP BY u.user_id, u.name
HAVING COUNT(*) > (SELECT AVG(c) FROM (SELECT COUNT(*) c FROM feedback_items
                                       WHERE resolution_status IN ('open','in_progress','reopened')
                                       GROUP BY assigned_user_id) s);
```

Output on the demo data

name	open_items
Pooja R	2
Vikram T	2

Q12. Full-text search over proposals (GIN index)

```
SELECT project_id, title,
       ts_rank(to_tsvector('english', coalesce(title,'') || ' ' || coalesce(abstract,'') || ' ' ||
coalesce(problem_statement,'')), q) AS rank
FROM projects, plainto_tsquery('english', $1) q
WHERE to_tsvector('english', coalesce(title,'') || ' ' || coalesce(abstract,'') || ' ' || coalesce(problem_statement,'')) @@
q
ORDER BY rank DESC;
```

Output on the demo data

project_id	title	rank
1	CampusConnect: Event Management and Volunteer Coordination Portal	0.08654518
5	CollegeEvents Hub: Campus Event Management and Volunteer Coordination Platform	0.08654518
6	FestFlow: Event Registration System	0.082745634

Q13. Feedback turnaround time per guide

```
SELECT cu.name AS guide, COUNT(*) AS items,
       ROUND(AVG(EXTRACT(EPOCH FROM (f.resolved_at - f.created_at)) / 86400)::numeric, 1) AS avg_days_to_resolve
FROM feedback_items f JOIN users cu ON cu.user_id = f.created_by
WHERE f.resolved_at IS NOT NULL GROUP BY cu.user_id, cu.name;
```

Output on the demo data

guide	items	avg_days_to_resolve
Prof. Meera Rao	3	9.0

guide	items	avg_days_to_resolve
Dr. Nagasundari S	2	5.5

Q14. Projects whose risk is worse than a week ago (self-join on snapshots)

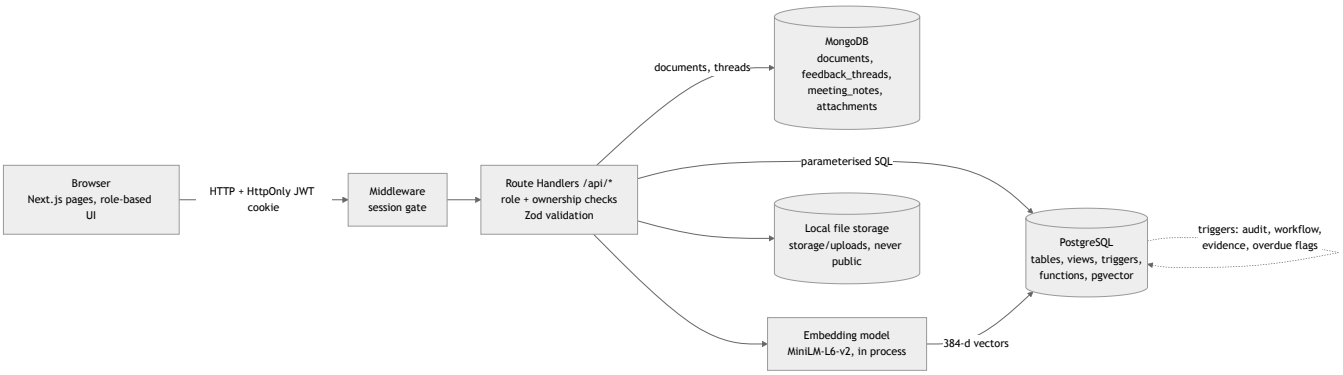
```
SELECT n.project_id, o.risk_score AS last_week, n.risk_score AS now
FROM (SELECT DISTINCT ON (project_id) * FROM risk_assessments ORDER BY project_id, calculated_at DESC) n
JOIN (SELECT DISTINCT ON (project_id) * FROM risk_assessments WHERE calculated_at < now() - interval '7 days'
      ORDER BY project_id, calculated_at DESC) o USING (project_id)
WHERE n.risk_score > o.risk_score;
```

Output on the demo data

project_id	last_week	now
2	43.00	59.21
3	19.00	27.17
4	24.00	25.06

DML coverage. INSERT, UPDATE, DELETE and SELECT are exercised by the application (create proposal / task / deliverable / feedback; controlled status updates; deleting a draft deliverable or a task assignment, both audited) and by negative tests: an INSERT that violates a foreign key, an UPDATE with an illegal transition, a DELETE blocked by `ON DELETE RESTRICT`.

13. System architecture and security design



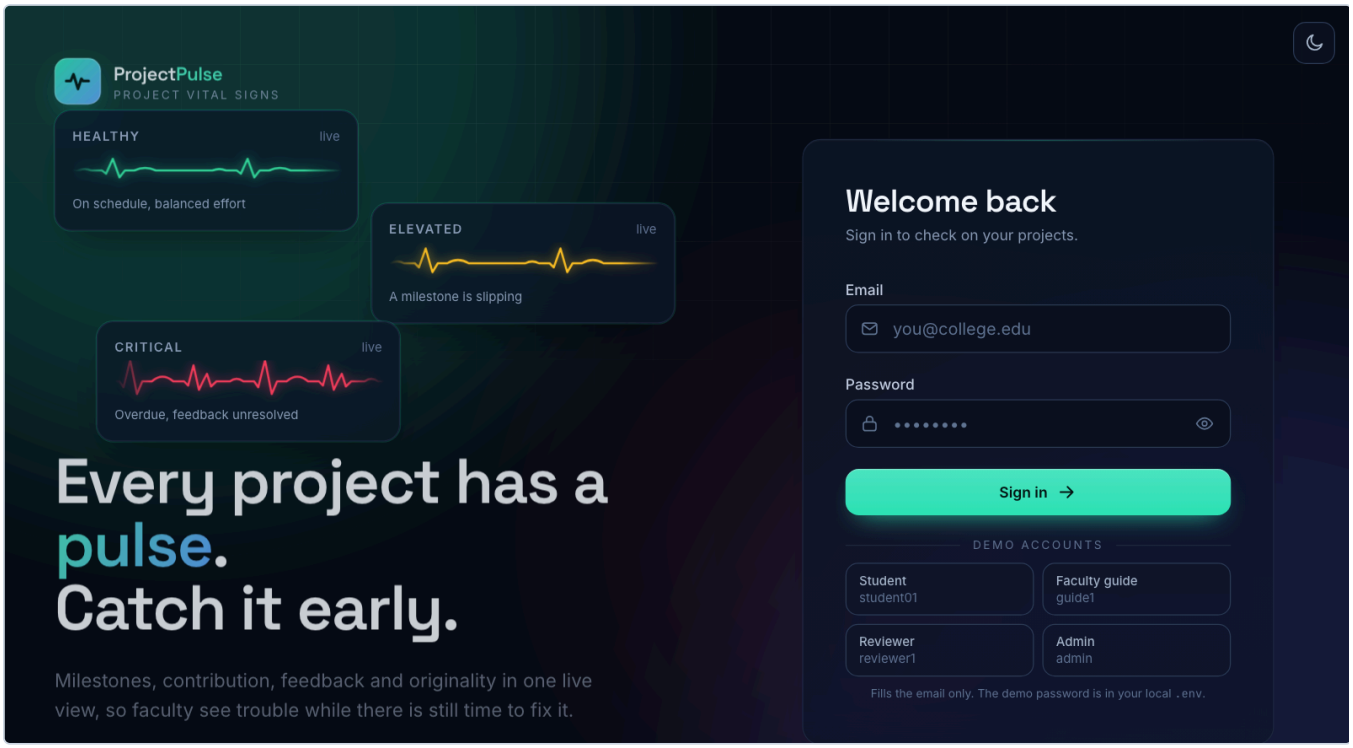
Layers. The browser talks only to Next.js. Middleware blocks anonymous users; every Route Handler then authenticates, authorises by **role and ownership loaded from the database** (never from the browser), validates input with Zod, runs parameterised SQL (in a transaction when it writes more than once) and returns consistent JSON errors with friendly messages (database error codes are mapped, raw SQL errors never leak). Integrity rules that must hold for any client (CHECKs, foreign keys, triggers) live in the database; orchestration that needs the current user, files or two databases lives in the handlers.

Concern	Design
Passwords	bcrypt (cost 12), salted, never logged or returned; minimum 8 characters with a letter and a digit
Sessions	JWT (HS256) in an `HttpOnly`, `SameSite=Lax` cookie, 8-hour expiry; account status and roles re-read from the database on every request, so a suspended user is locked out immediately
Authorisation	Role plus relationship (team member, allocated guide / reviewer, admin) on every route; a non-participant gets 404 so the existence of other teams' projects is not revealed
Brute force	Five failures lock the account for 15 minutes; one generic message for unknown users and wrong passwords
Injection	Parameterised SQL only; dynamic `ORDER BY` from a whitelist; React escapes output; CSV cells that start with `= + - @` are neutralised
CSRF	SameSite cookies plus an `Origin` check on state-changing requests
Uploads	Extension allow-list, size limit, content check (magic bytes), random storage names outside `public/`, SHA-256 checksum, served only through an authorised route with `attachment` and `nosniff`
Secrets	Only in `.env` (git-ignored); `.env.example` holds placeholders; a test scans the repository for leaked values
Database	Least-privilege role `pp_app`; append-only audit and proposal history; every status change audited with the acting user

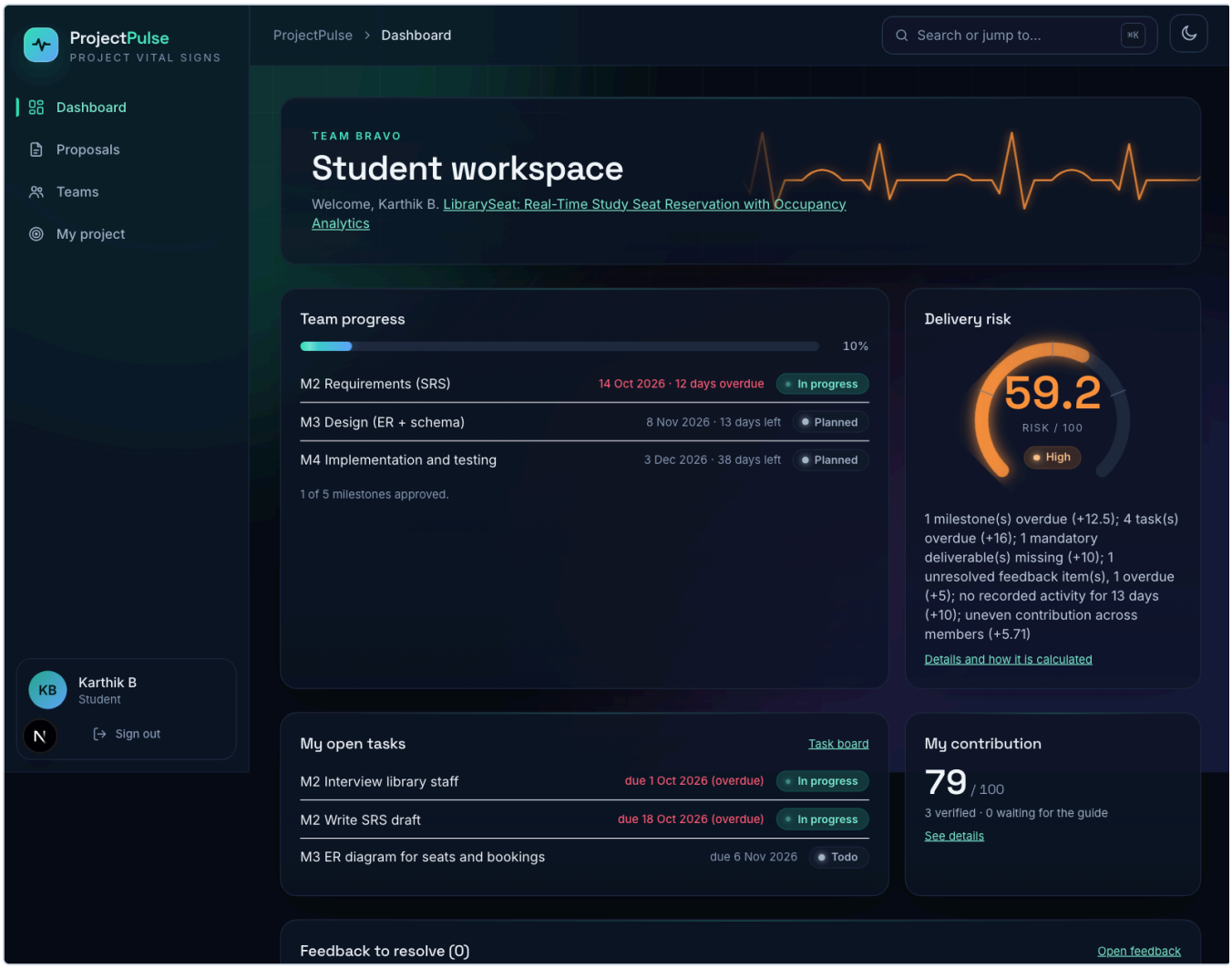
Concern	Design
Headers	`X-Content-Type-Options`, `Referrer-Policy`, `X-Frame-Options`

14. Application screenshots

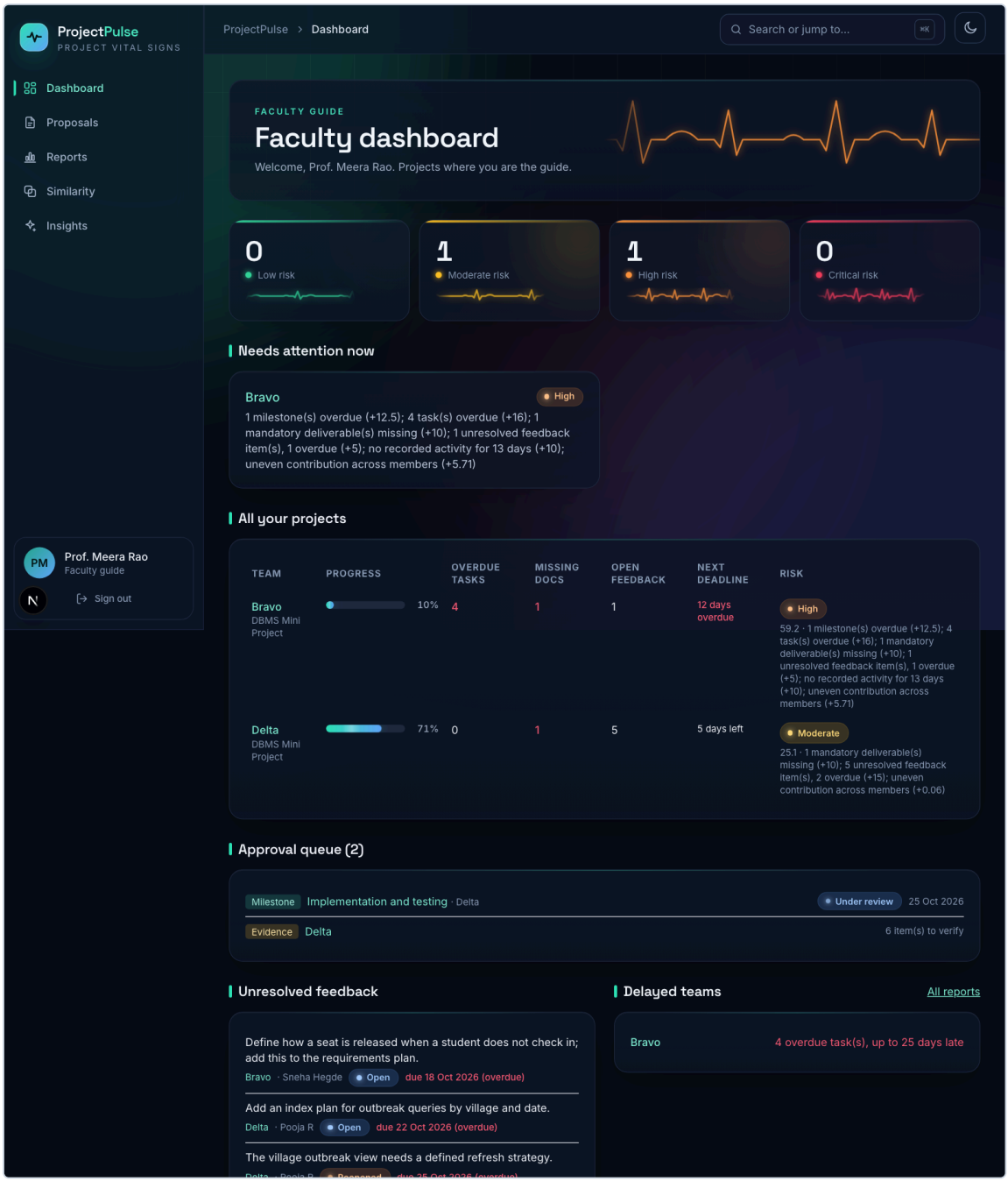
All screenshots are of the running application with the seeded demo data, taken by `npm run docs:screenshots` (real sessions through the login API, no hand editing). The six demo teams show different situations: **Alpha** healthy, **Bravo** delayed (high risk), **Charlie** unequal contribution, **Delta** unresolved feedback and a missing test report, **Echo** a near-duplicate of Alpha awaiting similarity review, **Foxtrot** another course.



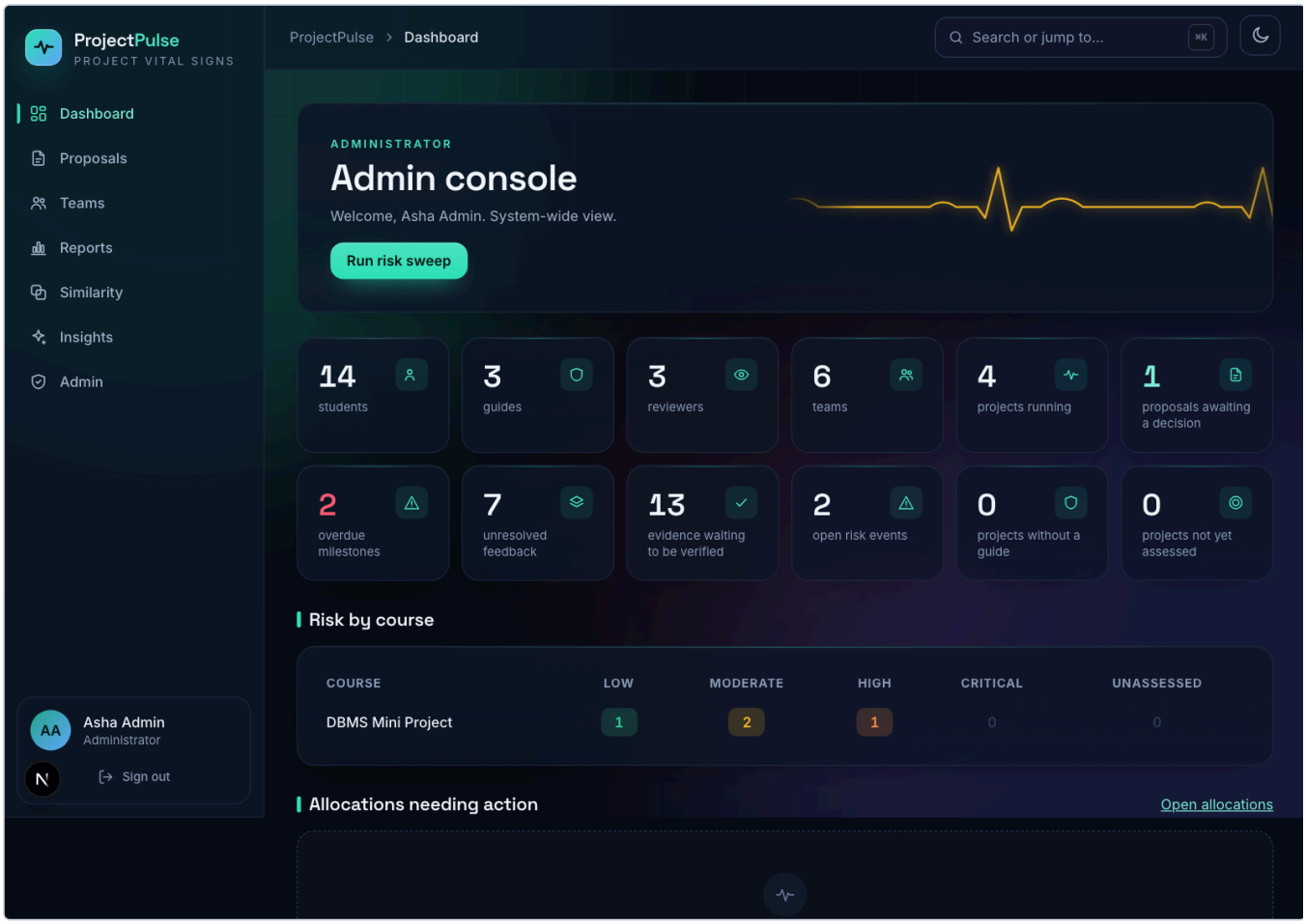
S-01 Login



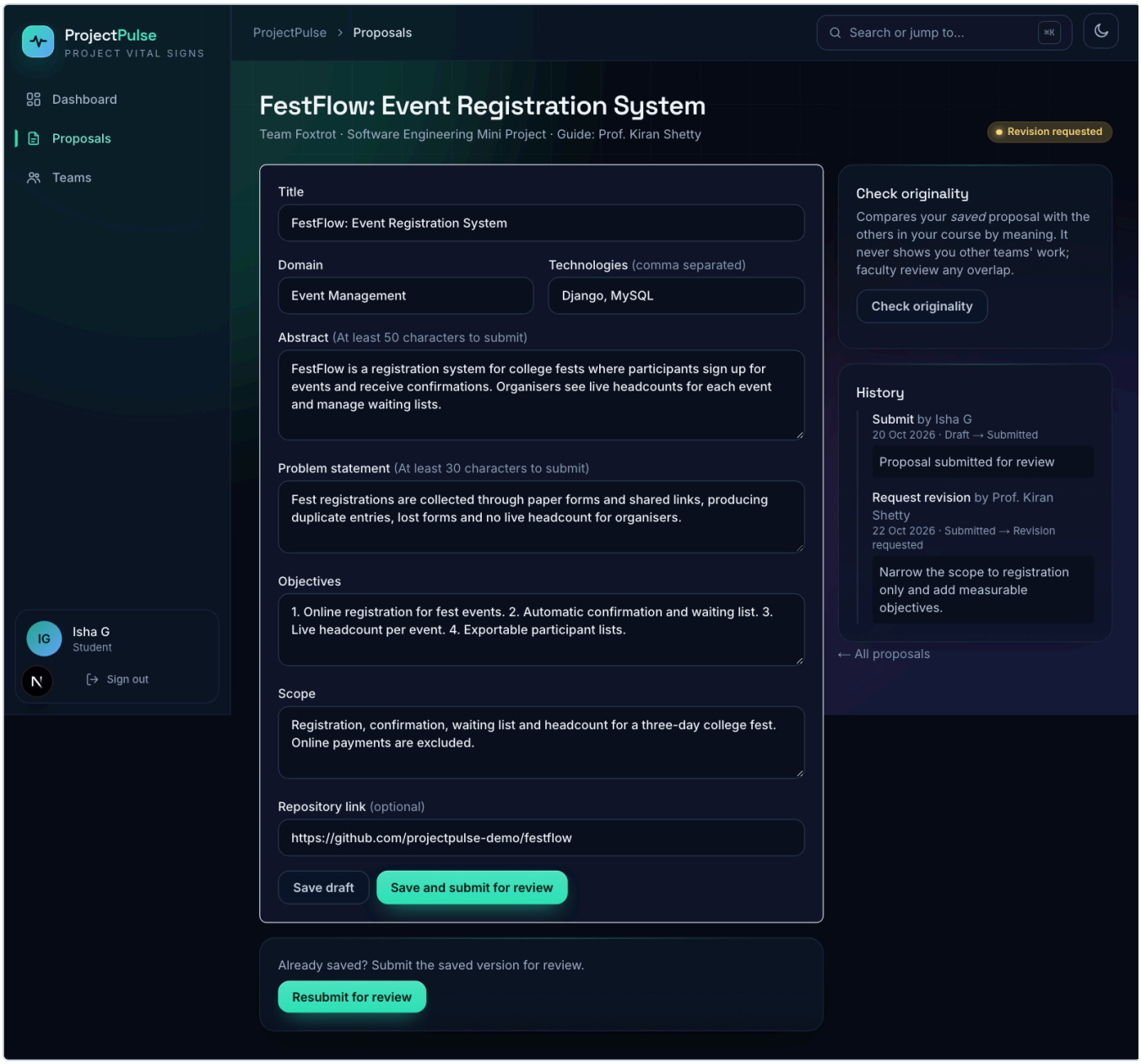
S-02 Student dashboard: team progress, deadlines, my tasks, feedback, contribution, risk



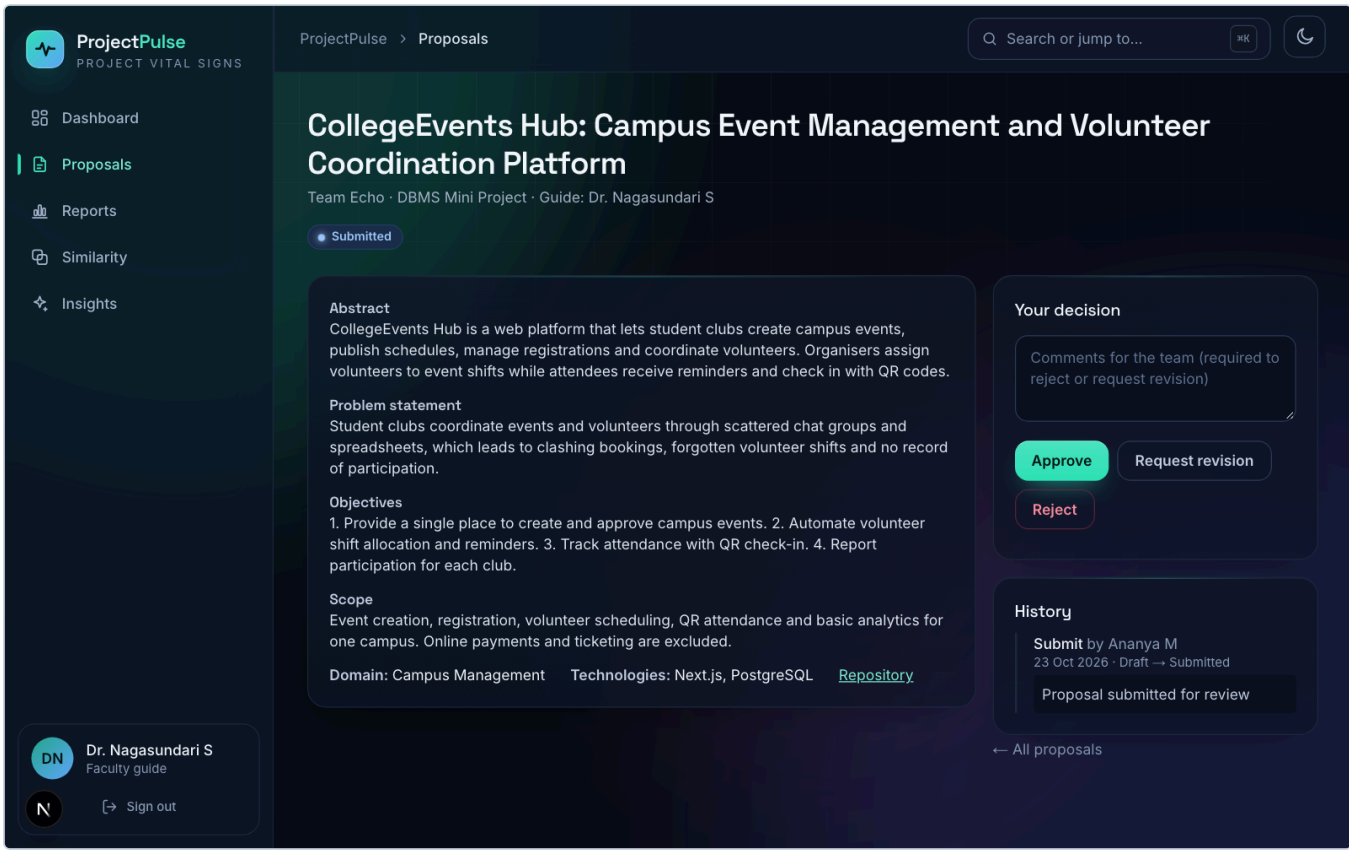
S-03 Faculty dashboard: risk cards, high-risk projects, approval queue, unresolved feedback, delayed teams



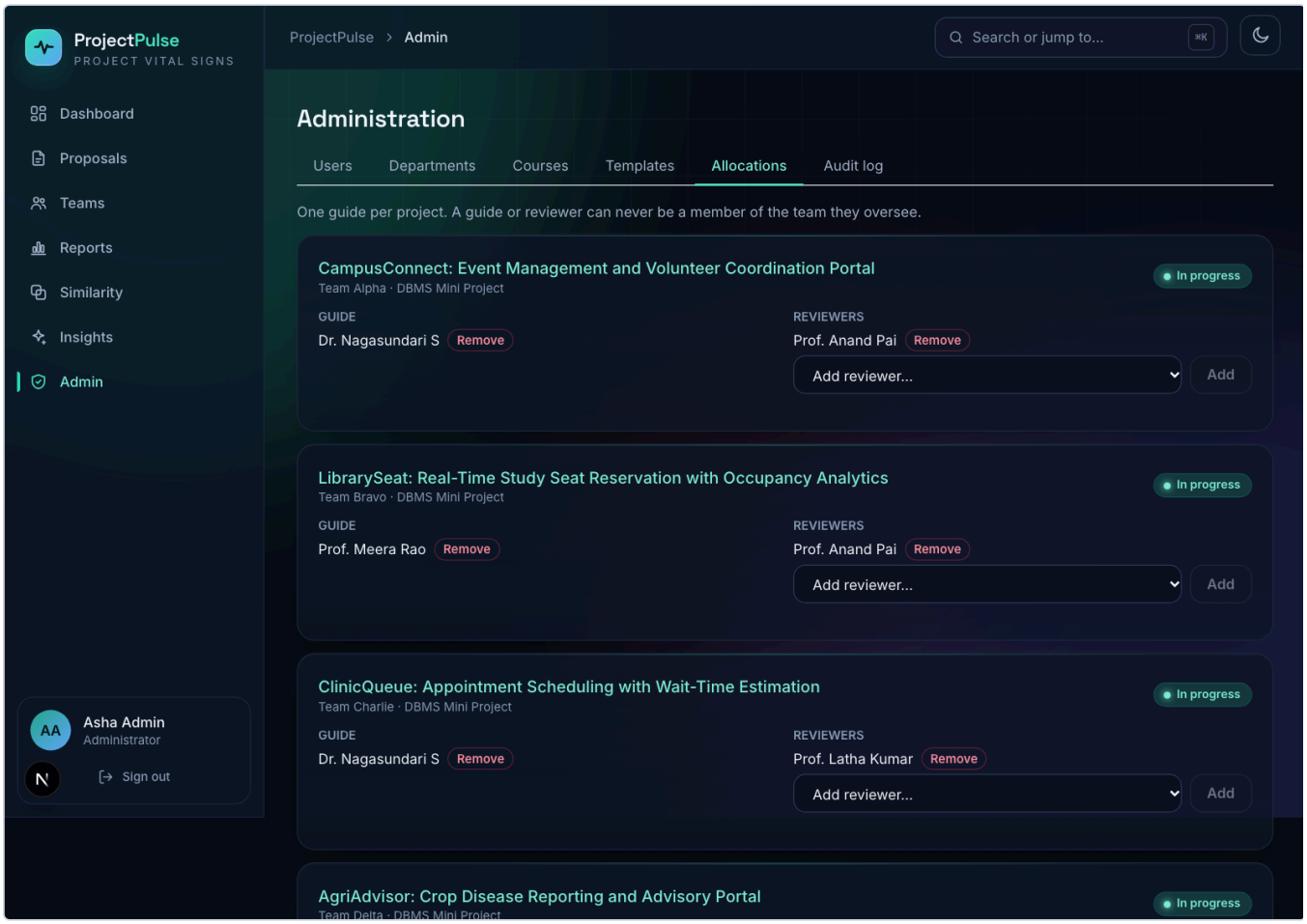
S-04 Admin console: system totals and risk by course



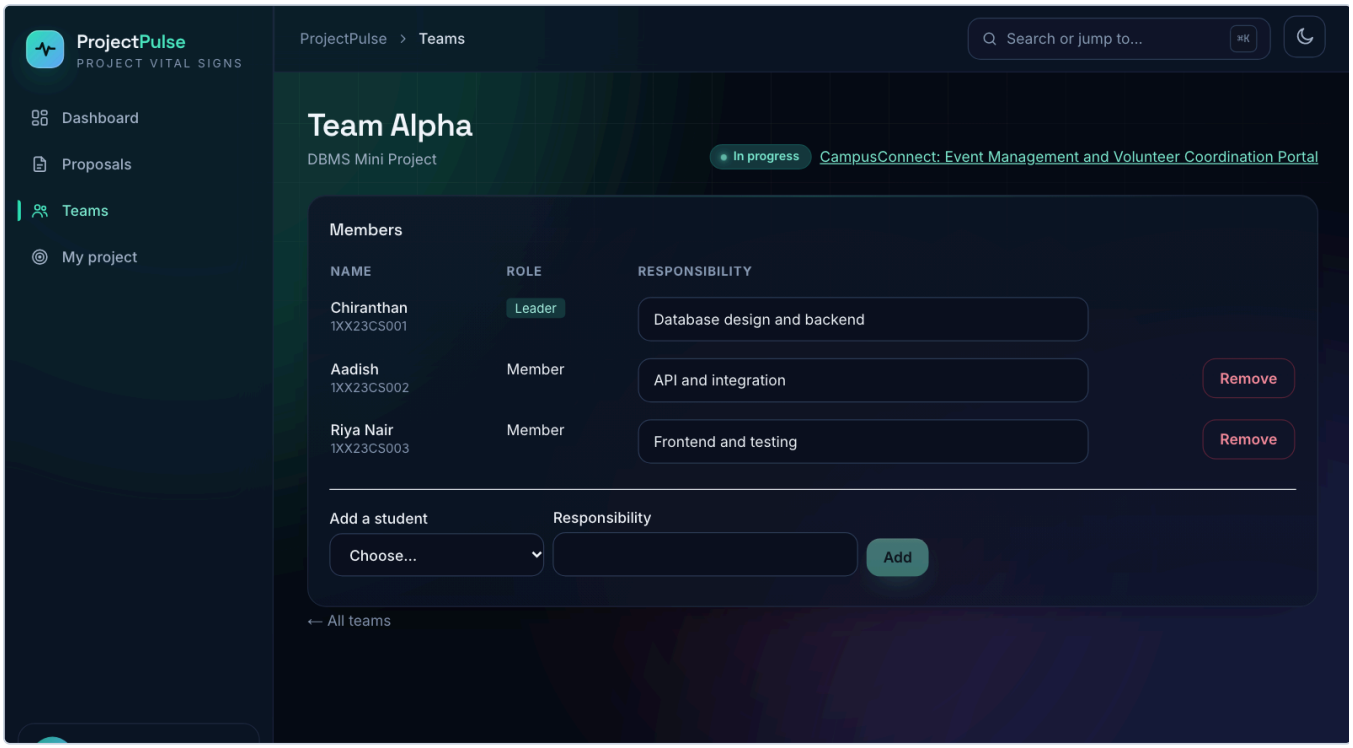
S-05 Proposal form (revision requested) with "Check originality"



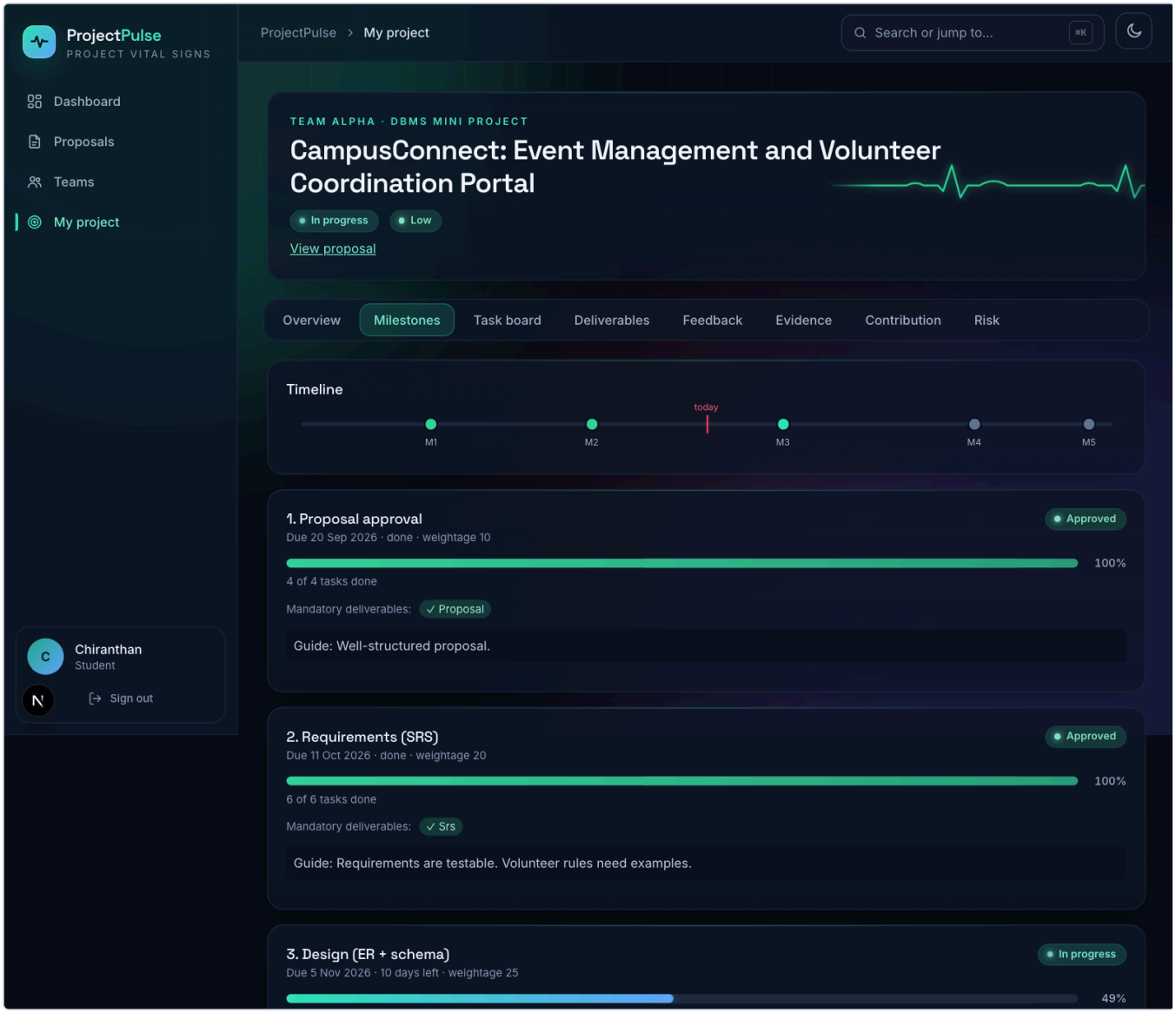
S-06 Proposal decision panel and immutable history



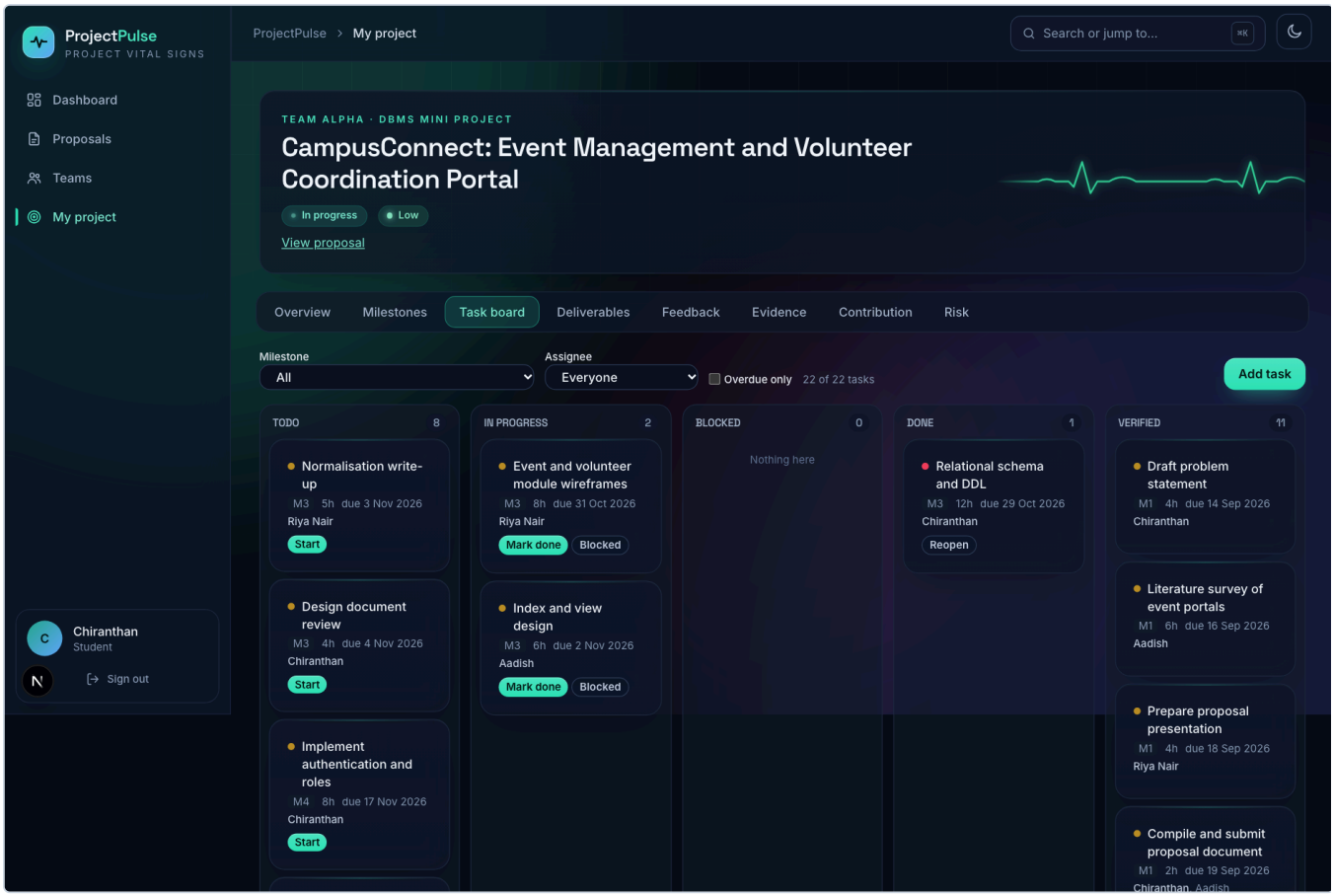
S-07 Guide and reviewer allocation (admin)



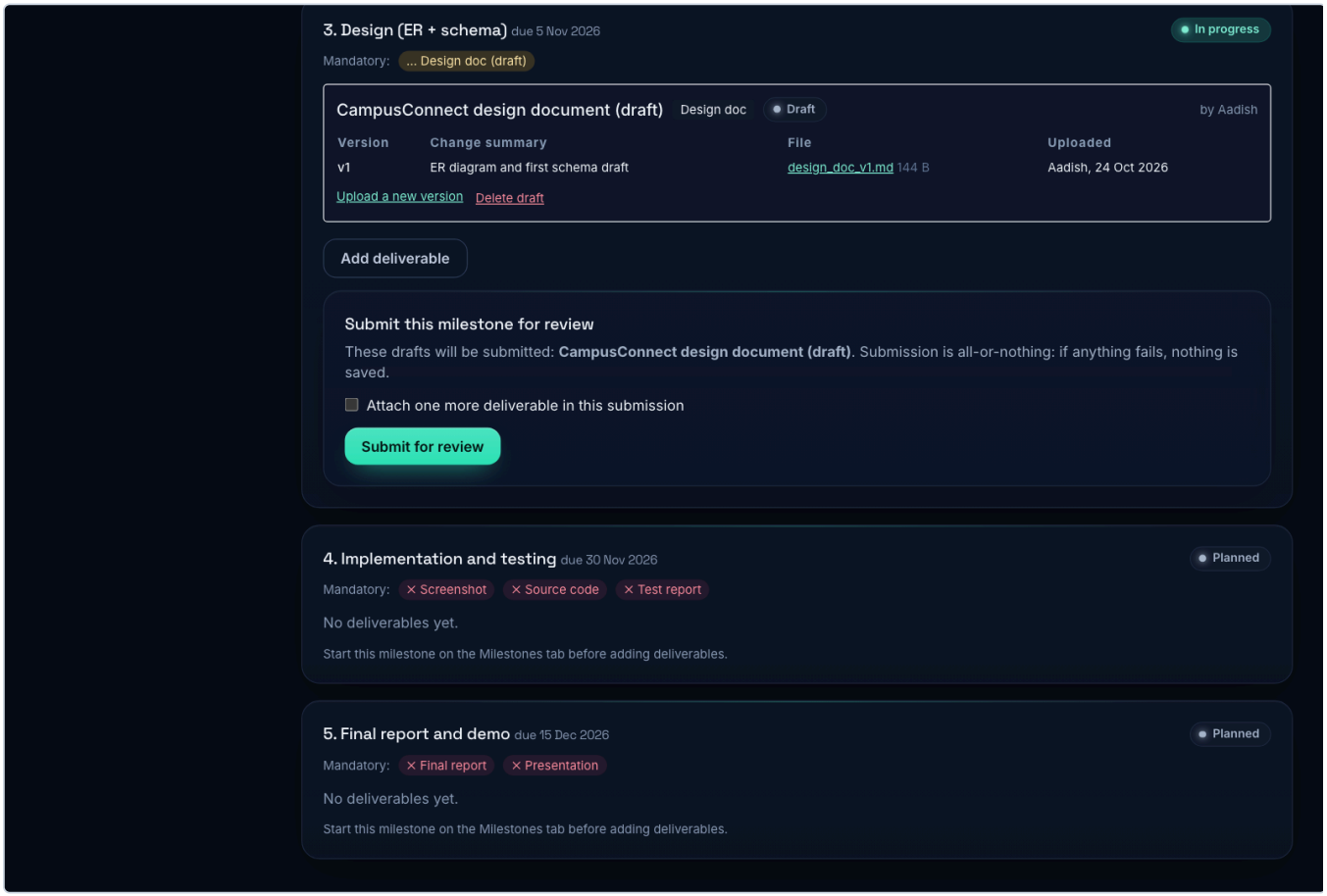
S-07b Team members and responsibilities



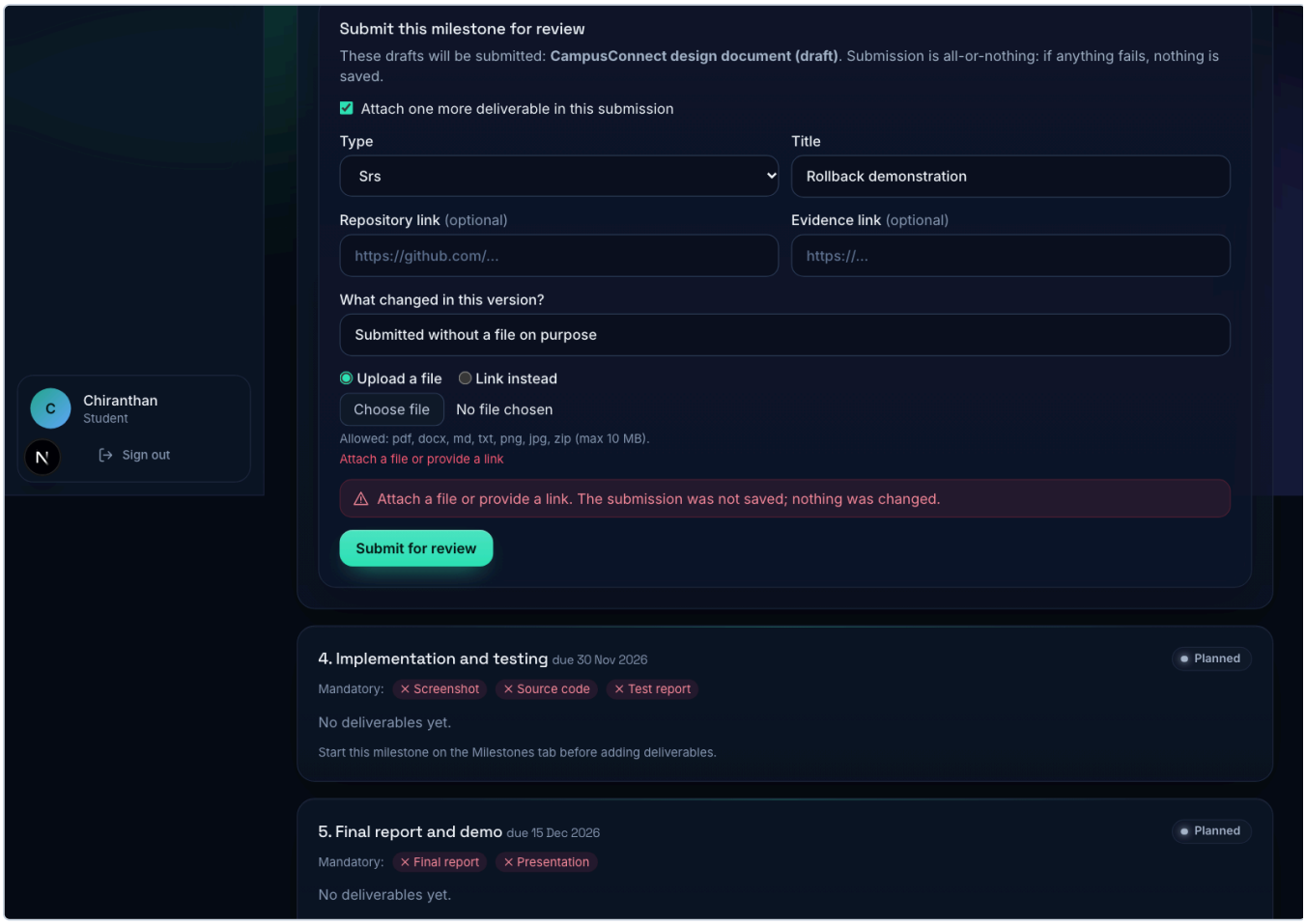
S-08 Milestone timeline, progress and mandatory deliverables



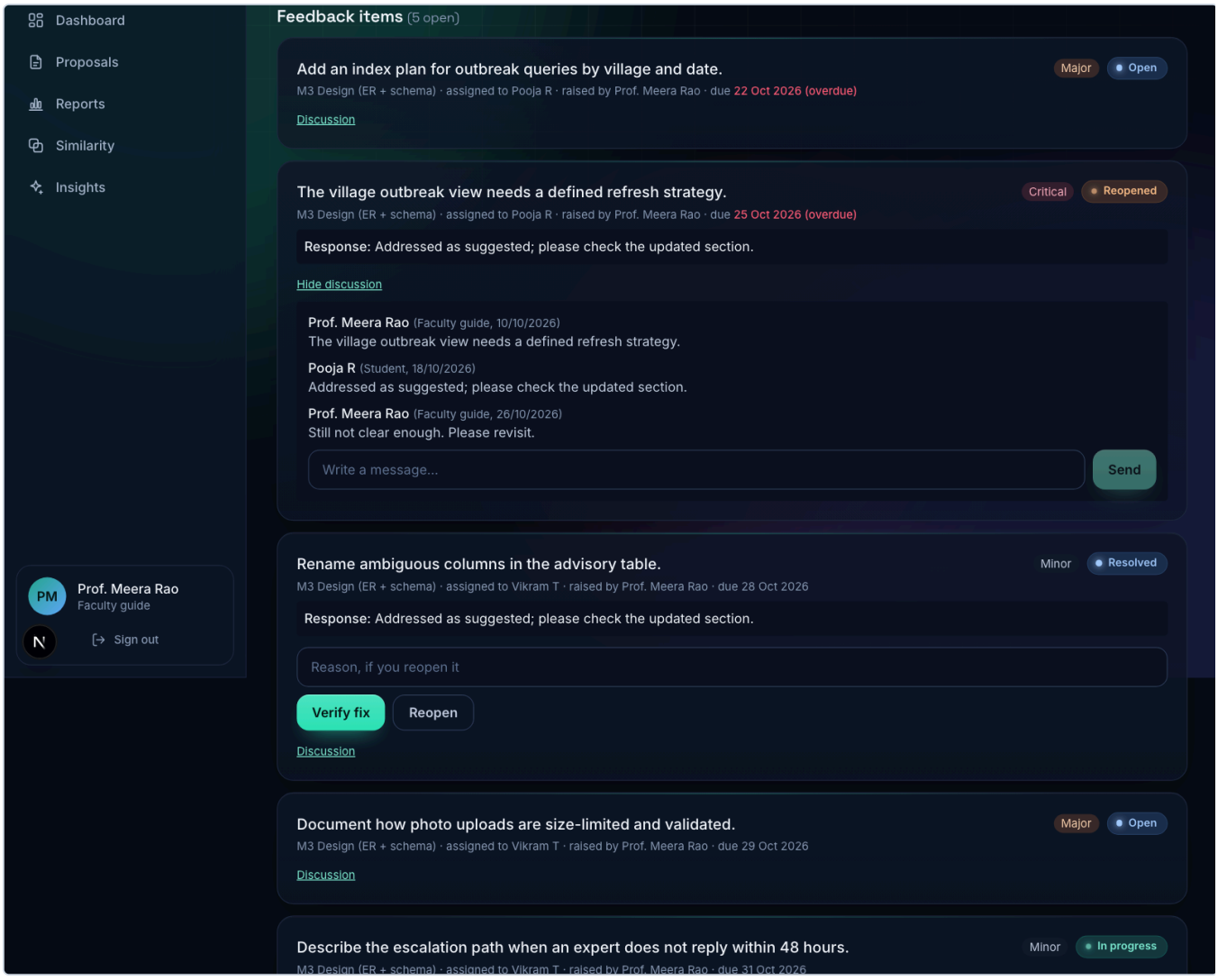
S-09 Task Kanban board (todo, in progress, blocked, done, verified) with filters



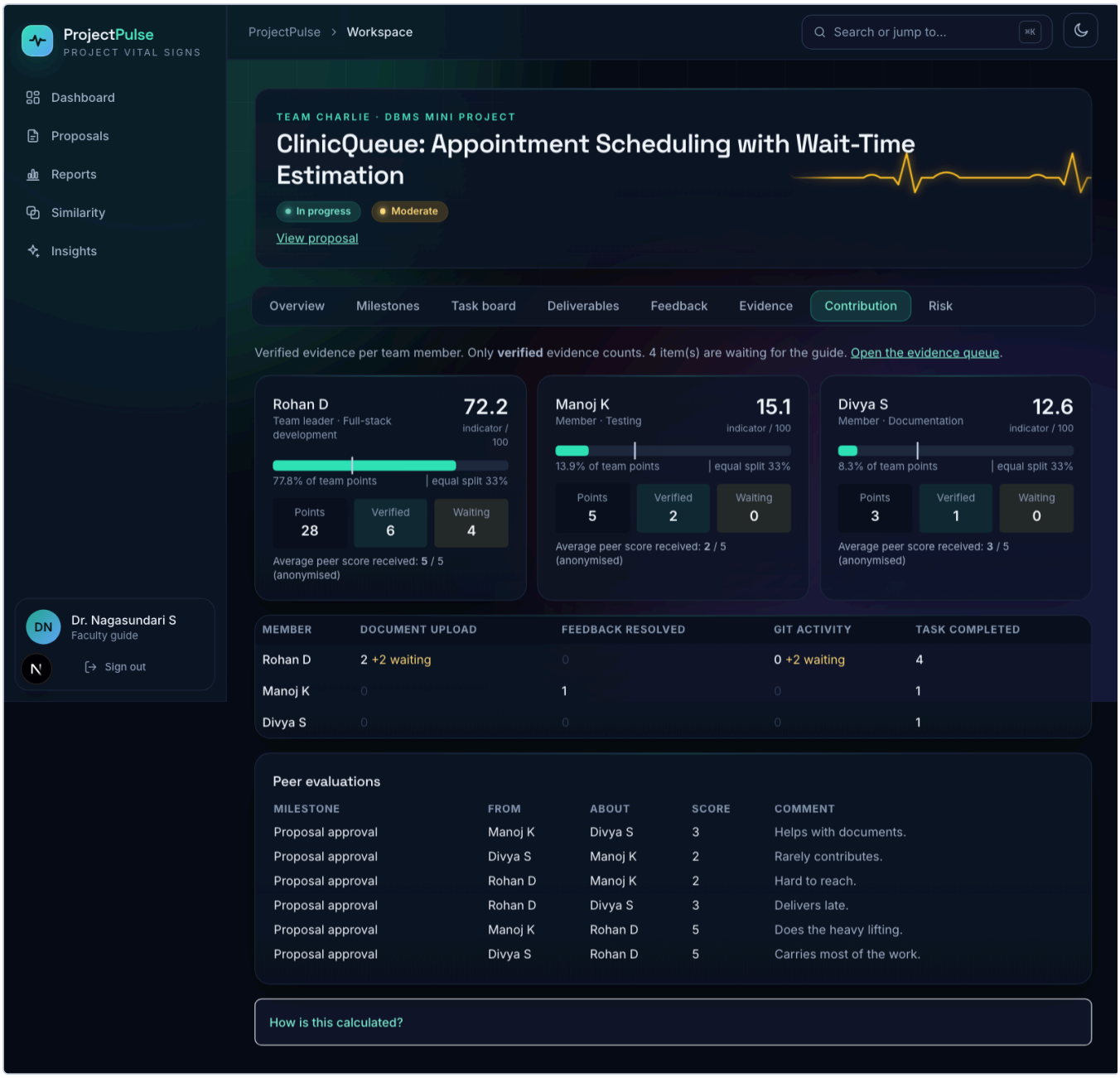
S-10 Deliverables with version history and the submission panel



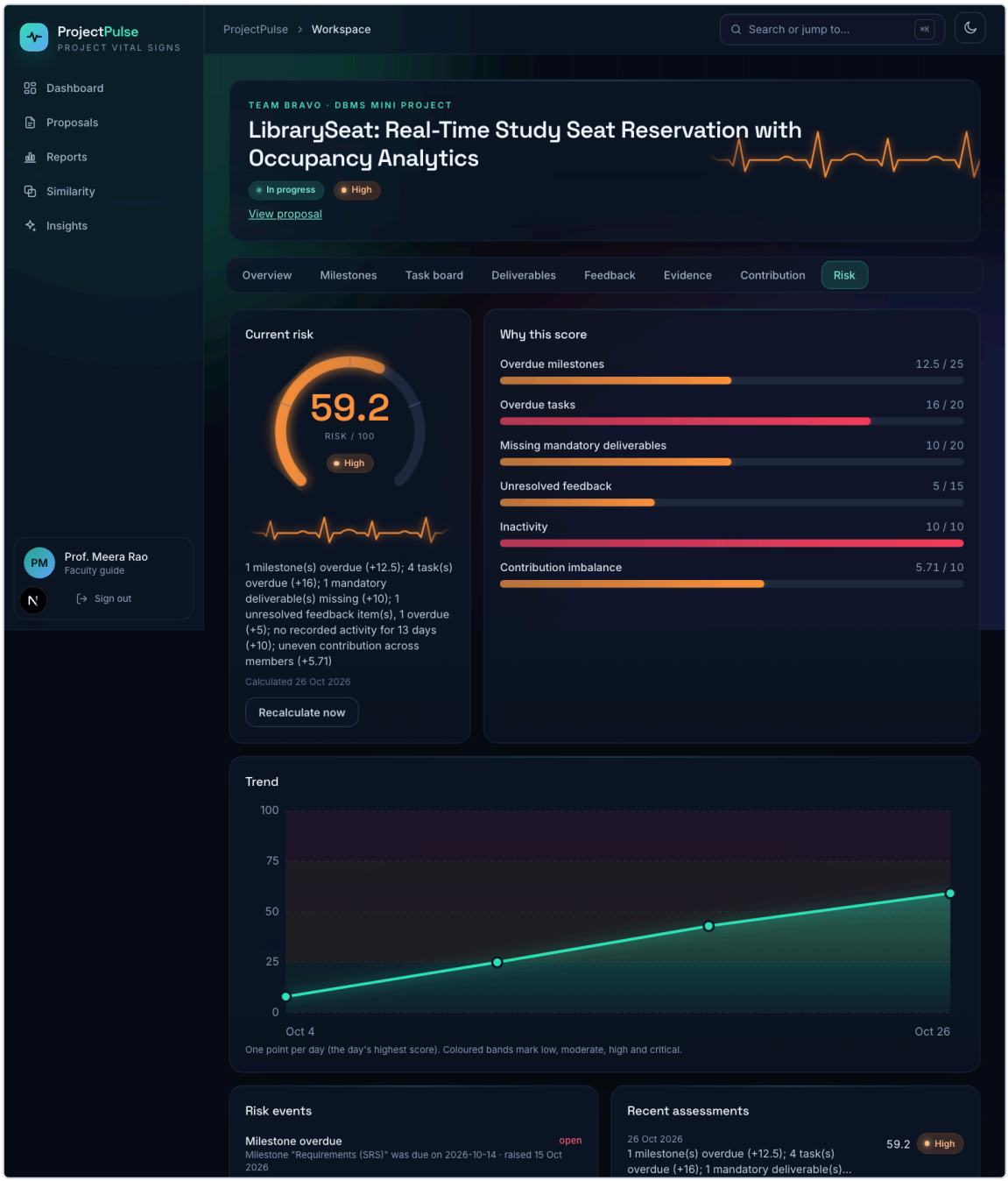
S-11 A failed submission: the user is told nothing was saved



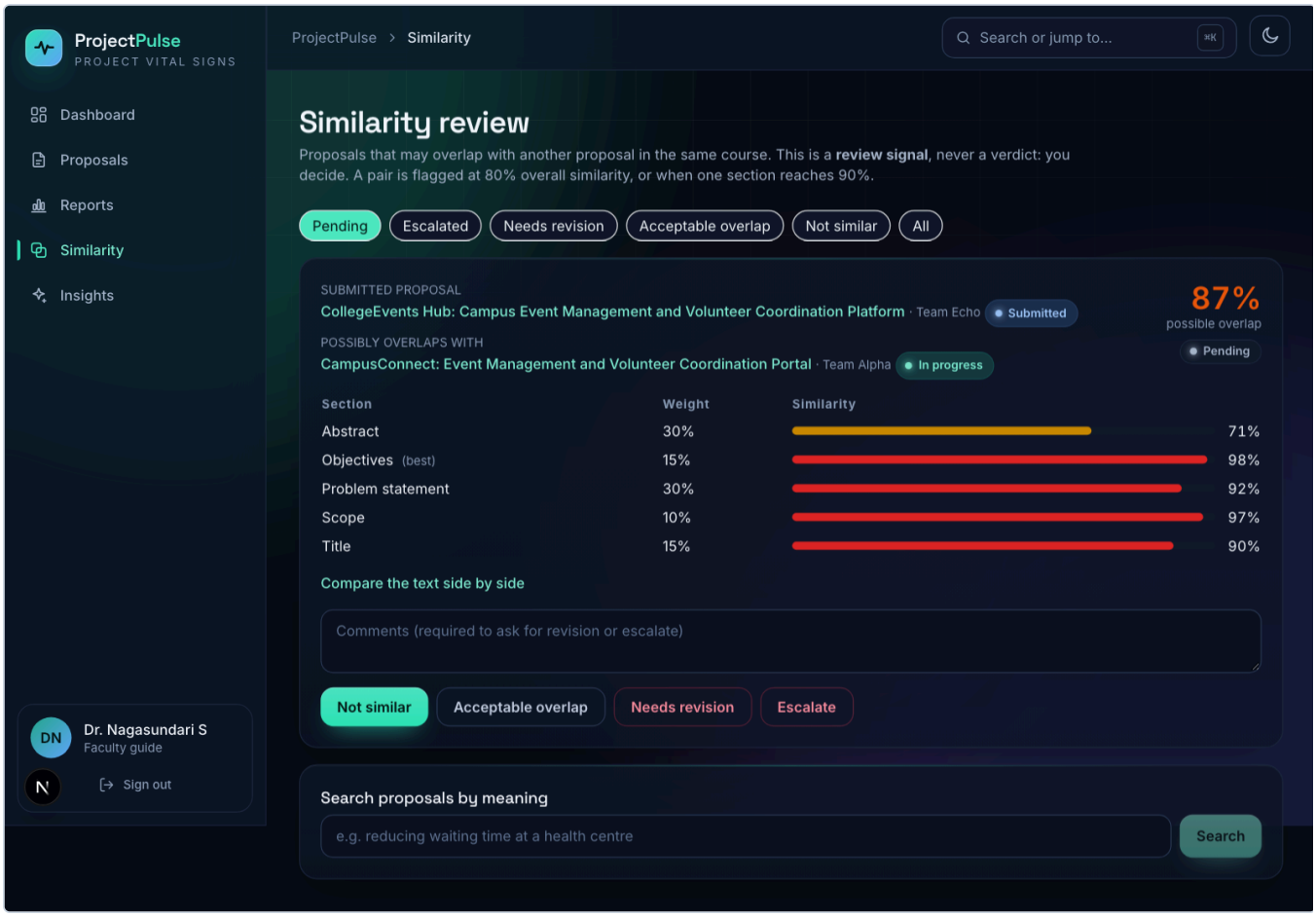
S-12 Feedback workflow with the MongoDB discussion thread



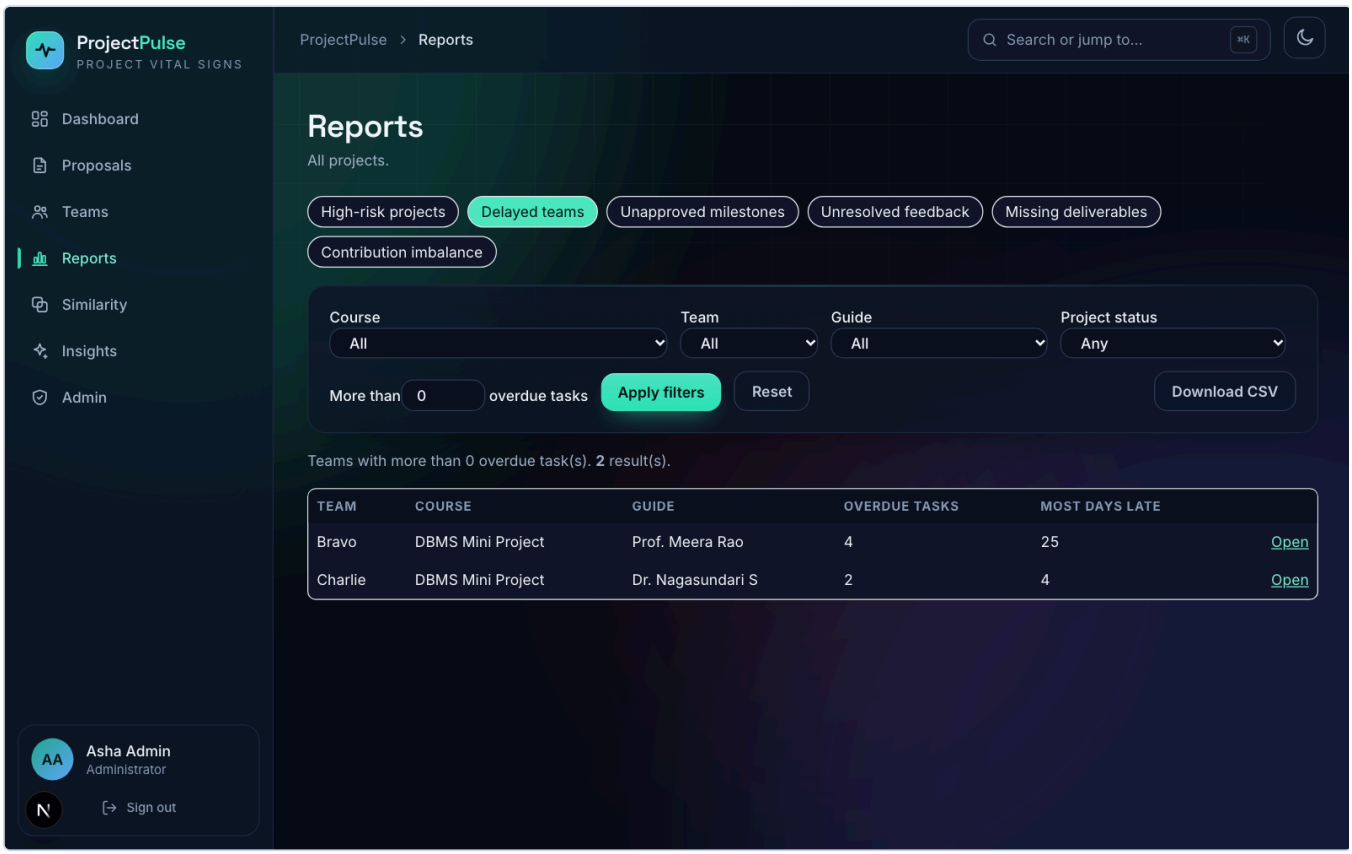
S-13 Contribution view (Charlie): one member carries most of the verified evidence



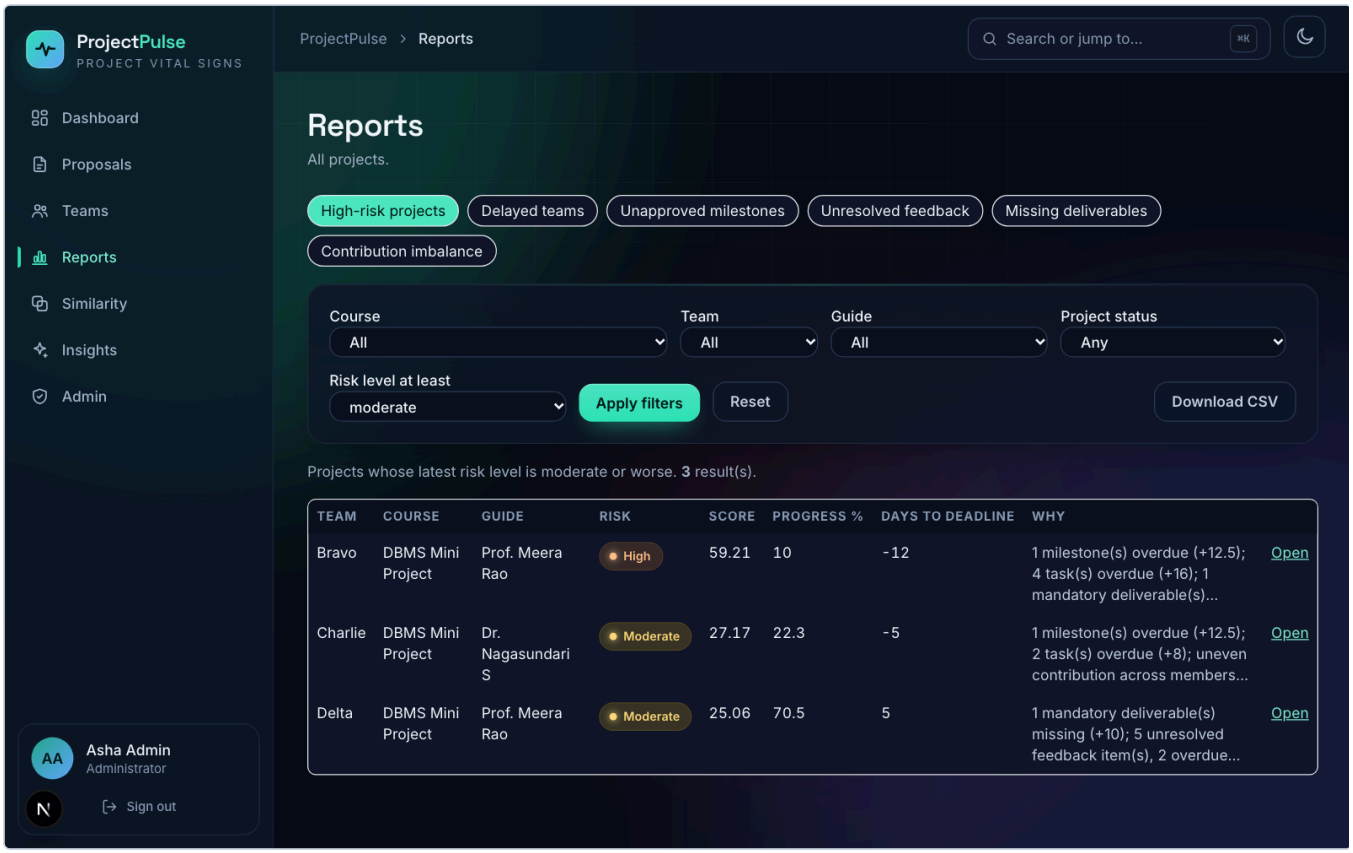
S-14 Risk detail: score, factor breakdown, trend, events



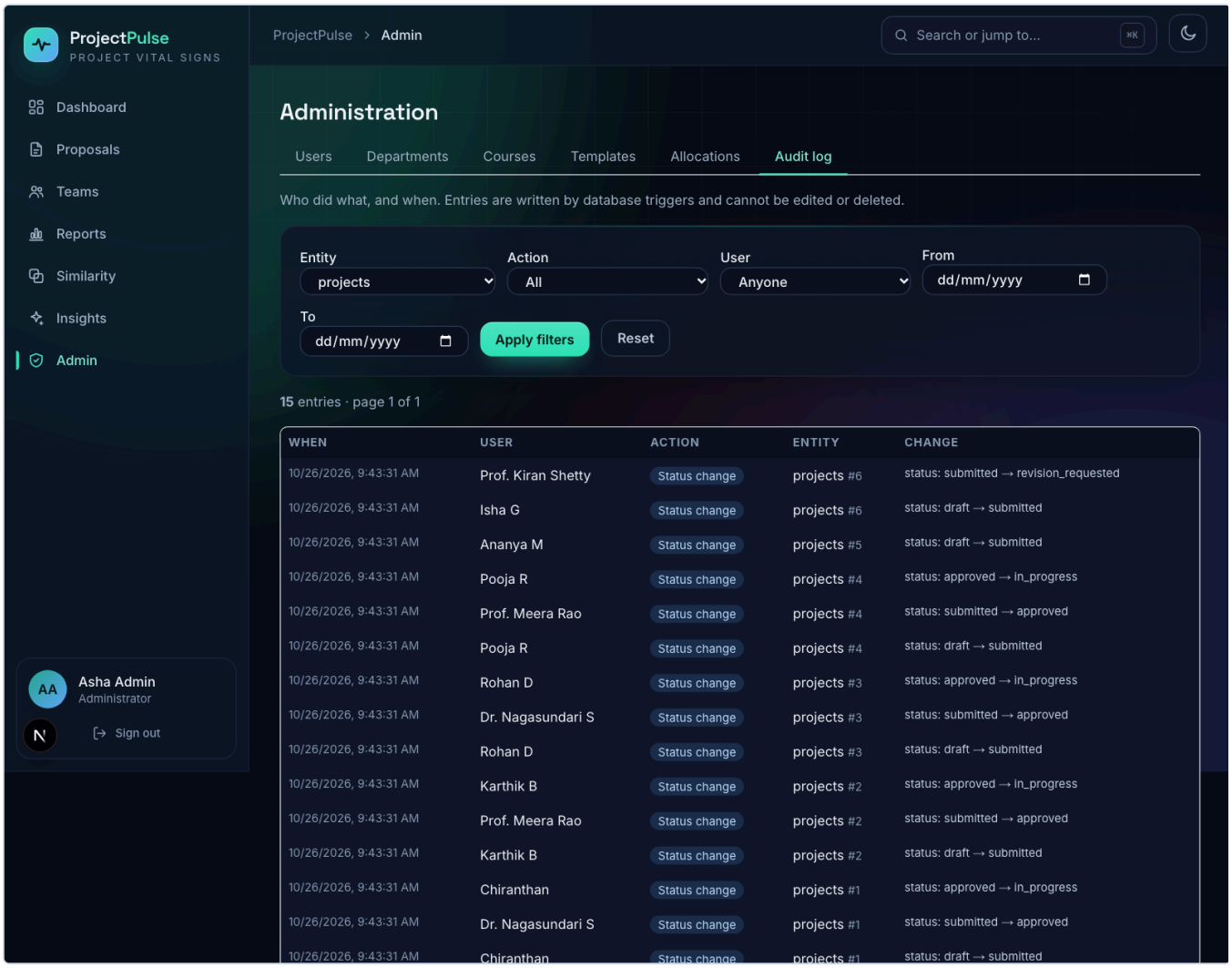
S-15 Similarity review: score, per-section similarity, decision form



S-16 Reports with filters (delayed teams)



S-16b Reports: high-risk projects with the risk level filter



S-17 Audit log with filters

```
$ npm run db:reset

> projectpulse@0.1.0 db:reset
> tsx scripts/reset.ts
Dropping schema projectpulse ...
migrating [████████████▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 5/14  006_evidence_risk_similarity.sql
migrating [████████████████████████████████] 14/14  014_reference_data.sql
Applied 14 migration(s).
```

S-18 Migrations with a progress bar

```
$ npm run db:seed

> projectpulse@0.1.0 db:seed
> tsx scripts/seed.ts
seeding [████▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 4/33 users
seeding [████████▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 8/33 users
seeding [████████████▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 12/33 users
seeding [████████████████▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒] 16/33 users
seeding [████████████████████▒▒▒▒▒▒▒▒▒▒▒▒] 20/33 users
seeding [████████████████████████████▒▒▒▒] 32/33 Mongo notes
MongoDB: 11 documents, 10 feedback threads, 3 meeting notes.
dtype not specified for "model". Using the default dtype (fp32) for this device (cpu).
embeddings [████████████████████████████████] 8/8 screening FestFlow: Event Registration
Embeddings: 6 proposal(s) embedded, 1 pair(s) flagged for faculty review.
Faculty risk dashboard (latest assessment):
┌─────────┬─────────────┬────────────────┬──────────────┬───────────────┬──────────────────┬──────────────────────┬──────────────┬──────────────┐
│ (index) │ team_name   │ project_status │ progress_pct │ overdue_tasks │ missing_evidence │ unresolved_feedback  │ risk_score   │ risk_level   │
├─────────┼─────────────┼────────────────┼──────────────┼───────────────┼──────────────────┼──────────────────────┼──────────────┼──────────────┤
│ 0       │ 'Bravo'     │ 'in_progress'  │ '10.0'       │ '4'           │ '1'              │ '1'                  │ '59.21'      │ 'high'       │
│ 1       │ 'Charlie'   │ 'in_progress'  │ '22.3'       │ '2'           │ '0'              │ '0'                  │ '27.17'      │ 'moderate'   │
│ 2       │ 'Delta'     │ 'in_progress'  │ '70.5'       │ '0'           │ '1'              │ '5'                  │ '25.06'      │ 'moderate'   │
│ 3       │ 'Alpha'     │ 'in_progress'  │ '42.2'       │ '0'           │ '0'              │ '1'                  │ '4.74'       │ 'low'        │
└─────────┴─────────────┴────────────────┴──────────────┴───────────────┴──────────────────┴──────────────────────┴──────────────┴──────────────┘
Demo accounts (password = SEED_DEFAULT_PASSWORD from your .env):
```

S-18b Seeding with progress bars (PostgreSQL, MongoDB, embeddings)

```
$ psql -d projectpulse (function, view, trigger)

== Function: delivery risk for Bravo, calculated inside the database
 risk_score | risk_level |                                                
```

```
$ MongoDB aggregations (db/mongo/aggregations.ts)
// A1  version counts per document, project 1
[
  {
    "documentType": "SRS",
    "deliverableType": "srs",
    "title": "CampusConnect SRS",
    "versionCount": 3,
    "lastUpdated": "2026-10-09T04:13:31.728Z"
  },
  {
    "documentType": "Proposal",
    "deliverableType": "proposal",
    "title": "CampusConnect proposal",
    "versionCount": 2,
    "lastUpdated": "2026-09-18T04:13:31.728Z"
  },
  {
    "documentType": "Design",
    "deliverableType": "design_doc",
    "title": "CampusConnect design document (draft)",
    "versionCount": 1,
    "lastUpdated": "2026-10-24T04:13:31.728Z"
  }
]

// A2  document-type distribution (projects 1–4)
[
  {
    "_id": "Proposal",
    "documents": 4,
    "avgVersions": 1.5
  },
  {
    "_id": "SRS",
    "documents": 3,
    "avgVersions": 2.3333333333333335
  },
  {
    "_id": "Design",
    "documents": 2,
    "avgVersions": 1.5
  },
  {
    "_id": "Other",
    "documents": 2,
    "avgVersions": 1.5
  }
]

// A4  open feedback themes, project 4
[
  {
    "_id": "design",
    "openThreads": 2,
    "avgMessages": 2
  },
  {
    "_id": "documentation",
    "openThreads": 1,
    "avgMessages": 1
  },
  {
    "_id": "requirements",
    "openThreads": 1,
    "avgMessages": 1
  }
]

// validator: insert { documentType: 'Poster', versions: [] }
rejected by the $jsonSchema validator (code 121): Document failed validation
```

S-20 MongoDB aggregations and validator rejection

```
$ npm test (database + unit tests)

 RUN  v2.1.9 /Users/aadi/Desktop/DBMS Mini-Project/projectpulse

 ✓ db/tests/constraints_triggers.test.ts (32 tests) 760ms
 ✓ db/tests/mongo.test.ts (4 tests) 64ms
 ✓ db/tests/queries.test.ts (8 tests) 80ms
 ✓ db/tests/seed_scenarios.test.ts (7 tests) 79ms
 ✓ db/tests/vector.test.ts (7 tests) 47ms
 ✓ tests/unit/csv.test.ts (5 tests) 5ms
 ✓ tests/unit/security.test.ts (4 tests) 82ms
 ✓ tests/unit/similarity.test.ts (10 tests) 3ms

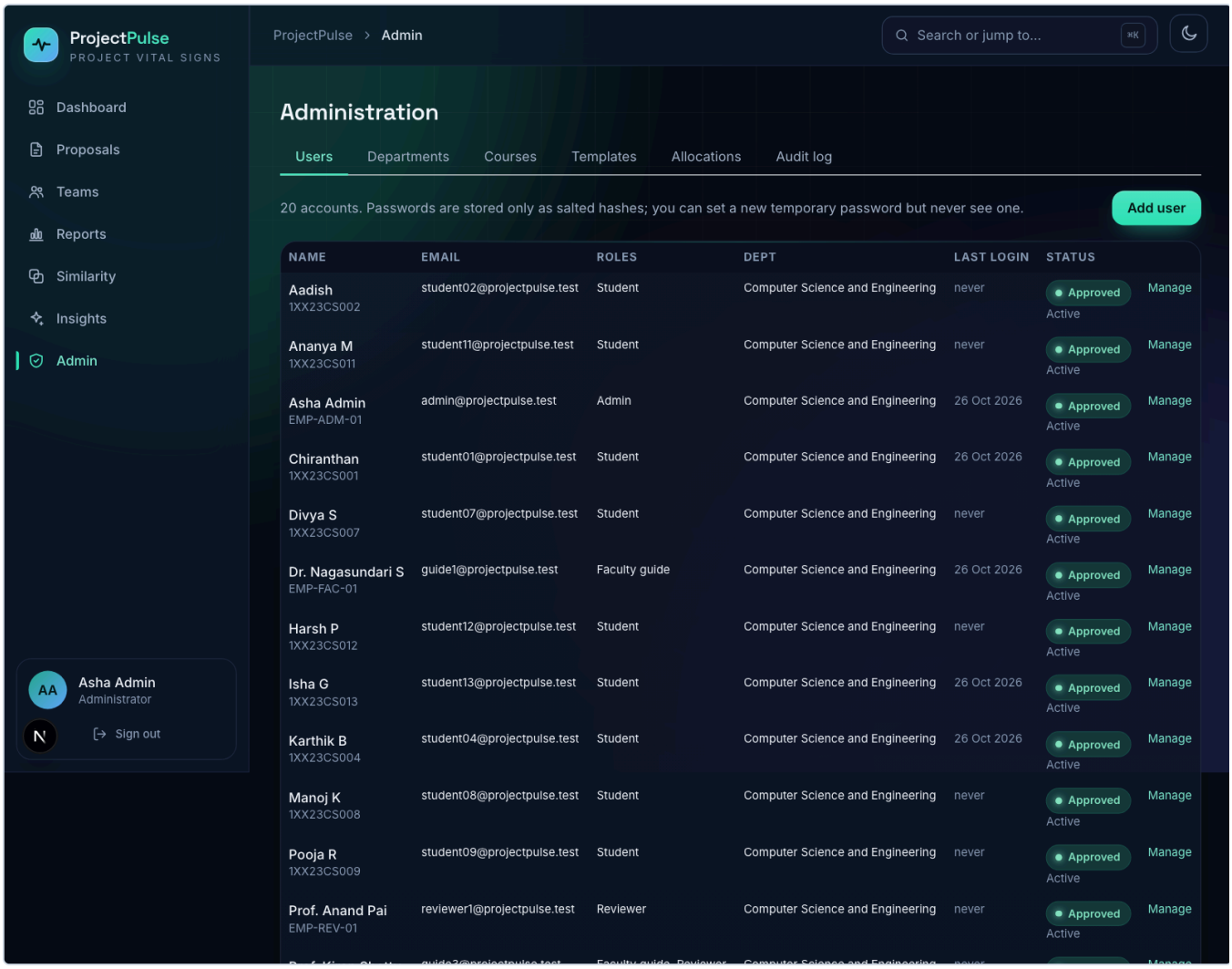
 Test Files  8 passed (8)
      Tests  77 passed (77)
   Start at  09:43:40
   Duration  2.73s (transform 331ms, setup 0ms, collect 255ms, tests 1.12s, environment 1ms, prepare 269ms)
```

S-21 Automated test results

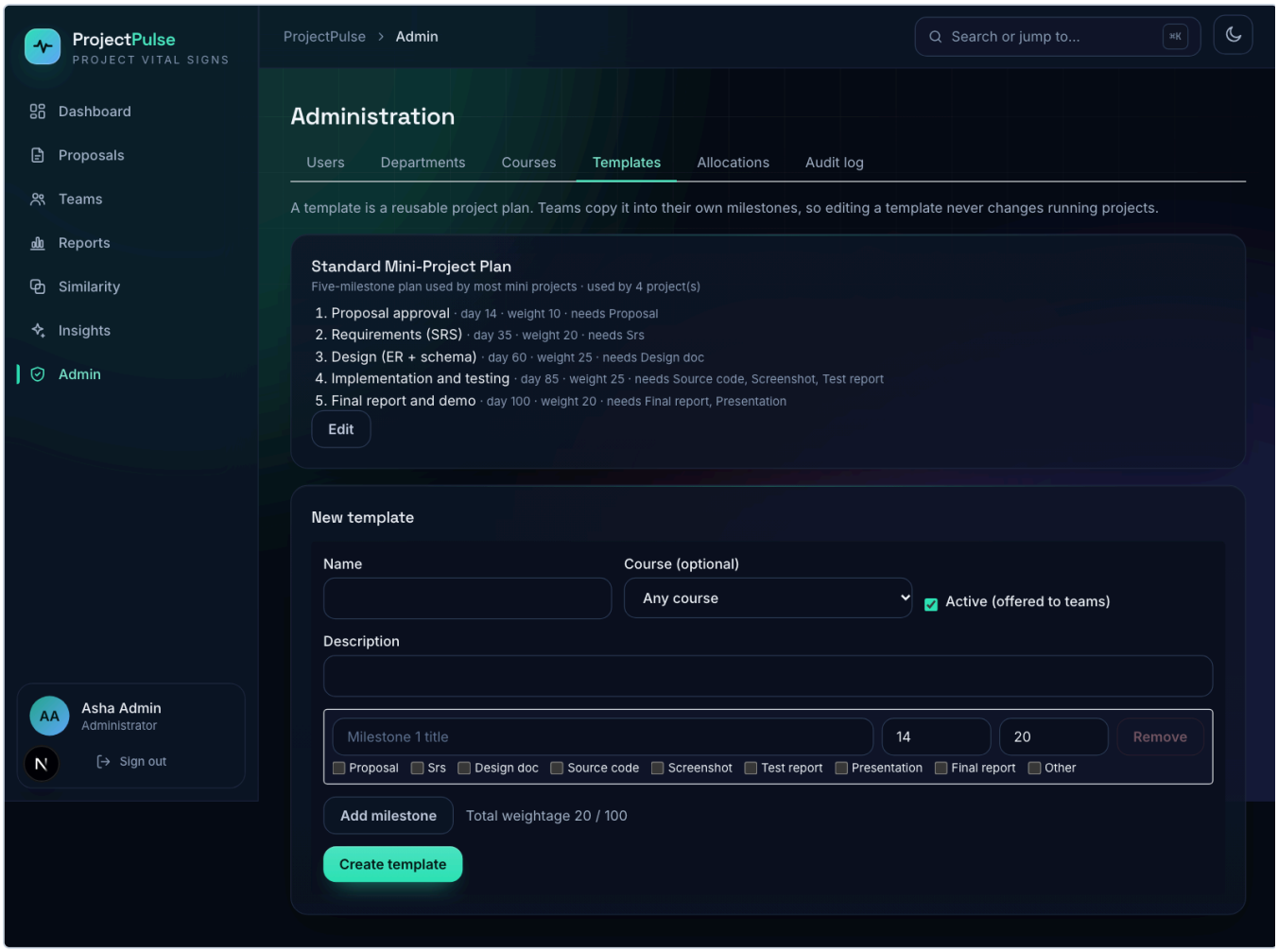
```
$ git shortlog -sn --all; git log --stat
     6  Sonuaadi0706 <sonuaadish0706as@gmail.com>

f208024 docs: decisions, generated data dictionary, ER diagrams, EXPLAIN evidence and test results
 docs/DECISIONS.md                                |  50 ++
 docs/EXPLAIN_ANALYZE.md                          | 125 ++++
 docs/REPORT.md                                   | 410 ++++++++++++++
 docs/TEST_RESULTS.md                             |  64 ++
 docs/data_dictionary.md                          | 671 ++++++++++++++++++++++
 docs/er_diagram.mmd                              | 391 ++++++++++++
 docs/er_overview.mmd                             | 221 +++++++
 docs/relational_schema.md                        |  66 ++
 docs/report.css                                  |  33 +
 docs/screenshots/S-01-login.png                  | Bin 0 -> 22659 bytes
 docs/screenshots/S-02-student-dashboard.png      | Bin 0 -> 171045 bytes
 docs/screenshots/S-03-guide-dashboard.png        | Bin 0 -> 273781 bytes
 docs/screenshots/S-04-admin-dashboard.png        | Bin 0 -> 98115 bytes
 docs/screenshots/S-05-proposal-form.png          | Bin 0 -> 236911 bytes
 .../screenshots/S-06-proposal-approval-history.png | Bin 0 -> 183064 bytes
 docs/screenshots/S-07-guide-allocation.png       | Bin 0 -> 157201 bytes
 docs/screenshots/S-07b-team.png                  | Bin 0 -> 81904 bytes
 docs/screenshots/S-08-milestone-timeline.png     | Bin 0 -> 156642 bytes
 docs/screenshots/S-09-task-kanban.png            | Bin 0 -> 221051 bytes
 docs/screenshots/S-10-deliverables-versions.png  | Bin 0 -> 131518 bytes
 docs/screenshots/S-11-submission-rollback.png    | Bin 0 -> 142849 bytes
 docs/screenshots/S-12-feedback-thread.png        | Bin 0 -> 202376 bytes
 docs/screenshots/S-13-contribution.png           | Bin 0 -> 192387 bytes
 docs/screenshots/S-14-risk-detail.png            | Bin 0 -> 232448 bytes
 docs/screenshots/S-15-similarity-review.png      | Bin 0 -> 157463 bytes
 docs/screenshots/S-16-reports.png                | Bin 0 -> 95515 bytes
 docs/screenshots/S-16b-reports-high-risk.png     | Bin 0 -> 163516 bytes
 docs/screenshots/S-17-audit-log.png              | Bin 0 -> 262634 bytes
 docs/screenshots/S-18-migrations-progress.png    | Bin 0 -> 34372 bytes
 docs/screenshots/S-18b-seed-progress.png         | Bin 0 -> 129947 bytes
 docs/screenshots/S-19-sql-features.png           | Bin 0 -> 186358 bytes
 docs/screenshots/S-20-mongo-aggregation.png      | Bin 0 -> 169622 bytes
 docs/screenshots/S-21-test-results.png           | Bin 0 -> 74578 bytes
 docs/screenshots/S-23-admin-users.png            | Bin 0 -> 282914 bytes
 docs/screenshots/S-24-admin-templates.png        | Bin 0 -> 146608 bytes
 docs/screenshots/S-25-insights.png               | Bin 0 -> 84332 bytes
 36 files changed, 2031 insertions(+)
c36bd11 test: database, unit and API test suites
 db/tests/constraints_triggers.test.ts | 313 ++++++++++++++++++++++++++++++++++++
```

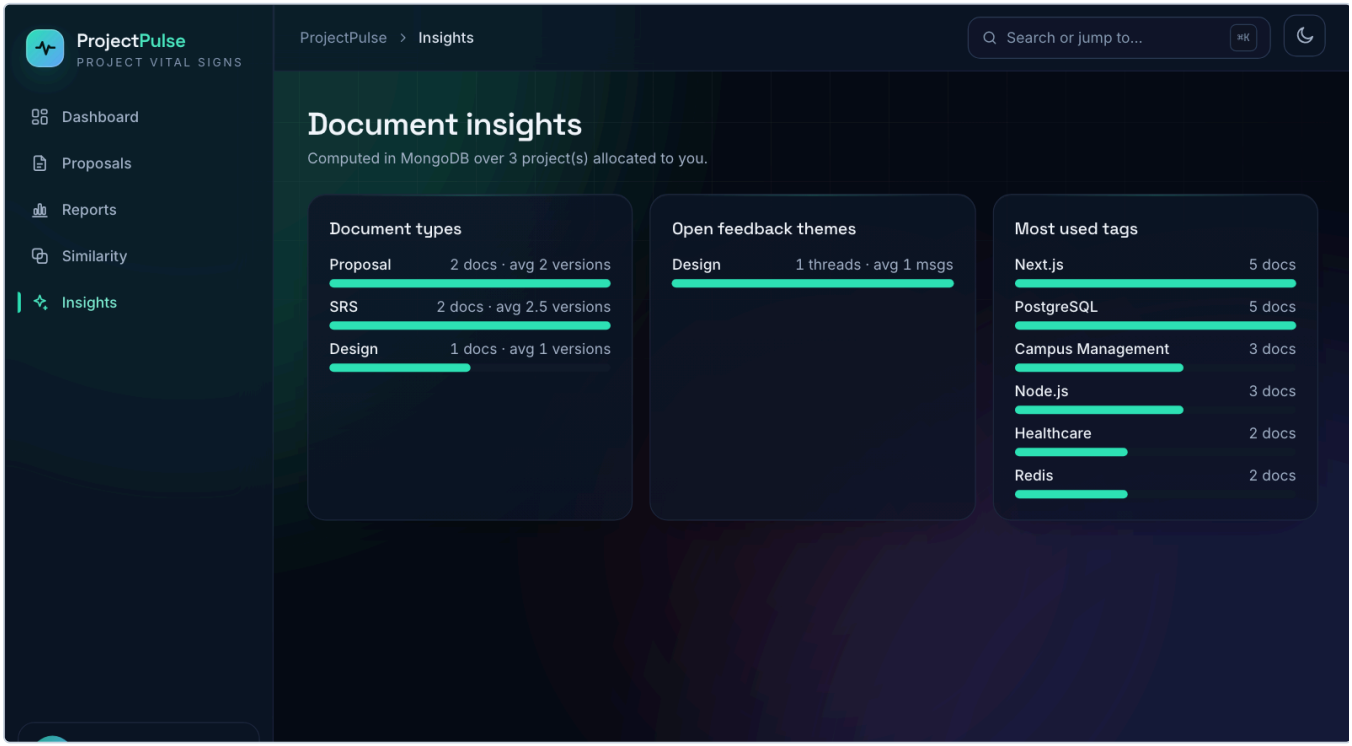
S-22 Repository history



S-23 Administration: users and roles



S-24 Administration: project templates



S-25 Document insights computed in MongoDB

15. Testing results

159 of 159 automated tests pass: database tests run inside rolled-back transactions; API tests call the running application over HTTP; unit tests cover scoring, chunking, CSV safety and the repository secret scan. The tests that change data (workflow, uploads, rollback, similarity decisions, peer evaluation, administration) restore the demo automatically afterwards. Every case T-01 to T-30 of the specification has automated evidence below.

159 of 159 tests passed (0 failed, 0 skipped).

Spec test cases T-01 to T-30

ID	Level	Test case	Result	Automated evidence
T-01	API	Login as each of the four roles; role-specific dashboard shown	PASS (1/1)	tests/api/auth.test.ts
T-02	API	Wrong password 5 times: account locked 15 minutes, generic message	PASS (1/1)	tests/api/auth.test.ts
T-03	DB	users.password_hash holds bcrypt hashes only; no plain text	PASS (1/1)	tests/api/auth.test.ts
T-04	API	Student calls guide-only endpoints: 403 and nothing changes	PASS (2/2)	tests/api/access.test.ts, tests/api/workflow.test.ts
T-05	API	Student of team A requests team B's documents, feedback, evidence: no data leaked	PASS (3/3)	tests/api/access.test.ts, tests/api/feedback.test.ts
T-06	DB/API	Invalid dates, negative effort or weightage are rejected (CHECK constraints)	PASS (5/5)	db/tests/constraints_triggers.test.ts, tests/api/workflow.test.ts
T-07	DB/API	Duplicate active project, student in two teams, duplicate title: unique violations	PASS (6/6)	db/tests/constraints_triggers.test.ts, tests/api/workflow.test.ts
T-08	DB	Milestone weightage total above 100 rejected by the constraint trigger	PASS (2/2)	db/tests/constraints_triggers.test.ts, tests/api/workflow.test.ts
T-09	DB	Illegal status change (planned to approved, todo to verified) rejected	PASS (4/4)	db/tests/constraints_triggers.test.ts, tests/api/workflow.test.ts
T-10	DB	UPDATE / DELETE on proposal_actions and audit_logs: append-only	PASS (3/3)	db/tests/constraints_triggers.test.ts
T-11	DB	Guide allocated to own team, or non-guide allocated as guide: rejected	PASS (5/5)	db/tests/constraints_triggers.test.ts
T-12	DB	Overdue milestone creates one risk event, no duplicates on repeat	PASS (1/1)	db/tests/constraints_triggers.test.ts
T-13	DB	sp_recalculate_risk matches the hand calculation; breakdown present	PASS (3/3)	db/tests/constraints_triggers.test.ts
T-14	DB	risk_assessments with a level that does not match the score: CHECK violation	PASS (3/3)	db/tests/constraints_triggers.test.ts
T-15	API+DB	Failed milestone submission rolls back completely (deliverable, version, review, audit, file, Mongo)	PASS (4/4)	tests/api/deliverables.test.ts
T-16	API+DB	Valid submission: deliverable, version, status, review requests, audit row, Mongo confirmed	PASS (1/1)	tests/api/deliverables.test.ts
T-17	DB	Versions inserted without a number are numbered 1, 2, 3	PASS (2/2)	db/tests/constraints_triggers.test.ts, tests/api/deliverables.test.ts
T-18	API+DB	Approve / reject / request revision: status, immutable history row, audit row; reject needs a comment	PASS (1/1)	tests/api/workflow.test.ts
T-19	DB	Status changes write a STATUS_CHANGE audit row with the user id	PASS (1/1)	db/tests/constraints_triggers.test.ts
T-20	API+DB	A draft deliverable can be deleted (audited); a submitted one cannot	PASS (2/2)	tests/api/deliverables.test.ts
T-21	DB	Deleting a project with milestones, or a user referenced by tasks, is blocked (RESTRICT)	PASS (3/3)	db/tests/constraints_triggers.test.ts
T-22	API+vector	Similarity search for Echo returns Alpha above threshold; other course never returned	PASS (3/3)	tests/api/similarity-flow.test.ts, tests/api/similarity.test.ts

ID	Level	Test case	Result	Automated evidence
T-23	API+UI	Faculty decision saved with reviewer, time and audit; student hint is generic	PASS (2/2)	tests/api/similarity-flow.test.ts
T-24	DB	Verifying a task creates verified evidence; unverified evidence is excluded from the summary	PASS (1/1)	db/tests/constraints_triggers.test.ts
T-25	DB	Approving a milestone with a missing mandatory deliverable type is blocked	PASS (3/3)	db/tests/constraints_triggers.test.ts, tests/api/deliverables.test.ts
T-26	Mongo	Aggregations A1-A4 match the seeded data	PASS (4/4)	tests/api/similarity.test.ts
T-27	DB	EXPLAIN ANALYZE before/after indexes; HNSW and partial indexes used	PASS (2/2)	db/tests/vector.test.ts
T-28	DB	Full-text search for "event" returns Alpha, Echo, Foxtrot ranked	PASS (2/2)	db/tests/vector.test.ts
T-29	Repo	No secrets committed; .env ignored	PASS (4/4)	tests/unit/security.test.ts
T-30	UI	Each role's dashboard renders all required widgets with the seed data	PASS (1/1)	tests/api/reports.test.ts

All test files

File	Tests	Passed
db/tests/constraints_triggers.test.ts	32	32
db/tests/mongo.test.ts	4	4
db/tests/queries.test.ts	8	8
db/tests/seed_scenarios.test.ts	7	7
db/tests/vector.test.ts	7	7
tests/api/access.test.ts	8	8
tests/api/admin.test.ts	6	6
tests/api/auth.test.ts	5	5
tests/api/deliverables.test.ts	10	10
tests/api/feedback.test.ts	9	9
tests/api/peer.test.ts	4	4
tests/api/reports.test.ts	10	10
tests/api/risk.test.ts	5	5
tests/api/similarity-flow.test.ts	4	4
tests/api/similarity.test.ts	13	13
tests/api/workflow.test.ts	8	8
tests/unit/csv.test.ts	5	5
tests/unit/security.test.ts	4	4
tests/unit/similarity.test.ts	10	10

16. Conclusion and future enhancements

Conclusion. ProjectPulse turns scattered project supervision into an evidence-based, explainable workflow. Delays, unequal contribution, missing documentation, unresolved feedback and overlapping proposals become visible early, with the reason attached, while faculty keep every decision about grades and originality. The three database paradigms each do what they are best at: PostgreSQL guarantees the workflow and integrity (constraints, triggers, transactions, views, functions), MongoDB holds flexible documents and discussions, and pgvector finds proposals that mean the same thing. All of it is exercised by a working Next.js application and by 159 automated tests.

Known limitations. Notifications, rate limiting and a strict Content-Security-Policy were optional and are not built. Only proposal sections are embedded (not the text of uploaded documents). Risk is a transparent rule-based score, not a predictive model. Commit references are entered by hand.

Future enhancements. GitHub OAuth sync of commits and pull requests; OCR and PDF text extraction with per-section document embeddings; e-mail / WhatsApp notifications; machine-learning risk prediction; assisted similarity reports with source highlighting (still human-decided); LMS / ERP integration; PostgreSQL row-level security; analytics across semesters; a mobile-friendly PWA.

17. Team contributions and repository

The team has two members, **Chiranthan** and **Aadish**. As agreed in the proposal, the database work (ER diagram, relational schema, DDL, constraints, indexes, views, triggers, functions, queries, test data) and the application work (Next.js interface, authentication and authorisation, MongoDB model, semantic-search module, dashboards, testing) are split between the members, and each also contributes to the other half. The **repository history is the record of individual contributions** (S-22):

```
7    Sonuaadi0706 <sonuaadish0706as@gmail.com>
```

The GitHub repository link is submitted together with this report. The README explains setup and execution step by step; every design decision and every deviation from the specification is recorded in `docs/DECISIONS.md`.

18. References

1. Next.js documentation, App Router and Route Handlers. https://nextjs.org/docs
2. PostgreSQL 16+ documentation: constraints, triggers, transactions, full-text search. https://www.postgresql.org/docs/
3. pgvector: open-source vector similarity search for PostgreSQL (HNSW, cosine distance). https://github.com/pgvector/pgvector
4. MongoDB manual: `$jsonSchema` validation and the aggregation framework. https://www.mongodb.com/docs/manual/
5. Reimers and Gurevych, *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*, 2019; model `sentence-transformers/all-MiniLM-L6-v2`, ONNX port `Xenova/all-MiniLM-L6-v2`. https://huggingface.co/Xenova/all-MiniLM-L6-v2
6. Transformers.js (`@huggingface/transformers`). https://huggingface.co/docs/transformers.js
7. OWASP Cheat Sheets: Password Storage, SQL Injection Prevention, CSRF Prevention, CSV Injection.
8. Mermaid (diagrams) and `marked` (Markdown) used to build this report.

Appendix A: complete data dictionary

Generated by `npm run docs:data-dictionary` from the live PostgreSQL catalogue (schema `projectpulse`): 32 tables, 7 views, 23 functions and procedures, 39 triggers.

Legend: **PK** primary key · **FK** foreign key (with its ON DELETE rule) · **UQ** unique · **CHECK** validation rule enforced by the database.

audit_logs

Append-only log of important actions written by triggers and the API.

Column	Type	Null	Default	Constraints and notes
log_id	bigint	NO		**PK** identity (auto-generated)
user_id	integer	yes		**FK** → users(user_id) ON DELETE SET NULL
action	varchar(30)	NO		
entity_type	varchar(40)	NO		
entity_id	varchar(40)	yes		
old_values	jsonb	yes		
new_values	jsonb	yes		
created_at	timestamptz	NO	`now()`	

Indexes:

- `INDEX idx_audit_entity ON audit_logs USING btree (entity_type, entity_id, created_at DESC)`
- `INDEX idx_audit_user_time ON audit_logs USING btree (user_id, created_at DESC)`

Triggers: `trg_audit_logs_immutable`

contribution_evidence

Verifiable per-student evidence (tasks, uploads, feedback fixes, commits). Only verified rows count.

Column	Type	Null	Default	Constraints and notes
evidence_id	integer	NO		**PK** identity (auto-generated)
project_id	integer	NO		**FK** → projects(project_id) ON DELETE RESTRICT
user_id	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT **UQ** with (user_id, evidence_type, source_table, source_id)
task_id	integer	yes		**FK** → tasks(task_id) ON DELETE SET NULL
evidence_type	varchar(25)	NO		CHECK (((evidence_type)::text = ANY ((ARRAY['task_completed'::character varying, 'document_upload'::character varying, 'feedback_resolved'::character varying, 'git_activity'::character varying, 'review_participation'::character varying])::text[]))) **UQ** with (user_id, evidence_type, source_table, source_id)
description	varchar(300)	yes		
weight	numeric(4,2)	NO	`1`	CHECK (((weight > (0)::numeric) AND (weight <= (10)::numeric)))
verified_flag	boolean	NO	`false`	
verified_by	integer	yes		**FK** → users(user_id) ON DELETE SET NULL
verified_at	timestamptz	yes		
source_table	varchar(40)	yes		**UQ** with (user_id, evidence_type, source_table, source_id)
source_id	integer	yes		**UQ** with (user_id, evidence_type, source_table, source_id)
created_at	timestamptz	NO	`now()`	

Table-level rules:

- `CHECK (((NOT verified_flag) OR ((verified_by IS NOT NULL) AND (verified_at IS NOT NULL))))`

Indexes:

- `INDEX idx_evidence_project_user ON contribution_evidence USING btree (project_id, user_id) WHERE verified_flag`

courses

Project courses per semester and academic year.

Column	Type	Null	Default	Constraints and notes
course_id	integer	NO		**PK** identity (auto-generated)
dept_id	integer	NO		**FK** → departments(dept_id) ON DELETE RESTRICT
course_code	varchar(20)	NO		**UQ** with (course_code, semester, academic_year)
course_name	varchar(150)	NO		
semester	varchar(20)	NO		**UQ** with (course_code, semester, academic_year)
academic_year	varchar(9)	NO		CHECK (((academic_year)::text ~ '^[0-9]{4}-[0-9]{4}$'::text)) **UQ** with (course_code, semester, academic_year)
is_active	boolean	NO	`true`	
created_at	timestamptz	NO	`now()`	

deliverables

Submission metadata (type, links, status) linked to a MongoDB document.

Column	Type	Null	Default	Constraints and notes
deliverable_id	integer	NO		**PK** identity (auto-generated)
milestone_id	integer	NO		**FK** → milestones(milestone_id) ON DELETE RESTRICT
deliverable_type	varchar(30)	NO		CHECK (((deliverable_type)::text = ANY ((ARRAY['proposal'::character varying, 'srs'::character varying, 'design_doc'::character varying, 'source_code'::character varying, 'screenshot'::character varying, 'test_report'::character varying, 'presentation'::character varying, 'final_report'::character varying, 'other'::character varying])::text[])))
title	varchar(200)	NO		
repo_link	text	yes		CHECK (((repo_link IS NULL) OR (repo_link ~* '^https?://'::text)))
evidence_url	text	yes		CHECK (((evidence_url IS NULL) OR (evidence_url ~* '^https?://'::text)))
mongo_doc_id	varchar(40)	yes		
status	varchar(10)	NO	`'draft'::character varying`	CHECK (((status)::text = ANY ((ARRAY['draft'::character varying, 'submitted'::character varying, 'accepted'::character varying, 'rejected'::character varying])::text[])))
submitted_by	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT
submitted_at	timestamptz	yes		
created_at	timestamptz	NO	`now()`	
updated_at	timestamptz	NO	`now()`	

Table-level rules:

- `CHECK ((((status)::text = 'draft'::text) OR (submitted_at IS NOT NULL)))`

Indexes:

- `INDEX idx_deliverables_milestone ON deliverables USING btree (milestone_id, deliverable_type, status)`

Triggers: `trg_deliverables_audit`, `trg_deliverables_row_audit`, `trg_deliverables_transition`, `trg_deliverables_updated`

departments

Academic departments.

Column	Type	Null	Default	Constraints and notes
dept_id	integer	NO		**PK** identity (auto-generated)
dept_code	varchar(10)	NO		**UQ**
dept_name	varchar(120)	NO		**UQ**
created_at	timestamptz	NO	`now()`	

document_versions

Every uploaded or linked version of a deliverable, with checksum and size.

Column	Type	Null	Default	Constraints and notes
version_id	integer	NO		**PK** identity (auto-generated)
deliverable_id	integer	NO		**FK** → deliverables(deliverable_id) ON DELETE CASCADE **UQ** with (deliverable_id, version_number)
version_number	integer	NO		CHECK ((version_number > 0)) **UQ** with (deliverable_id, version_number)
change_summary	varchar(500)	NO		
file_name	varchar(255)	yes		
file_url	text	yes		
file_size_bytes	bigint	yes		CHECK (((file_size_bytes IS NULL) OR (file_size_bytes >= 0)))
checksum_sha256	character(64)	yes		
uploaded_by	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT
uploaded_at	timestamptz	NO	`now()`	

Table-level rules:

- `CHECK (((file_url IS NOT NULL) OR (file_name IS NOT NULL)))`

Indexes:

- `INDEX idx_docver_deliverable ON document_versions USING btree (deliverable_id, version_number DESC)`

Triggers: `trg_docver_evidence`, `trg_docver_number`

feedback_items

Trackable feedback assigned to one student: open, in progress, resolved, verified, reopened.

Column	Type	Null	Default	Constraints and notes
feedback_id	integer	NO		**PK** identity (auto-generated)
review_id	integer	NO		**FK** → reviews(review_id) ON DELETE RESTRICT
created_by	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT
assigned_user_id	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT
description	text	NO		CHECK ((length(TRIM(BOTH FROM description)) > 0))
severity	varchar(8)	NO	`'minor'::character varying`	CHECK (((severity)::text = ANY ((ARRAY['minor'::character varying, 'major'::character varying, 'critical'::character varying])::text[])))
resolution_status	varchar(12)	NO	`'open'::character varying`	CHECK (((resolution_status)::text = ANY ((ARRAY['open'::character varying, 'in_progress'::character varying, 'resolved'::character varying, 'verified'::character varying, 'reopened'::character varying])::text[])))
response_text	text	yes		
due_date	date	yes		
resolved_at	timestamptz	yes		
verified_by	integer	yes		**FK** → users(user_id) ON DELETE SET NULL
verified_at	timestamptz	yes		

Column	Type	Null	Default	Constraints and notes
created_at	timestamptz	NO	`now()`	
updated_at	timestamptz	NO	`now()`	

Table-level rules:

- `CHECK ((((resolution_status)::text <> ALL ((ARRAY['resolved'::character varying, 'verified'::character varying])::text[])) OR (resolved_at IS NOT NULL)))`
- `CHECK ((((resolution_status)::text <> 'verified'::text) OR ((verified_by IS NOT NULL) AND (verified_at IS NOT NULL))))`

Indexes:

- `INDEX idx_feedback_assignee_open ON feedback_items USING btree (assigned_user_id) WHERE ((resolution_status)::text = ANY ((ARRAY['open'::character varying, 'in_progress'::character varying, 'reopened'::character varying])::text[]))`
- `INDEX idx_feedback_status ON feedback_items USING btree (resolution_status)`

Triggers: `trg_feedback_assignee_member`, `trg_feedback_audit`, `trg_feedback_evidence`, `trg_feedback_transition`, `trg_feedback_updated`

git_activities

Manually recorded commit references.

Column	Type	Null	Default	Constraints and notes
git_id	integer	NO		**PK** identity (auto-generated)
project_id	integer	NO		**FK** → projects(project_id) ON DELETE RESTRICT
task_id	integer	yes		**FK** → tasks(task_id) ON DELETE SET NULL
user_id	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT
repo_url	text	NO		CHECK ((repo_url ~* '^https?://'::text)) **UQ** with (repo_url, commit_ref)
commit_ref	varchar(64)	NO		**UQ** with (repo_url, commit_ref)
commit_message	varchar(300)	yes		
committed_at	timestamptz	yes		
recorded_by	integer	yes		**FK** → users(user_id) ON DELETE SET NULL
created_at	timestamptz	NO	`now()`	

guide_allocations

Which faculty guide (one) and reviewers are allocated to a project.

Column	Type	Null	Default	Constraints and notes
allocation_id	integer	NO		**PK** identity (auto-generated)
project_id	integer	NO		**FK** → projects(project_id) ON DELETE CASCADE **UQ** with (project_id, user_id, allocation_role)
user_id	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT **UQ** with (project_id, user_id, allocation_role)
allocation_role	varchar(10)	NO		CHECK (((allocation_role)::text = ANY ((ARRAY['guide'::character varying, 'reviewer'::character varying])::text[]))) **UQ** with (project_id, user_id, allocation_role)
allocated_by	integer	yes		**FK** → users(user_id) ON DELETE SET NULL
allocated_at	timestamptz	NO	`now()`	

Indexes:

- `UNIQUE INDEX uq_one_guide_per_project ON guide_allocations USING btree (project_id) WHERE ((allocation_role)::text = 'guide'::text)`

Triggers: `trg_guide_alloc_audit`, `trg_guide_alloc_role`

milestone_required_deliverables

Deliverable types that must exist before a milestone can be approved.

Column	Type	Null	Default	Constraints and notes
milestone_id	integer	NO		**FK** → milestones(milestone_id) ON DELETE CASCADE **PK**
deliverable_type	varchar(30)	NO		CHECK (((deliverable_type)::text = ANY ((ARRAY['proposal'::character varying, 'srs'::character varying, 'design_doc'::character varying, 'source_code'::character varying, 'screenshot'::character varying, 'test_report'::character varying, 'presentation'::character varying, 'final_report'::character varying, 'other'::character varying])::text[]))) **PK**

milestones

Dated, weighted milestones of a project with a status workflow.

Column	Type	Null	Default	Constraints and notes
milestone_id	integer	NO		**PK** identity (auto-generated)
project_id	integer	NO		**FK** → projects(project_id) ON DELETE RESTRICT **UQ** with (project_id, sequence_no)
sequence_no	integer	NO		CHECK ((sequence_no > 0)) **UQ** with (project_id, sequence_no)
title	varchar(150)	NO		
description	text	yes		
start_date	date	yes		
due_date	date	NO		
status	varchar(20)	NO	`'planned'::character varying`	CHECK (((status)::text = ANY ((ARRAY['planned'::character varying, 'in_progress'::character varying, 'submitted'::character varying, 'under_review'::character varying, 'approved'::character varying, 'revision_requested'::character varying, 'rejected'::character varying])::text[])))
weightage	numeric(5,2)	NO		CHECK (((weightage > (0)::numeric) AND (weightage <= (100)::numeric)))
submitted_at	timestamptz	yes		
approved_by	integer	yes		**FK** → users(user_id) ON DELETE SET NULL
approved_at	timestamptz	yes		
approval_comments	text	yes		
created_at	timestamptz	NO	`now()`	
updated_at	timestamptz	NO	`now()`	
required_evidence_type	varchar(30)	yes		CHECK (((required_evidence_type IS NULL) OR ((required_evidence_type)::text = ANY ((ARRAY['proposal'::character varying, 'srs'::character varying, 'design_doc'::character varying, 'source_code'::character varying, 'screenshot'::character varying, 'test_report'::character varying, 'presentation'::character varying, 'final_report'::character varying, 'other'::character varying])::text[]))))

Table-level rules:

- `CHECK (((start_date IS NULL) OR (due_date >= start_date)))`
- `CHECK ((((status)::text <> 'approved'::text) OR ((approved_by IS NOT NULL) AND (approved_at IS NOT NULL))))`

Indexes:

- `INDEX idx_milestones_due_status ON milestones USING btree (due_date, status)`
- `INDEX idx_milestones_project ON milestones USING btree (project_id, sequence_no)`

Triggers: `trg_milestone_block_approval`, `trg_milestone_overdue`, `trg_milestone_weightage`, `trg_milestones_audit`, `trg_milestones_transition`, `trg_milestones_updated`

notifications

In-app notifications (reserved; not used in version 1).

Column	Type	Null	Default	Constraints and notes
notification_id	integer	NO		**PK** identity (auto-generated)
user_id	integer	NO		**FK** → users(user_id) ON DELETE CASCADE
type	varchar(30)	NO		
message	varchar(300)	NO		
link	varchar(200)	yes		
is_read	boolean	NO	`false`	
created_at	timestamptz	NO	`now()`	

Indexes:

- `INDEX idx_notifications_unread ON notifications USING btree (user_id) WHERE (NOT is_read)`

peer_evaluations

Controlled teammate ratings, one per pair per milestone.

Column	Type	Null	Default	Constraints and notes
evaluation_id	integer	NO		**PK** identity (auto-generated)
project_id	integer	NO		**FK** → projects(project_id) ON DELETE RESTRICT **UQ** with (project_id, milestone_id, evaluator_id, evaluatee_id)
milestone_id	integer	yes		**FK** → milestones(milestone_id) ON DELETE SET NULL **UQ** with (project_id, milestone_id, evaluator_id, evaluatee_id)
evaluator_id	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT **UQ** with (project_id, milestone_id, evaluator_id, evaluatee_id)
evaluatee_id	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT **UQ** with (project_id, milestone_id, evaluator_id, evaluatee_id)
score	numeric(3,1)	NO		CHECK (((score >= (1)::numeric) AND (score <= (5)::numeric)))
comments	varchar(500)	yes		
created_at	timestamptz	NO	`now()`	

Table-level rules:

- `CHECK ((evaluator_id <> evaluatee_id))`

project_embeddings

pgvector table: 384-dimensional sentence embeddings of proposal sections.

Column	Type	Null	Default	Constraints and notes
embedding_id	integer	NO		**PK** identity (auto-generated)
project_id	integer	NO		**FK** → projects(project_id) ON DELETE CASCADE **UQ** with (project_id, section_type, chunk_index, model_name)
course_id	integer	NO		**FK** → courses(course_id) ON DELETE RESTRICT
section_type	varchar(20)	NO		CHECK (((section_type)::text = ANY ((ARRAY['title'::character varying, 'abstract'::character varying, 'problem_statement'::character varying, 'objectives'::character varying, 'scope'::character varying, 'document_chunk'::character varying])::text[])) **UQ** with (project_id, section_type, chunk_index, model_name)
chunk_index	integer	NO	`0`	CHECK ((chunk_index >= 0)) **UQ** with (project_id, section_type, chunk_index, model_name)

Column	Type	Null	Default	Constraints and notes
chunk_text	text	NO		
embedding	vector(384)	NO		
model_name	varchar(80)	NO		**UQ** with (project_id, section_type, chunk_index, model_name)
is_current	boolean	NO	`true`	
access_level	varchar(10)	NO	`'faculty'::character varying`	CHECK (((access_level)::text = ANY ((ARRAY['faculty'::character varying, 'admin'::character varying])::text[])))
created_at	timestamptz	NO	`now()`	

Indexes:

- `INDEX idx_embeddings_filter ON project_embeddings USING btree (course_id, section_type) WHERE is_current`
- `INDEX idx_embeddings_hnsw ON project_embeddings USING hnsw (embedding vector_cosine_ops) WITH (m='16', ef_construction='64')`

project_templates

Reusable project plans managed by admins.

Column	Type	Null	Default	Constraints and notes
template_id	integer	NO		**PK** identity (auto-generated)
course_id	integer	yes		**FK** → courses(course_id) ON DELETE SET NULL
name	varchar(120)	NO		**UQ**
description	text	yes		
is_active	boolean	NO	`true`	
created_by	integer	yes		**FK** → users(user_id) ON DELETE SET NULL
created_at	timestamptz	NO	`now()`	

projects

A team's proposal and project: text, status workflow, dates. One active project per team.

Column	Type	Null	Default	Constraints and notes
project_id	integer	NO		**PK** identity (auto-generated)
team_id	integer	NO		**FK** → teams(team_id, course_id) ON DELETE RESTRICT
course_id	integer	NO		**FK** → teams(team_id, course_id) ON DELETE RESTRICT
template_id	integer	yes		**FK** → project_templates(template_id) ON DELETE SET NULL
title	varchar(200)	NO		CHECK ((length(TRIM(BOTH FROM title)) >= 5))
abstract	text	yes		
problem_statement	text	yes		
objectives	text	yes		
scope	text	yes		
domain	varchar(80)	yes		
technologies	text[]	NO	`'{}'::text[]`	
repo_link	text	yes		CHECK (((repo_link IS NULL) OR (repo_link ~* '^https?://'::text)))
status	varchar(20)	NO	`'draft'::character varying`	CHECK (((status)::text = ANY ((ARRAY['draft'::character varying, 'submitted'::character varying, 'revision_requested'::character varying, 'approved'::character varying, 'rejected'::character varying, 'in_progress'::character varying, 'completed'::character varying, 'archived'::character varying])::text[])))
start_date	date	yes		
end_date	date	yes		
created_by	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT
submitted_at	timestamptz	yes		

Column	Type	Null	Default	Constraints and notes
approved_by	integer	yes		**FK** → users(user_id) ON DELETE SET NULL
approved_at	timestamptz	yes		
created_at	timestamptz	NO	`now()`	
updated_at	timestamptz	NO	`now()`	

Table-level rules:

- `CHECK (((end_date IS NULL) OR (start_date IS NULL) OR (end_date >= start_date)))`
- `CHECK ((((status)::text = 'draft'::text) OR ((abstract IS NOT NULL) AND (length(abstract) >= 50) AND (problem_statement IS NOT NULL) AND (length(problem_statement) >= 30) AND (objectives IS NOT NULL) AND (scope IS NOT NULL) AND (domain IS NOT NULL))))`

Indexes:

- `INDEX idx_projects_course_status ON projects USING btree (course_id, status)`
- `INDEX idx_projects_fts ON projects USING gin (to_tsvector('english'::regconfig, (((((COALESCE(title, ''::character varying))::text || ' '::text) || COALESCE(abstract, ''::text)) || ' '::text) || COALESCE(problem_statement, ''::text))))`
- `INDEX idx_projects_status ON projects USING btree (status)`
- `UNIQUE INDEX uq_projects_active_team ON projects USING btree (team_id) WHERE ((status)::text <> ALL ((ARRAY['rejected'::character varying, 'archived'::character varying])::text[]))`
- `UNIQUE INDEX uq_projects_course_title ON projects USING btree (course_id, lower((title)::text)) WHERE ((status)::text <> ALL ((ARRAY['rejected'::character varying, 'archived'::character varying])::text[]))`

Triggers: `trg_projects_audit`, `trg_projects_transition`, `trg_projects_updated`

proposal_actions

Append-only history of proposal decisions (submit, approve, reject, request revision).

Column	Type	Null	Default	Constraints and notes
action_id	integer	NO		**PK** identity (auto-generated)
project_id	integer	NO		**FK** → projects(project_id) ON DELETE RESTRICT
actor_id	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT
action	varchar(20)	NO		CHECK (((action)::text = ANY ((ARRAY['submit'::character varying, 'resubmit'::character varying, 'approve'::character varying, 'reject'::character varying, 'request_revision'::character varying])::text[])))
from_status	varchar(20)	NO		
to_status	varchar(20)	NO		
comments	text	yes		
created_at	timestamptz	NO	`now()`	

Table-level rules:

- `CHECK ((((action)::text <> ALL ((ARRAY['reject'::character varying, 'request_revision'::character varying])::text[])) OR (length(TRIM(BOTH FROM COALESCE(comments, ''::text))) > 0)))`

Triggers: `trg_proposal_actions_immutable`

review_scores

Rubric criteria and scores of a review.

Column	Type	Null	Default	Constraints and notes
review_id	integer	NO		**FK** → reviews(review_id) ON DELETE CASCADE **PK**
criterion	varchar(80)	NO		**PK**
max_score	numeric(5,2)	NO		CHECK ((max_score > (0)::numeric))
score	numeric(5,2)	NO		CHECK ((score >= (0)::numeric))

Table-level rules:

- `CHECK ((score <= max_score))`

reviews

A guide's or reviewer's evaluation of a milestone.

Column	Type	Null	Default	Constraints and notes
review_id	integer	NO		**PK** identity (auto-generated)
milestone_id	integer	NO		**FK** → milestones(milestone_id) ON DELETE RESTRICT
reviewer_id	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT
review_type	varchar(10)	NO	`'milestone'::character varying`	CHECK (((review_type)::text = ANY ((ARRAY['milestone'::character varying, 'final'::character varying])::text[])))
decision	varchar(20)	NO	`'pending'::character varying`	CHECK (((decision)::text = ANY ((ARRAY['pending'::character varying, 'approved'::character varying, 'revision_requested'::character varying, 'rejected'::character varying])::text[])))
comments	text	yes		
total_score	numeric(5,2)	yes		CHECK (((total_score IS NULL) OR ((total_score >= (0)::numeric) AND (total_score <= (100)::numeric))))
review_date	date	yes		
created_at	timestamptz	NO	`now()`	

Table-level rules:

- `CHECK ((((decision)::text = 'pending'::text) OR (review_date IS NOT NULL)))`

Indexes:

- `INDEX idx_reviews_milestone ON reviews USING btree (milestone_id)`

Triggers: `trg_reviews_audit`

risk_assessments

Historical delivery-risk snapshots with score, level, reasons and per-factor breakdown.

Column	Type	Null	Default	Constraints and notes
risk_id	integer	NO		**PK** identity (auto-generated)
project_id	integer	NO		**FK** → projects(project_id) ON DELETE RESTRICT
risk_score	numeric(5,2)	NO		CHECK (((risk_score >= (0)::numeric) AND (risk_score <= (100)::numeric)))
risk_level	varchar(10)	NO		CHECK (((risk_level)::text = ANY ((ARRAY['low'::character varying, 'moderate'::character varying, 'high'::character varying, 'critical'::character varying])::text[])))
reason	text	NO		
breakdown	jsonb	NO	`'{}'::jsonb`	
calculated_at	timestamptz	NO	`now()`	

Table-level rules:

- `CHECK (((((risk_level)::text = 'low'::text) AND (risk_score < (25)::numeric)) OR (((risk_level)::text = 'moderate'::text) AND (risk_score >= (25)::numeric) AND (risk_score < (50)::numeric)) OR (((risk_level)::text = 'high'::text) AND (risk_score >= (50)::numeric) AND (risk_score < (75)::numeric)) OR (((risk_level)::text = 'critical'::text) AND (risk_score >= (75)::numeric))))`

Indexes:

- `INDEX idx_risk_level ON risk_assessments USING btree (risk_level)`
- `INDEX idx_risk_project_time ON risk_assessments USING btree (project_id, calculated_at DESC)`

risk_events

Flags raised for overdue milestones and similar events.

Column	Type	Null	Default	Constraints and notes
event_id	integer	NO		**PK** identity (auto-generated)
project_id	integer	NO		**FK** → projects(project_id) ON DELETE RESTRICT
milestone_id	integer	yes		**FK** → milestones(milestone_id) ON DELETE SET NULL
event_type	varchar(30)	NO		CHECK (((event_type)::text = ANY ((ARRAY['milestone_overdue'::character varying, 'task_overdue'::character varying, 'feedback_unresolved'::character varying, 'inactivity'::character varying, 'missing_deliverable'::character varying, 'contribution_imbalance'::character varying])::text[])))
details	text	yes		
created_at	timestamptz	NO	`now()`	
resolved_at	timestamptz	yes		

Indexes:

- `UNIQUE INDEX uq_open_risk_event ON risk_events USING btree (project_id, event_type, COALESCE(milestone_id, 0)) WHERE (resolved_at IS NULL)`

roles

The four roles: student, faculty_guide, reviewer, admin.

Column	Type	Null	Default	Constraints and notes
role_id	integer	NO		**PK** identity (auto-generated)
role_name	varchar(20)	NO		CHECK (((role_name)::text = ANY ((ARRAY['student'::character varying, 'faculty_guide'::character varying, 'reviewer'::character varying, 'admin'::character varying])::text[]))) **UQ**
description	text	yes		

schema_migrations

Which migration files have been applied.

Column	Type	Null	Default	Constraints and notes
version	varchar(60)	NO		**PK**
applied_at	timestamptz	NO	`now()`	

similarity_reviews

Pairs of proposals flagged as possibly overlapping, with the faculty decision.

Column	Type	Null	Default	Constraints and notes
similarity_id	integer	NO		**PK** identity (auto-generated)
source_project_id	integer	NO		**FK** → projects(project_id) ON DELETE RESTRICT **UQ** with (source_project_id, matched_project_id)
matched_project_id	integer	NO		**FK** → projects(project_id) ON DELETE RESTRICT **UQ** with (source_project_id, matched_project_id)
similarity_score	numeric(5,4)	NO		CHECK (((similarity_score >= (0)::numeric) AND (similarity_score <= (1)::numeric)))
matched_section	varchar(20)	yes		
source_excerpt	text	yes		
matched_excerpt	text	yes		
faculty_decision	varchar(20)	NO	`'pending'::character varying`	CHECK (((faculty_decision)::text = ANY ((ARRAY['pending'::character varying, 'not_similar'::character varying, 'acceptable_overlap'::character varying, 'needs_revision'::character varying, 'escalated'::character varying])::text[])))

Column	Type	Null	Default	Constraints and notes
comments	text	yes		
reviewed_by	integer	yes		**FK** → users(user_id) ON DELETE SET NULL
created_at	timestamptz	NO	`now()`	
decided_at	timestamptz	yes		

Table-level rules:

- `CHECK ((source_project_id <> matched_project_id))`
- `CHECK ((((faculty_decision)::text = 'pending'::text) OR ((reviewed_by IS NOT NULL) AND (decided_at IS NOT NULL))))`

Indexes:

- `INDEX idx_similarity_decision ON similarity_reviews USING btree (faculty_decision)`

Triggers: `trg_similarity_audit`, `trg_similarity_transition`

status_transitions

Data-driven workflow: which status changes are legal for each entity type.

Column	Type	Null	Default	Constraints and notes
entity_type	varchar(30)	NO		**PK**
from_status	varchar(25)	NO		**PK**
to_status	varchar(25)	NO		**PK**

task_assignments

Task owner (exactly one) and collaborators.

Column	Type	Null	Default	Constraints and notes
task_id	integer	NO		**FK** → tasks(task_id) ON DELETE CASCADE **PK**
user_id	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT **PK**
assignment_type	varchar(12)	NO	`'owner'::character varying`	CHECK (((assignment_type)::text = ANY ((ARRAY['owner'::character varying, 'collaborator'::character varying])::text[])))
assigned_by	integer	yes		**FK** → users(user_id) ON DELETE SET NULL
assigned_at	timestamptz	NO	`now()`	

Indexes:

- `INDEX idx_task_assign_user ON task_assignments USING btree (user_id)`
- `UNIQUE INDEX uq_one_owner_per_task ON task_assignments USING btree (task_id) WHERE ((assignment_type)::text = 'owner'::text)`

Triggers: `trg_task_assign_audit`, `trg_task_assign_member`

tasks

Work items under a milestone with planned and actual dates and a todo-to-verified workflow.

Column	Type	Null	Default	Constraints and notes
task_id	integer	NO		**PK** identity (auto-generated)
milestone_id	integer	NO		**FK** → milestones(milestone_id) ON DELETE RESTRICT
title	varchar(150)	NO		
description	text	yes		

Column	Type	Null	Default	Constraints and notes
status	varchar(12)	NO	`'todo'::character varying`	CHECK (((status)::text = ANY ((ARRAY['todo'::character varying, 'in_progress'::character varying, 'blocked'::character varying, 'done'::character varying, 'verified'::character varying])::text[])))
priority	varchar(8)	NO	`'medium'::character varying`	CHECK (((priority)::text = ANY ((ARRAY['low'::character varying, 'medium'::character varying, 'high'::character varying])::text[])))
effort_hours	numeric(6,2)	NO	`1`	CHECK ((effort_hours <= (200)::numeric)) CHECK ((effort_hours > (0)::numeric))
planned_start_date	date	yes		
planned_completion_date	date	NO		
actual_completion_date	date	yes		
due_date	date	NO		
verified_by	integer	yes		**FK** → users(user_id) ON DELETE SET NULL
verified_at	timestamptz	yes		
created_by	integer	yes		**FK** → users(user_id) ON DELETE SET NULL
created_at	timestamptz	NO	`now()`	
updated_at	timestamptz	NO	`now()`	

Table-level rules:

- `CHECK (((planned_start_date IS NULL) OR (planned_completion_date >= planned_start_date)))`
- `CHECK ((due_date >= COALESCE(planned_start_date, due_date)))`
- `CHECK (((actual_completion_date IS NULL) OR ((status)::text = ANY ((ARRAY['done'::character varying, 'verified'::character varying])::text[]))))`
- `CHECK ((((status)::text <> 'verified'::text) OR ((verified_by IS NOT NULL) AND (verified_at IS NOT NULL))))`

Indexes:

- `INDEX idx_tasks_due_open ON tasks USING btree (due_date) WHERE ((status)::text <> ALL ((ARRAY['done'::character varying, 'verified'::character varying])::text[]))`
- `INDEX idx_tasks_milestone_status ON tasks USING btree (milestone_id, status)`

Triggers: `trg_task_evidence`, `trg_tasks_audit`, `trg_tasks_row_audit`, `trg_tasks_side_effects`, `trg_tasks_transition`, `trg_tasks_updated`

team_members

Student membership with a leader flag and responsibility; one team per student per course.

Column	Type	Null	Default	Constraints and notes
team_id	integer	NO		**FK** → teams(team_id, course_id) ON DELETE CASCADE **PK**
course_id	integer	NO		**FK** → teams(team_id, course_id) ON DELETE CASCADE **UQ** with (course_id, user_id)
user_id	integer	NO		**FK** → users(user_id) ON DELETE RESTRICT **PK** **UQ** with (course_id, user_id)
member_role	varchar(10)	NO	`'member'::character varying`	CHECK (((member_role)::text = ANY ((ARRAY['leader'::character varying, 'member'::character varying])::text[])))
responsibility	varchar(200)	yes		
joined_at	timestamptz	NO	`now()`	

Indexes:

- `INDEX idx_team_members_user ON team_members USING btree (user_id)`
- `UNIQUE INDEX uq_one_leader_per_team ON team_members USING btree (team_id) WHERE ((member_role)::text = 'leader'::text)`

Triggers: `trg_team_member_student`, `trg_team_members_audit`

teams

Student teams, one set per course.

Column	Type	Null	Default	Constraints and notes
team_id	integer	NO		**PK** **UQ** with (team_id, course_id) identity (auto-generated)
course_id	integer	NO		**FK** → courses(course_id) ON DELETE RESTRICT **UQ** with (course_id, team_name) **UQ** with (team_id, course_id)
team_name	varchar(80)	NO		**UQ** with (course_id, team_name)
created_by	integer	yes		**FK** → users(user_id) ON DELETE SET NULL
created_at	timestamptz	NO	`now()`	

template_milestones

Milestones of a template (offset from project start, weightage, mandatory deliverable types).

Column	Type	Null	Default	Constraints and notes
template_milestone_id	integer	NO		**PK** identity (auto-generated)
template_id	integer	NO		**FK** → project_templates(template_id) ON DELETE CASCADE **UQ** with (template_id, sequence_no)
sequence_no	integer	NO		CHECK ((sequence_no > 0)) **UQ** with (template_id, sequence_no)
title	varchar(150)	NO		
offset_days	integer	NO		CHECK ((offset_days >= 0))
weightage	numeric(5,2)	NO		CHECK (((weightage > (0)::numeric) AND (weightage <= (100)::numeric)))
required_deliverable_types	text[]	NO	`'{}'::text[]`	

user_roles

Many-to-many link between users and roles.

Column	Type	Null	Default	Constraints and notes
user_id	integer	NO		**FK** → users(user_id) ON DELETE CASCADE **PK**
role_id	integer	NO		**FK** → roles(role_id) ON DELETE RESTRICT **PK**
assigned_at	timestamptz	NO	`now()`	
assigned_by	integer	yes		**FK** → users(user_id) ON DELETE SET NULL

Triggers: `trg_user_roles_audit`

users

Login identity, account status and lockout state. Passwords exist only as bcrypt hashes.

Column	Type	Null	Default	Constraints and notes
user_id	integer	NO		**PK** identity (auto-generated)
name	varchar(120)	NO		CHECK ((length(TRIM(BOTH FROM name)) > 0))
email	citext	NO		CHECK ((email ~* '^[^@[:space:]]+@[^@[:space:]]+.[^@[:space:]]+$'::citext)) **UQ**
password_hash	text	NO		
status	varchar(12)	NO	`'active'::character varying`	CHECK (((status)::text = ANY ((ARRAY['active'::character varying, 'inactive'::character varying, 'suspended'::character varying])::text[])))
dept_id	integer	yes		**FK** → departments(dept_id) ON DELETE SET NULL

Column	Type	Null	Default	Constraints and notes
identifier	varchar(30)	yes		**UQ**
failed_login_count	integer	NO	`0`	CHECK ((failed_login_count >= 0))
locked_until	timestamptz	yes		
last_login_at	timestamptz	yes		
created_at	timestamptz	NO	`now()`	
updated_at	timestamptz	NO	`now()`	

Triggers: `trg_users_updated`

Views

- `v_contribution_summary`
- `v_faculty_risk_dashboard`
- `v_latest_risk`
- `v_missing_mandatory_deliverables`
- `v_overdue_tasks_by_team`
- `v_project_progress`
- `v_unresolved_feedback`

Functions and procedures

- function `fn_audit_row()`
- function `fn_audit_status_change()`
- function `fn_auto_version_number()`
- function `fn_block_approval_without_evidence()`
- function `fn_block_modify_immutable()`
- function `fn_calculate_project_risk(p_project_id integer)`
- function `fn_check_allocation_role()`
- function `fn_check_assignee_in_team()`
- function `fn_check_team_member_is_student()`
- function `fn_check_weightage_total()`
- function `fn_contribution_indicator(p_project_id integer, p_user_id integer)`
- function `fn_documentation_completeness(p_project_id integer)`
- function `fn_evidence_from_docversion()`
- function `fn_evidence_from_feedback()`
- function `fn_evidence_from_task()`
- function `fn_flag_overdue_milestones()`
- function `fn_milestone_completion_pct(p_milestone_id integer)`
- function `fn_milestone_overdue_event()`
- function `fn_set_updated_at()`
- function `fn_task_side_effects()`
- function `fn_validate_transition()`
- procedure `sp_recalculate_risk(IN p_project_id integer)`
- procedure `sp_run_risk_sweep()`

Appendix B: how to run the project

```
cp .env.example .env            # fill in real values locally (never commit .env)
npm install
npm run db:migrate              # applies db/migrations/*.sql with a progress bar
npm run db:seed                 # demo data in PostgreSQL + MongoDB + embeddings (first run downloads the model)
npm run dev                     # http://localhost:3000
npm test                        # database + unit tests (API tests run when the app is up)
ALLOW_MUTATING_TESTS=1 npm test # also the workflow tests; the demo is restored afterwards
npm run risk:sweep              # store a fresh risk snapshot for every running project
npm run docs:report             # rebuild report.pdf from docs/REPORT.md
```

Demo accounts (password = `SEED_DEFAULT_PASSWORD` in your `.env`): `admin@projectpulse.test`, `guide1@` to `guide3@`, `reviewer1@`, `reviewer2@`, `student01@` to `student14@projectpulse.test`.